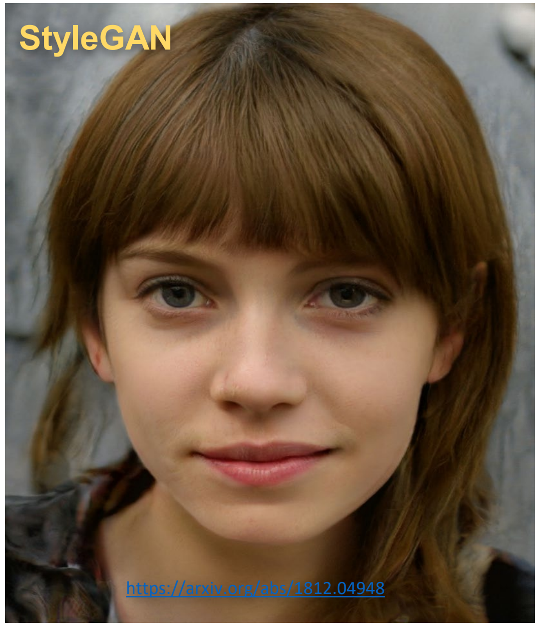
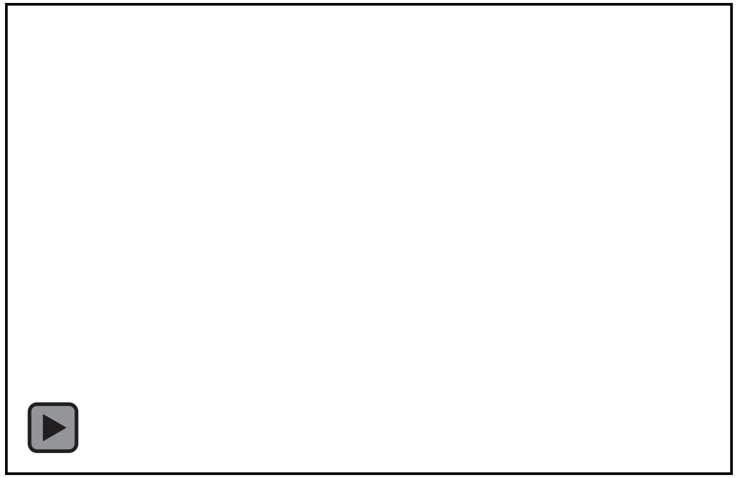
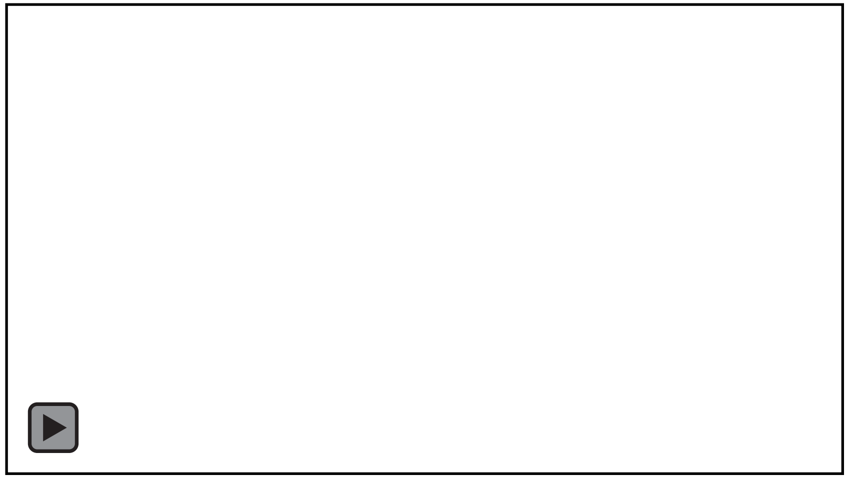
Generative models



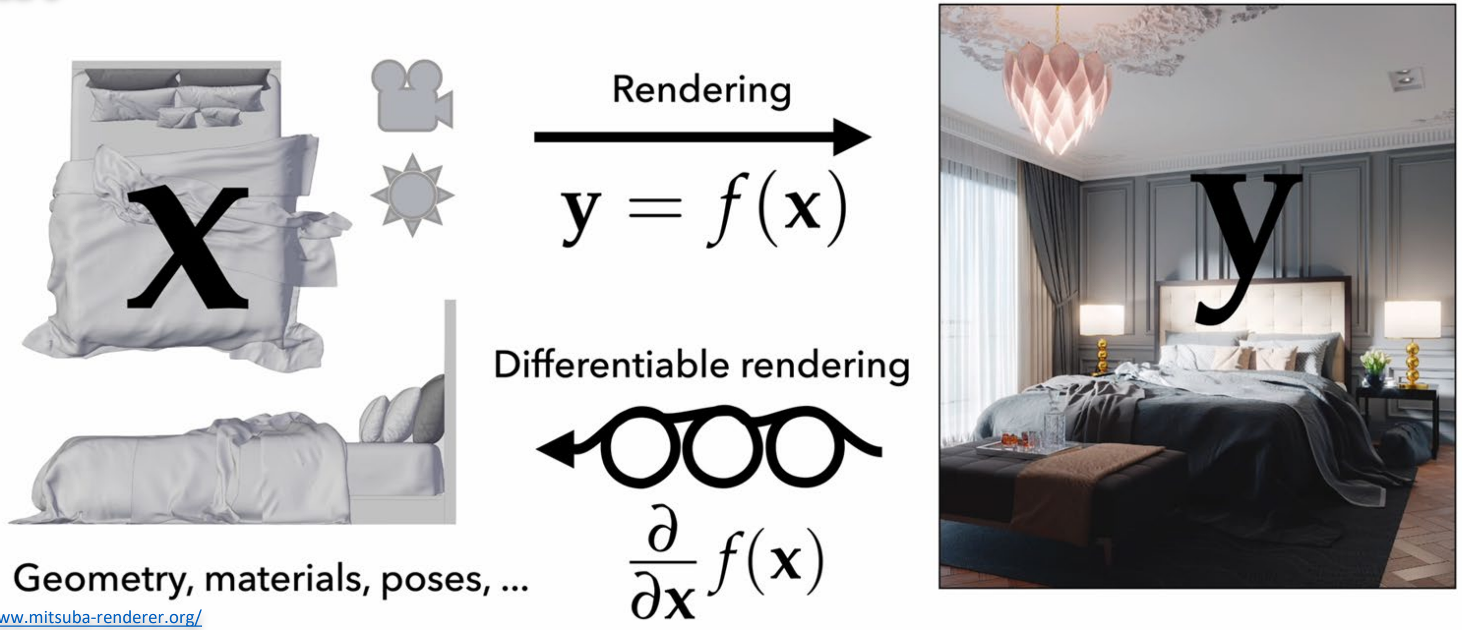
Neural rendering





Differentiable forward models for inverse rendering

Mitsuba 3



https://www.mitsuba-renderer.org/

GENERATIVE METHODS

Applications of Deep Learning – Spring 2024

Rubén Martínez-Cantín, Julio Marco

Master in Robotics, Graphics and Computer Vision – Universidad de Zaragoza

Slide credits

Many of these slides are based on content of the following courses and tutorials:

- **"CNNs for Visual Recognition", CS231n, Stanford University**
- "Deep Generative Methods", CS236, Stanford University
- "Neural rendering", CVPR 2020 Tutorial
- Generative Adversarial Networks, NIPS 2016 Tutorial
- Deep Generative Modeling, 6.S191, MIT

Please refer to these courses and tutorials for further information about today's topic.

Motivation

X

Motivation

What would you like to do?

X

Motivation

Infer information…

- Class of the image (e.g. dog)
- Breed, age, gender

What would you like to do?

X

Motivation

Infer information…

- Class of the image (e.g. dog)
- Breed, age, gender

p(Y = “golden retriever” | X = )

What would you like to do?

X

Motivation

Infer information…

- Class of the image (e.g. dog)
- Breed, age, gender

or maybe…

- New dog images? Random or not?
- Tweakable, meaningful features?
    - Breed, fur, age
- Change pose, haircut, crossbreeding, lighting, environment

What would you like to do?

X

Motivation

Infer information…

- Class of the image (e.g. dog)
- Breed, age, gender

or maybe…

- New dog images? Random or not?
- Tweakable, meaningful features?
    - Breed, fur, age
- Change pose, haircut, crossbreeding, lighting, environment

What would you like to do?

$\sim p(X)$

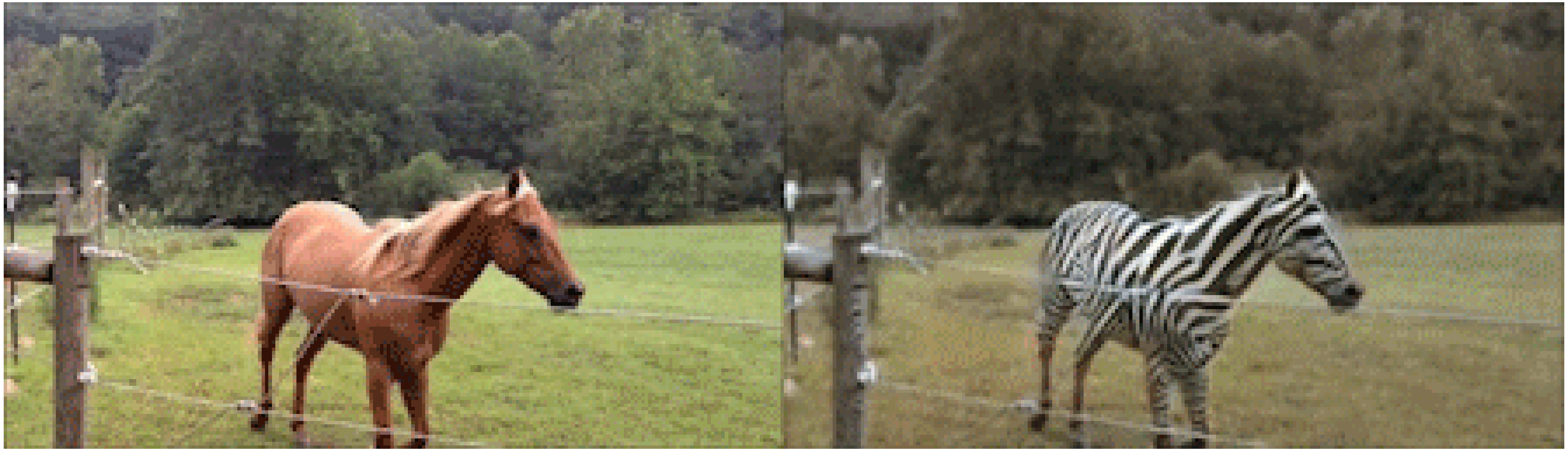
Motivation: Progress in generative models

Motivation: Progress in generative models

Song et al., Score-Based Generative Modeling through Stochastic Differential Equations, 2021

Motivation: Progress in generative models

Image translation:
Conditional generative model P(zebra images| horse images)



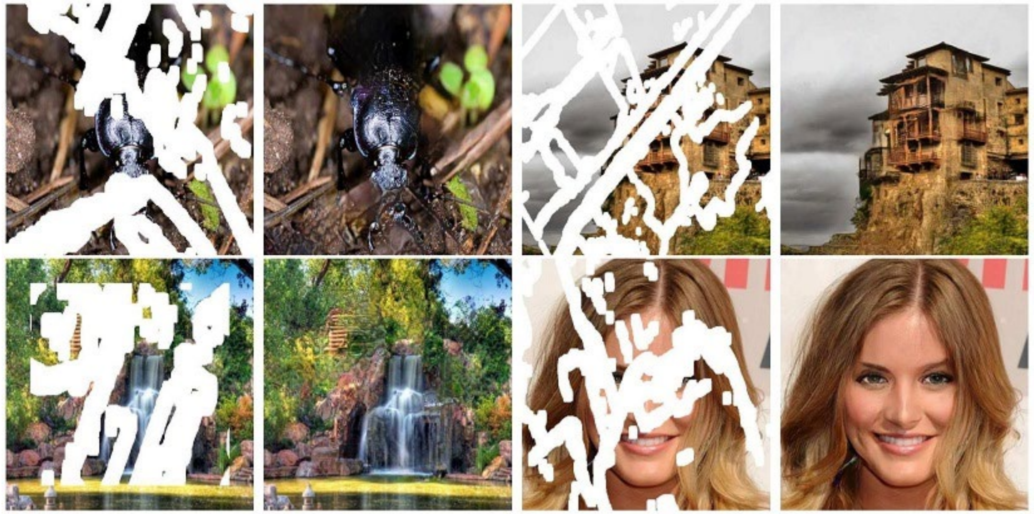
Zhu et al., 2017

Motivation: Progress in inverse problems

P(high resolution | low resolution)

Menon et al, 2020

P(full image| mask)



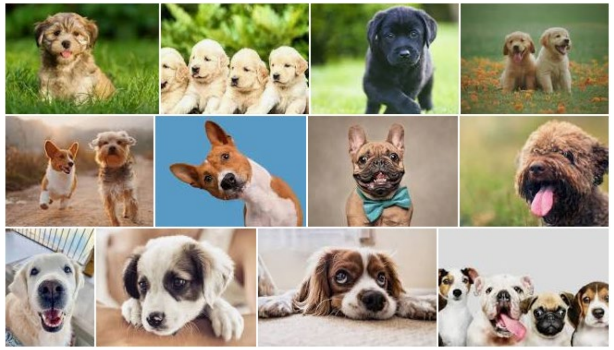
Liu al, 2018

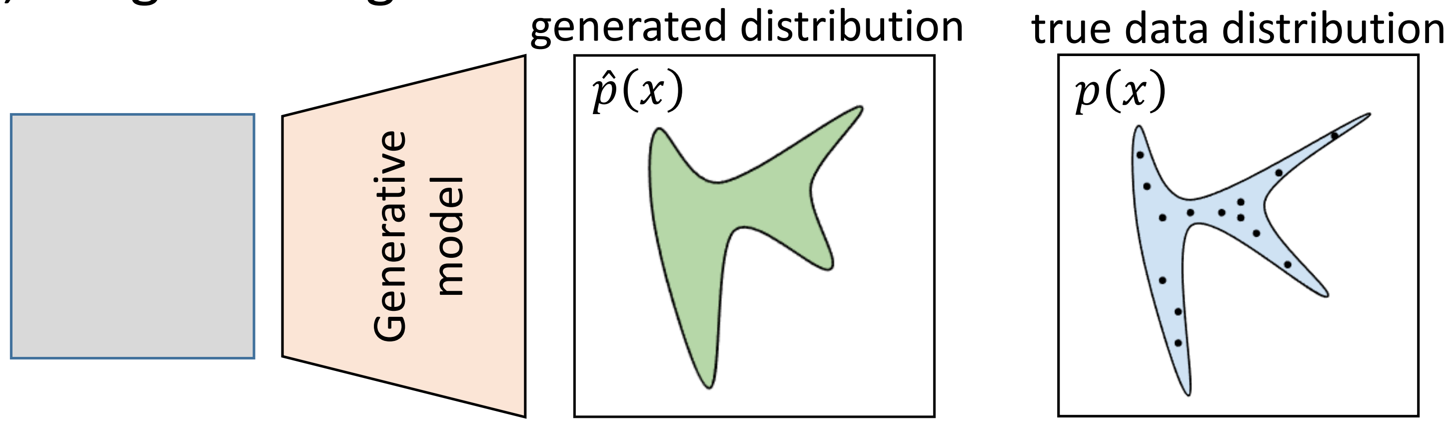
P(color image| greyscale)

Antic, 2020

Learning a generative model

Input: training set of examples, e.g., images of dogs





We want to learn a probability distribution $p(x)$ over the set of images such that:

- **Generation**: If we sample a new image $x_{new} \sim p(x)$, it should look like a dog (sampling)
- **Density estimation**: $p(x)$ should be high if (x) looks like a dog
- **Unsupervised representation learning**: If we learn this distribution, we also learn what these images have in common (ears, tail…)

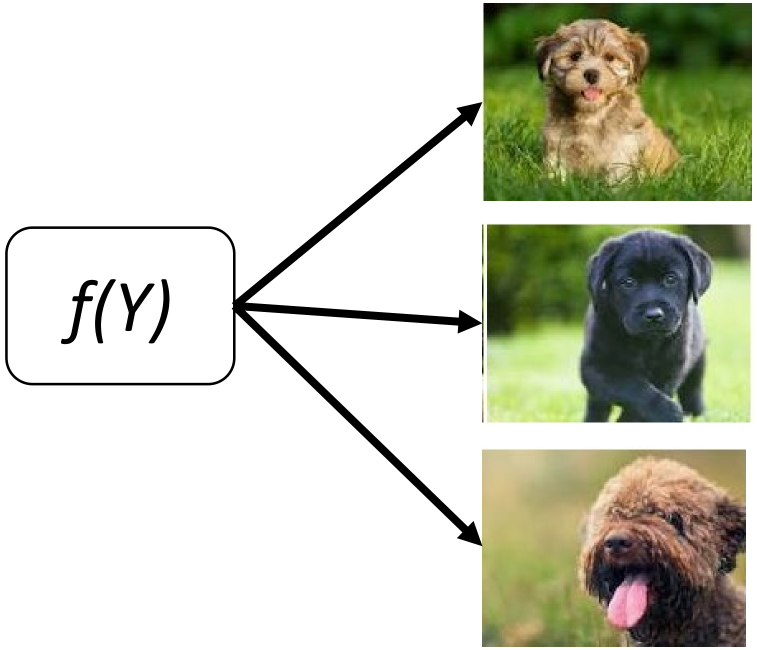
Key challenges

- **Representation**: how do we model the joint distribution of many random variables?
  - Need compact representation
- **Learning**: what is the right way to compare probability distributions?
- **Inference**: how do we invert the generation process (e.g., vision as inverse graphics)?
  - Unsupervised learning: recover high-level descriptions (features) from raw data

Challenges

How to define f(Y)?

- *Representative of data*
- *Computationally tractable*
- *Meaningful parameters*
- *Intuitive control*



Generative modeling: generation vs inference

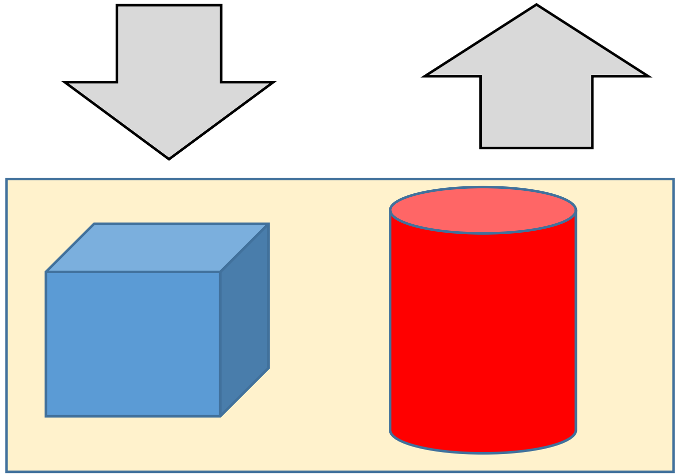
How to generate natural images with a computer?

High level description
Meaningful features

Cube(color=blue, position=, size=, …)
Cylinder(color=red, position=, size=,..)

Generation (graphics)

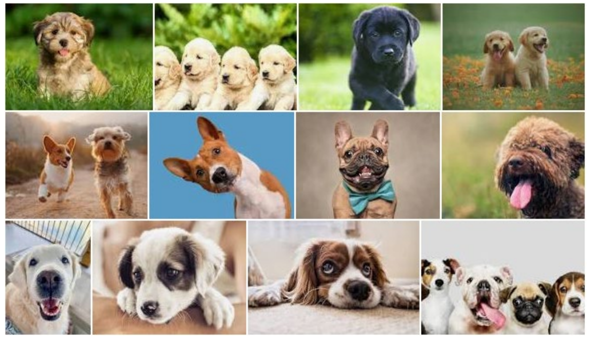
Inference (vision)



Raw sensory outputs

Our models will have **similar structure (generation + inference)**

Generative modeling



Data
(e.g., images of dogs)

+

Prior Knowledge
(e.g., physics, materials, ..)

Priors are useful in generative models

To what extent?

Generative modeling

Data

VAEs, GANs

Statistical modeling

NEURAL RENDERING

PHYSICALLY BASED RENDERING

Computer graphics

Prior Knowledge

(e.g., images of dogs)

(physics, materials, …)

Generative modeling

(e.g., images of dogs)

Data — Statistical modeling → Computer graphics — Prior Knowledge

VAEs, GANs

NEURAL RENDERING

PHYSICALLY BASED RENDERING

(physics, materials, …)

Pros:
- Fully automated training
- Good performance (real-time, interactive)

Cons:
- Requires lots of training data
- No guaranteed intuitive or semantic control of results*

Generative modeling

(e.g., images of dogs)

VAEs, GANs

NEURAL RENDERING

PHYSICALLY BASED RENDERING

Data

Prior Knowledge

(physics, materials, …)

Statistical modeling → **Computer graphics**

Statistical modeling

Pros:
- Fully automated training
- Good performance (real-time, interactive)

Cons:
- Requires lots of training data
- No guaranteed intuitive or semantic control of results*

Computer graphics

Pros:
- Full control of scene parameters (cameras, geometry, lighting)
- Faithful representation of real world

Cons:
- Lots of manual work (scene modeling, animation)
- Long generation (render) times

Generative modeling

Data

(e.g., images of dogs)

VAEs, GANs

NEURAL RENDERING

Differentiable?

PHYSICALLY BASED RENDERING

Statistical modeling

Computer graphics

Prior Knowledge

(physics, materials, ...)

Pros:
- Fully automated training
- Good performance (real-time, interactive)

Cons:
- Requires lots of training data
- No guaranteed intuitive or semantic control of results*

Pros:
- Full control of scene parameters (cameras, geometry, lighting)
- Faithful representation of real world

Cons:
- Lots of manual work (scene modeling, animation)
- Long generation (render) times

Statistical generative methods

Generative modeling

New dog images? Random or not?

Available data x follows some distribution $p_{data}(x)$ → Unknown!

Generative modeling

New dog images? Random or not?

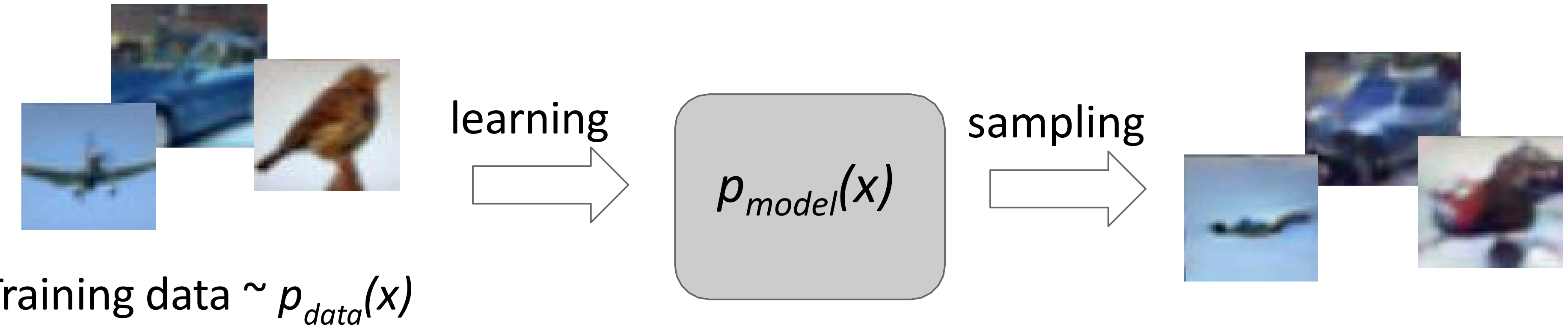
Common elements:

- Head & body shape
- Facial structure
- Snout, eyes, ears
- Fur color gamut
- Etc.

Available data x follows some distribution $p_{data}(x)$ → Unknown!

Generative modeling

Given training data, generate new samples from same distribution

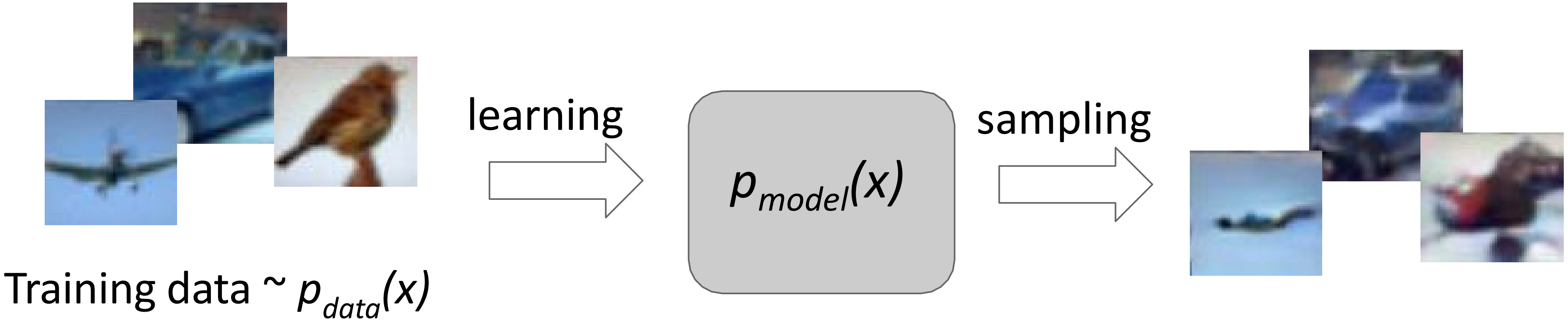


Objectives:

1. Learn $p_{model}(x)$ that approximates $p_{data}(x)$
2. **Sampling new x from $p_{model}(x)$**

Generative modeling

Given training data, generate new samples from same distribution

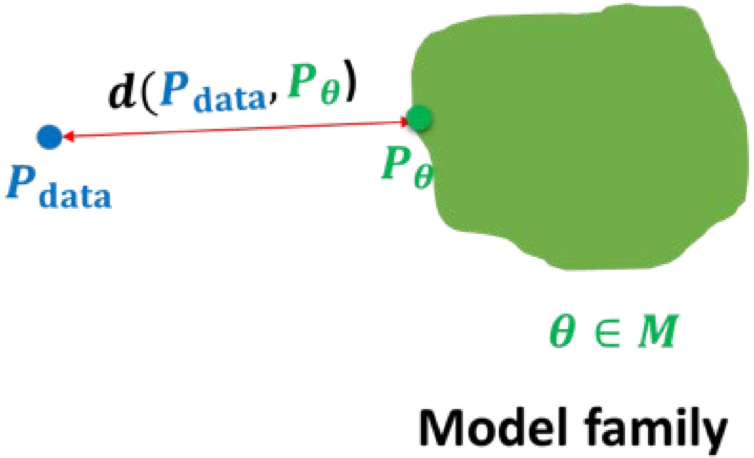


Formulate as density estimation problems:

- **Explicit density estimation**: explicitly define and solve for $p_{model}(x)$
- **Implicit density estimation**: learn model that can sample from $p_{model}(x)$ **without explicitly defining it.**

Generative modeling

$\mathbf{x}_i \sim P_{\text{data}}$
$i = 1, 2, \ldots, n$



We want to learn a probability distribution $p(x)$ over images x such that

- **Density estimation:** $p(x)$ should be high if x looks like a dog, and low otherwise (*anomaly detection*)
- **Generation:** If we sample $x_{new} \sim p(x)$, x_{new} should look like a dog (*sampling*)
- **Unsupervised representation learning:** We should be able to learn what these images have in common, e.g., ears, tail, etc. (*features*)

Taxonomy of Generative Models

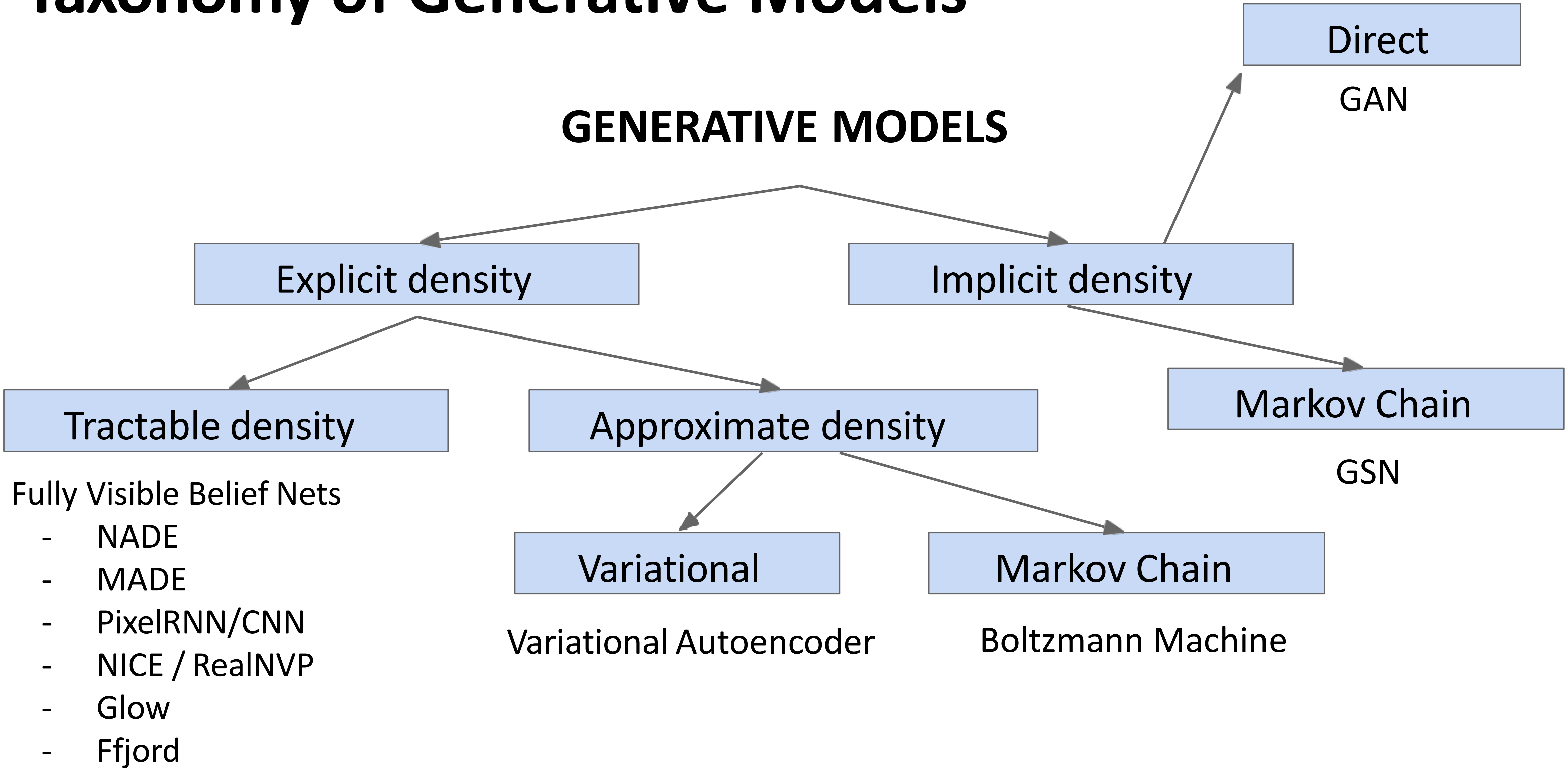


Figure copyright and adapted from Ian Goodfellow, Tutorial on Generative Adversarial Networks, 2017.

Taxonomy of Generative Models

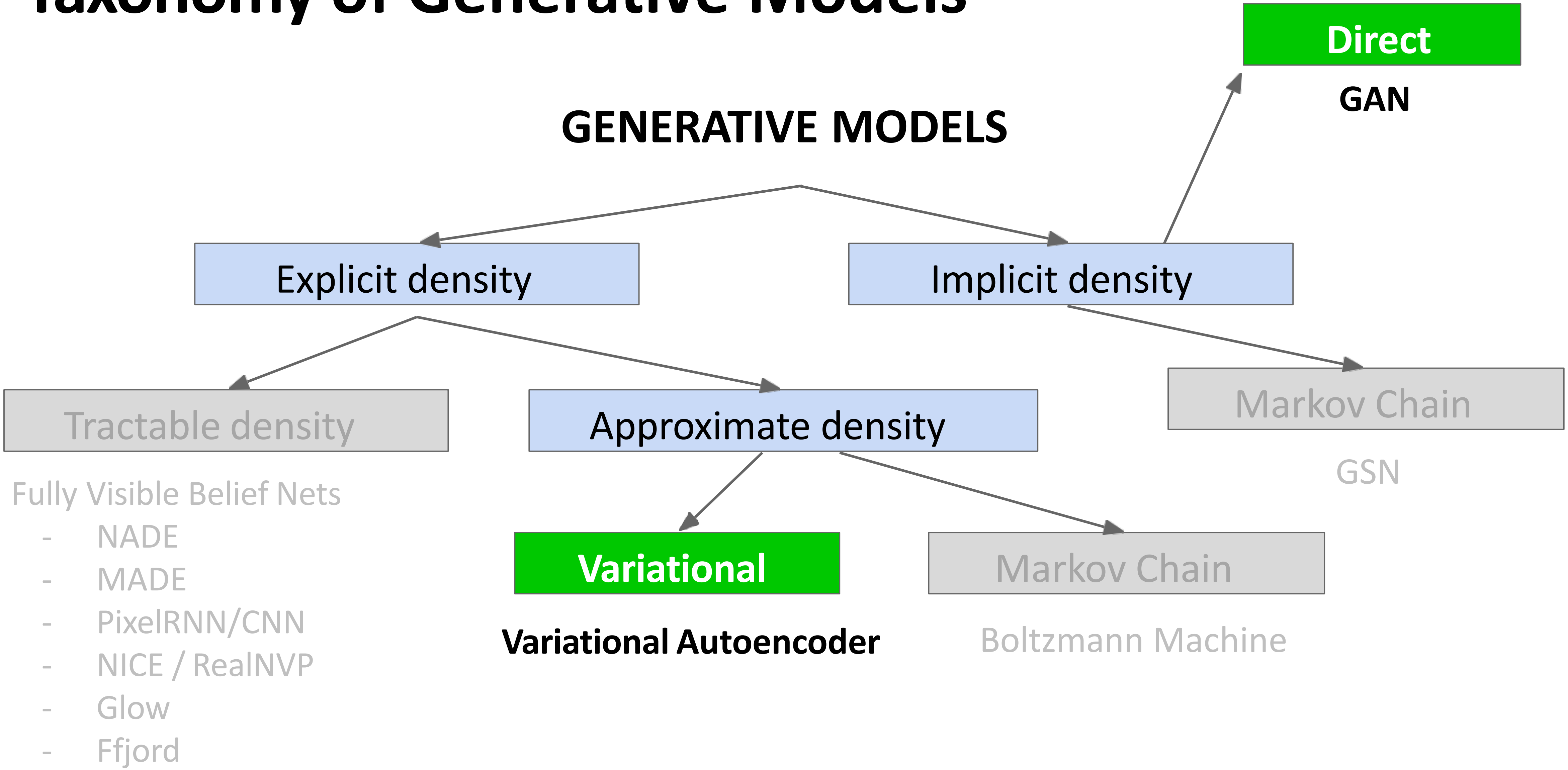


Figure copyright and adapted from Ian Goodfellow, Tutorial on Generative Adversarial Networks, 2017.

Questions?

Statistical generative methods: **Variational autoencoders**

Supervised vs Unsupervised Learning

Supervised Learning

Data: (x, y)
x is data, y is label

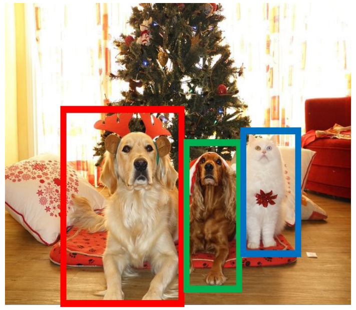
Goal: Learn a *function* to map x -> y

Examples: Classification, regression, object detection, semantic segmentation, image captioning, etc.

Classification

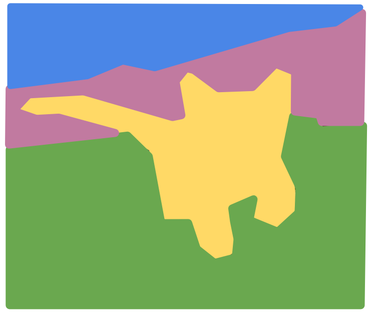
CAT

Object Detection



DOG
DOG
CAT

Semantic Segmentation



GRASS
CAT
TREE
SKY

Supervised vs Unsupervised Learning

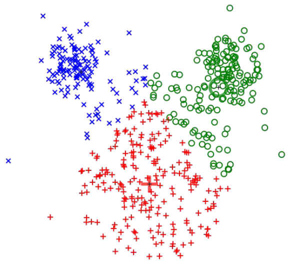
Unsupervised Learning

Data: x
Just data, no labels!

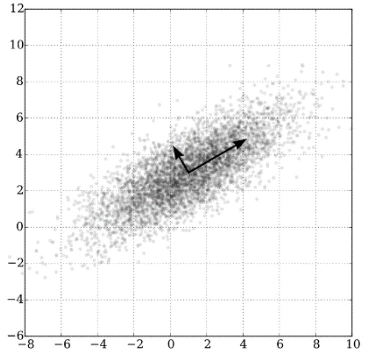
Goal: Learn some underlying hidden *structure* of the data

Examples: Clustering, dimensionality reduction, density estimation, etc.

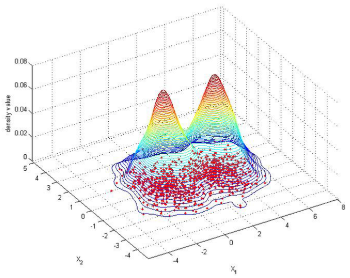
K-means clustering

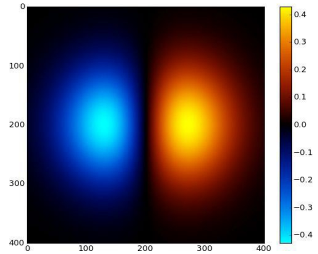


Principal Component Analysis (dim. reduction)



Density estimation





Supervised vs Unsupervised Learning

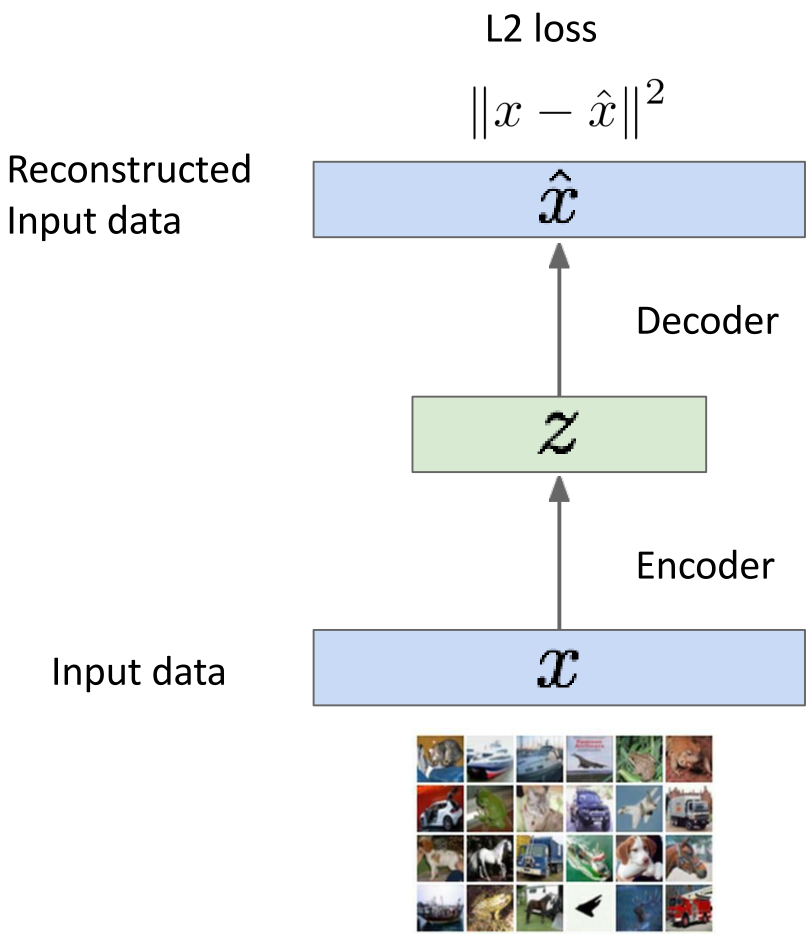
Unsupervised Learning

Data: x
Just data, no labels!

Goal: Learn some underlying hidden *structure* of the data

Examples: Clustering, dimensionality reduction, density estimation, etc.

Autoencoders



Taxonomy of Generative Models

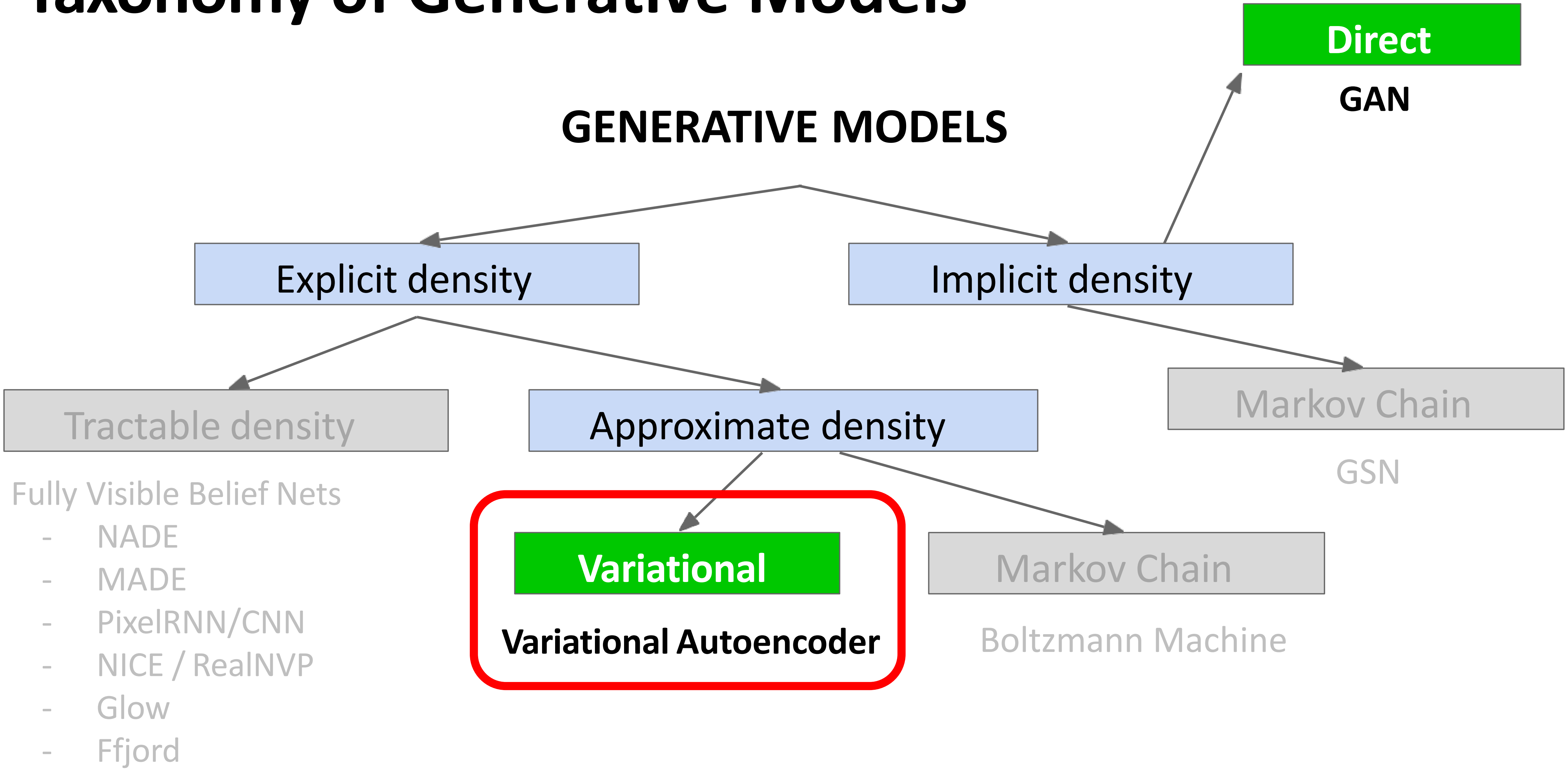


Figure copyright and adapted from Ian Goodfellow, Tutorial on Generative Adversarial Networks, 2017.

Variational Autoencoders

- New dog images? Random or not?
- **Structured content, features:**
  - Head & body shape
  - Facial structure
  - Snout, eyes, ears
  - Fur color gamut
  - Etc.

Available data x follows some distribution $p_{data}(x)$ → Unknown!

Variational Autoencoders

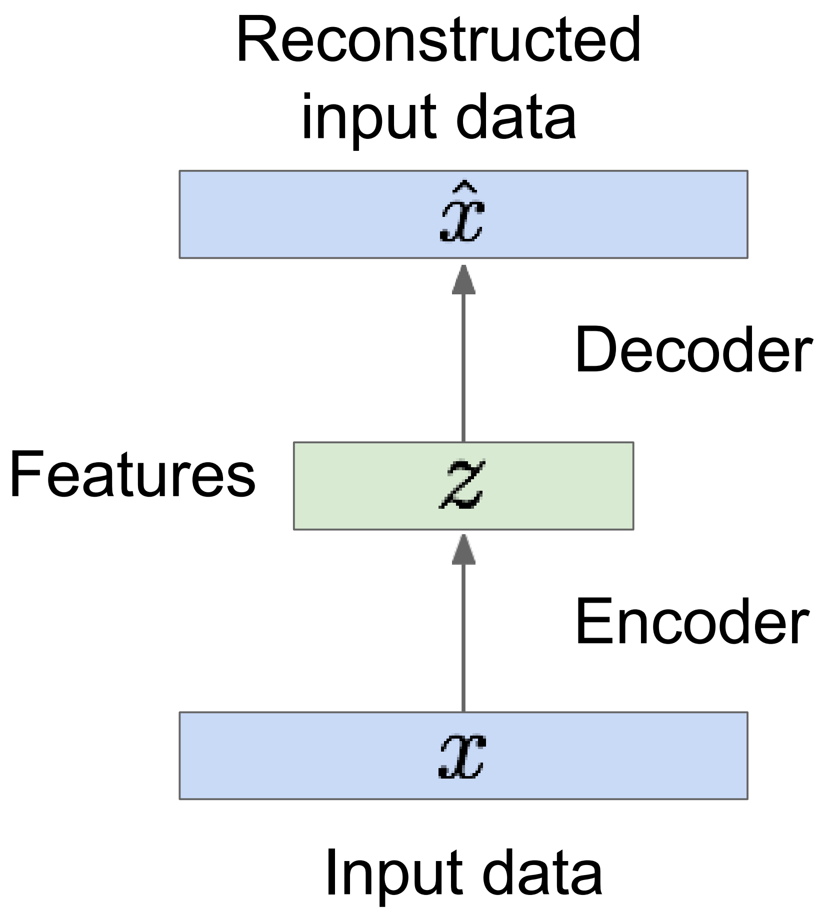
- New dog images? Random or not?
- **Structured content, features:**
  - Head & body shape
  - Facial structure
  - Snout, eyes, ears
  - Fur color gamut
  - Etc.

LATENT VARIABLES

Available data x follows some distribution $p_{data}(x)$ → Unknown!

Autoencoders

Unsupervised approach for learning a lower-dimensional feature representation from unlabeled training data

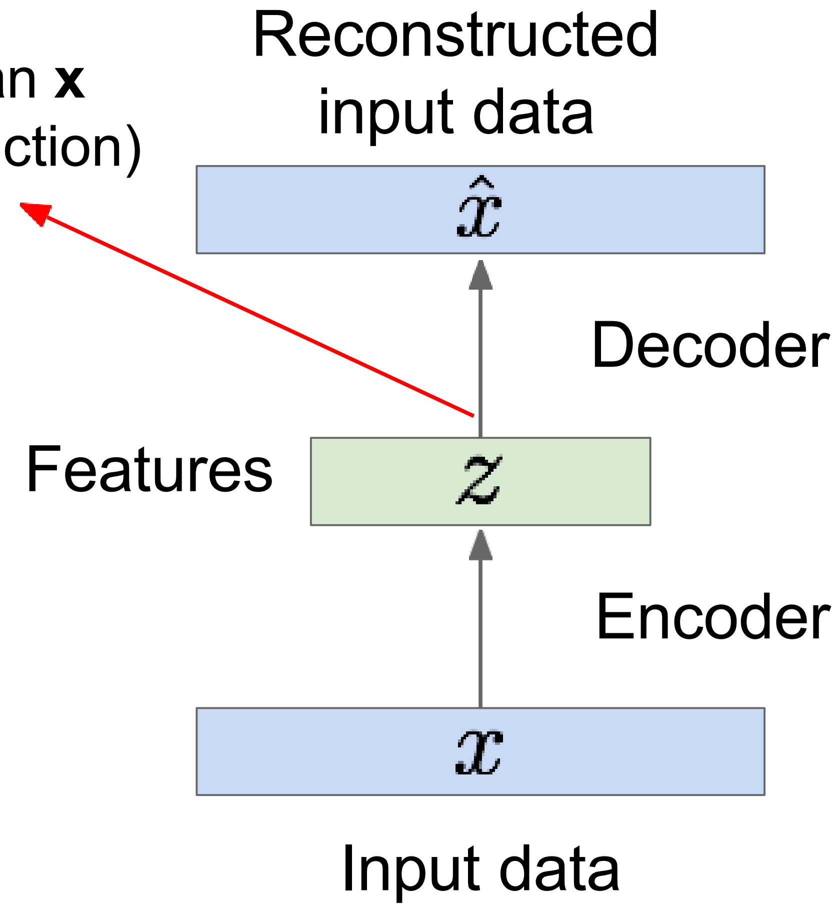


Autoencoders

Unsupervised approach for learning a lower-dimensional feature representation from unlabeled training data

z usually smaller than **x**
(dimensionality reduction)

Q: Why dimensionality reduction?



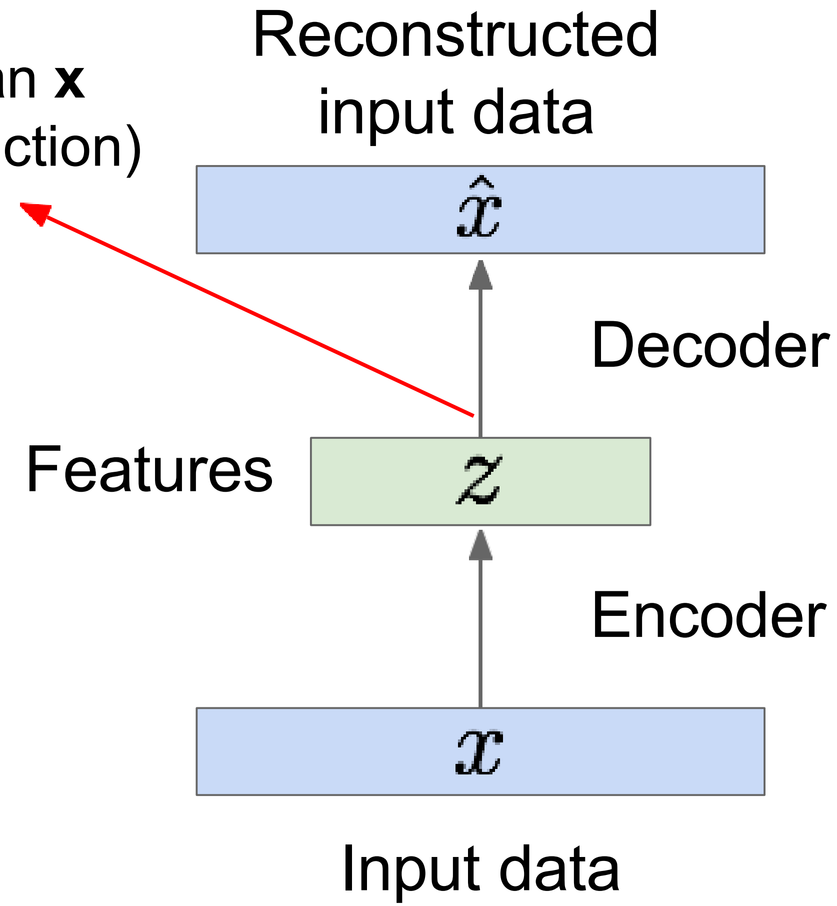
Autoencoders

Unsupervised approach for learning a lower-dimensional feature representation from unlabeled training data

z usually smaller than **x** (dimensionality reduction)

Q: Why dimensionality reduction?

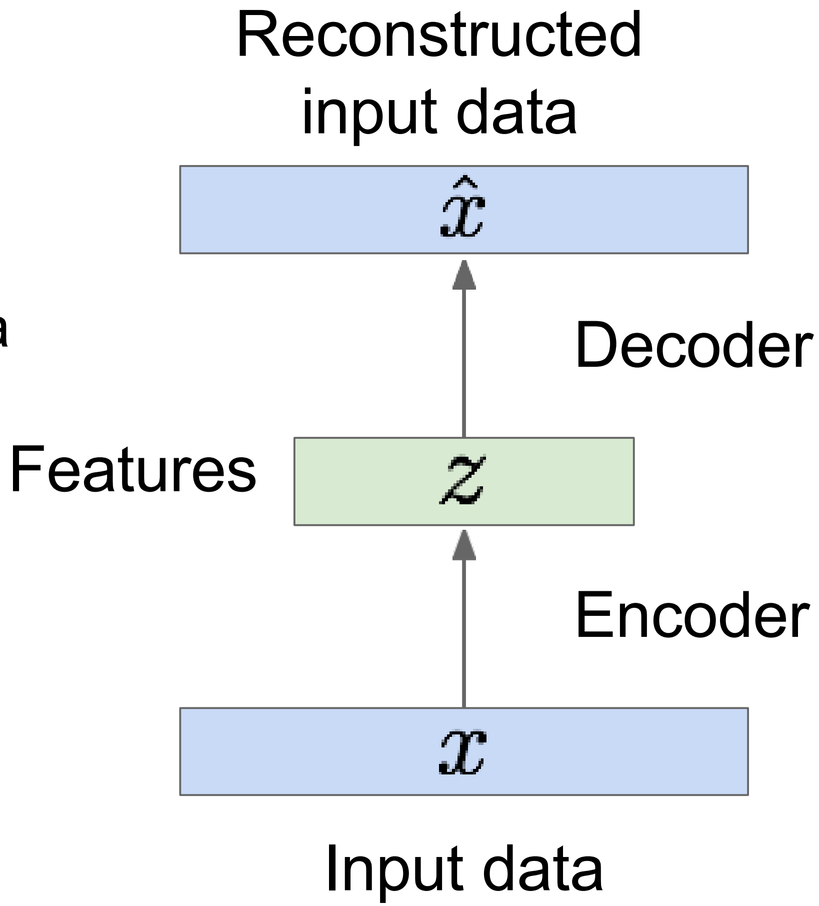
A: Want features to capture meaningful factors of variation in data

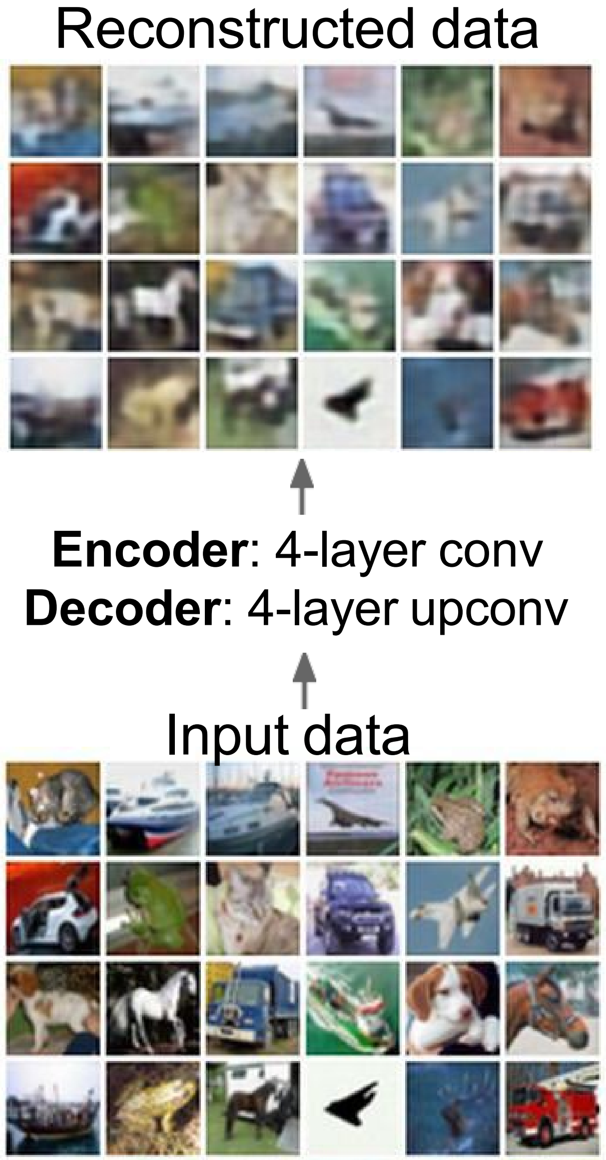


Autoencoders

How to learn this feature representation?

Train such that features can be used to reconstruct original data "Autoencoding" - encoding input itself





Autoencoders

Train such that features can be used to reconstruct original data

Doesn't use labels!

L2 Loss function:

$$\|x - \hat{x}\|^2$$

$\hat{x}$

Decoder

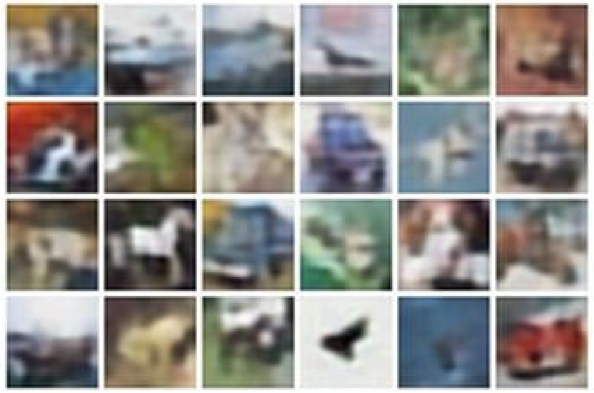
Features z

Encoder

x

Input data

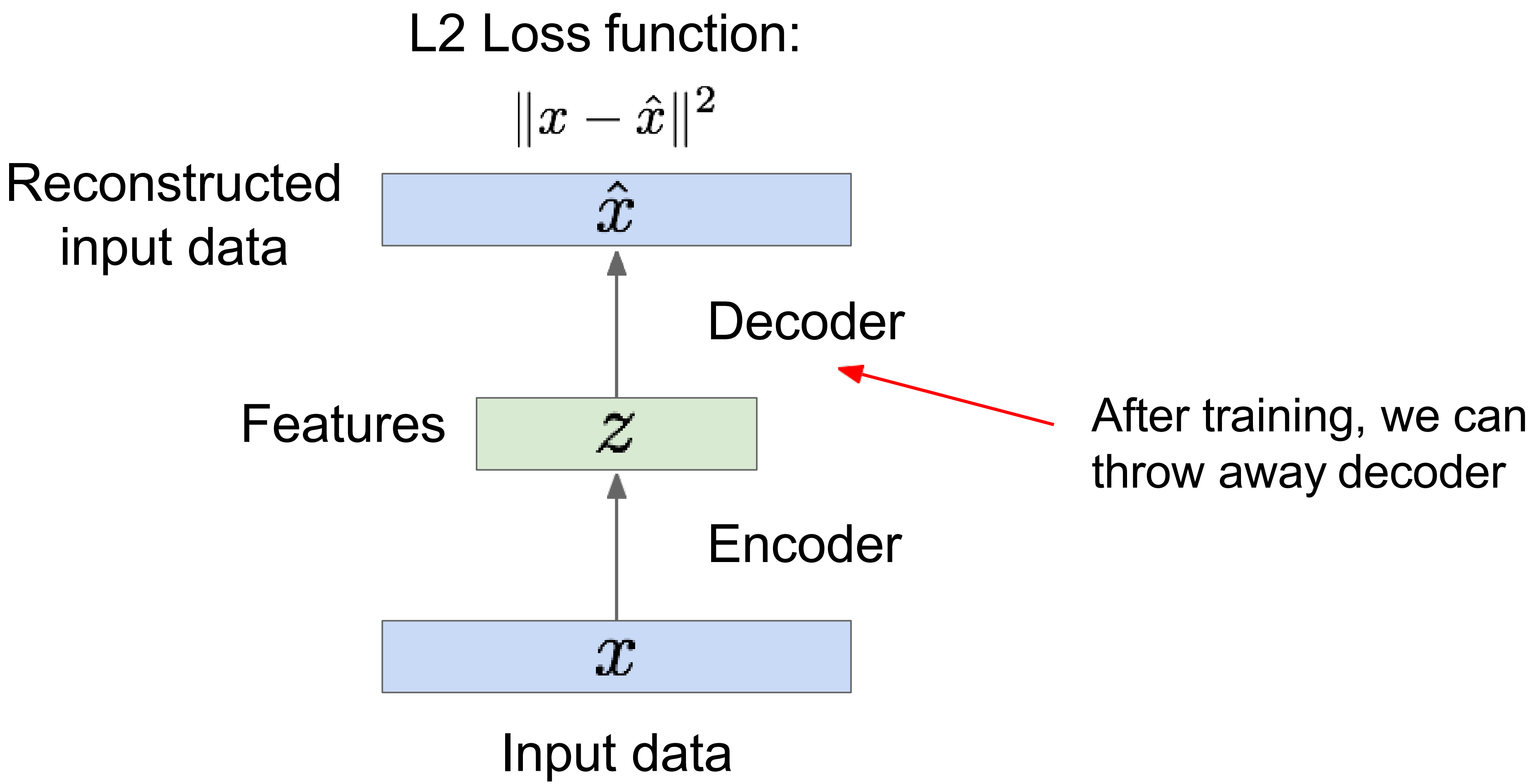
Reconstructed data



Encoder: 4-layer conv
Decoder: 4-layer upconv

Input data

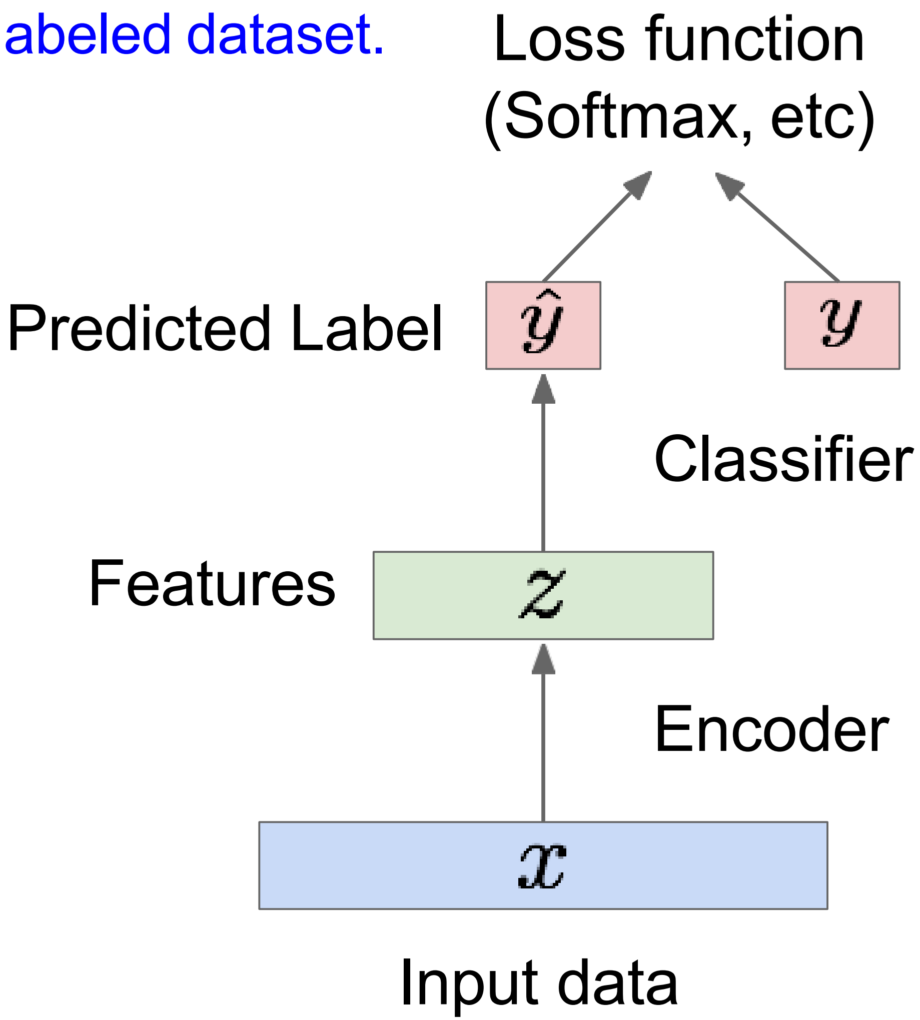
Autoencoders



Autoencoders

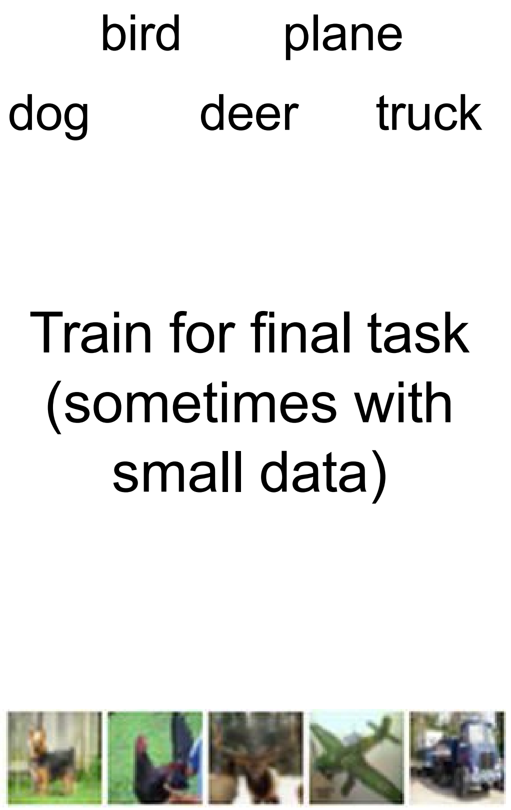
Transfer from large, unlabeled dataset to small, labeled dataset.

Encoder can be used to initialize a **supervised** model

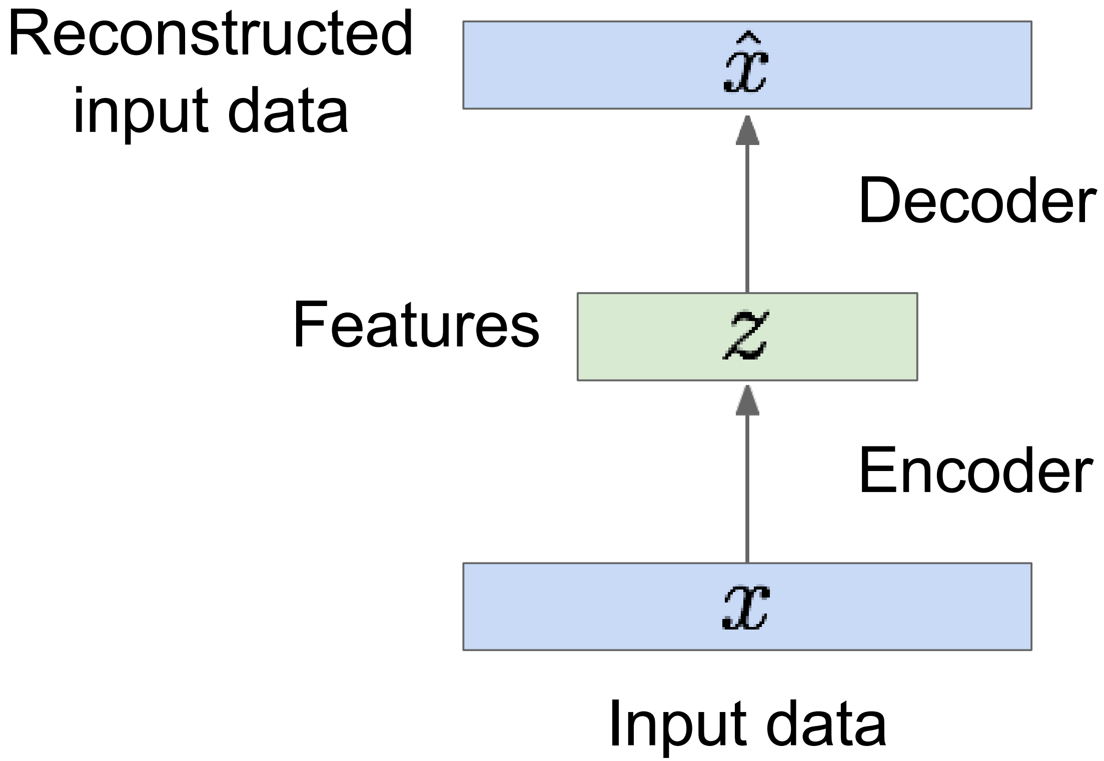


Fine-tune encoder jointly with classifier

Train for final task (sometimes with small data)



Autoencoders



Autoencoders can reconstruct data, and can learn features to initialize a supervised model

Features capture factors of variation in training data.

But we can't generate new images from an autoencoder because we don't know the space of z.

How do we make autoencoder a **generative model**?

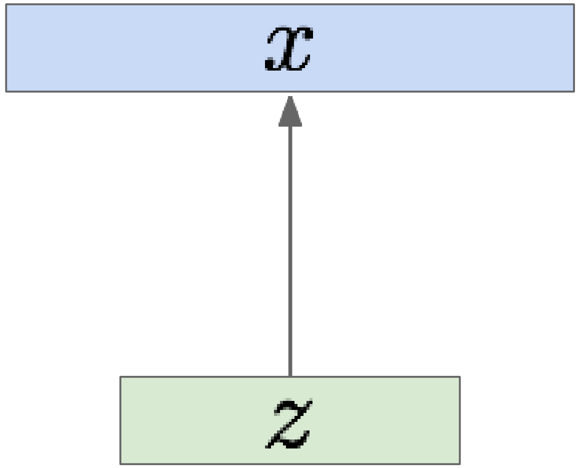
Variational Autoencoders

Probabilistic spin on autoencoders - will let us sample from the model to generate data!

Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from the distribution of unobserved (latent) representation **z**

Sample from true conditional

$p_{\theta^*}(x \mid z^{(i)})$



Sample from true prior

$z^{(i)} \sim p_{\theta^*}(z)$

Variational Autoencoders

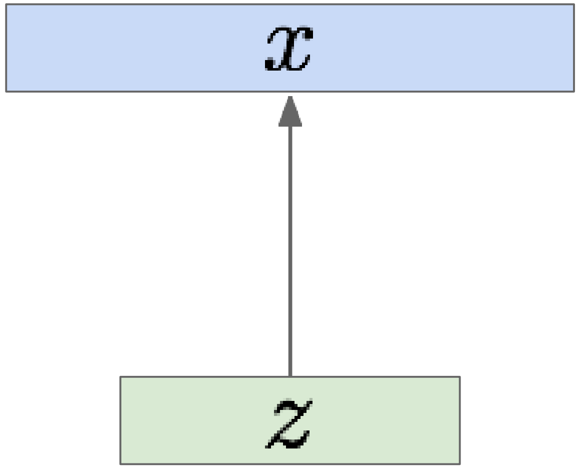
Probabilistic spin on autoencoders - will let us sample from the model to generate data!

Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from the distribution of unobserved (latent) representation **z**

Intuition (similar to autoencoders):
x is an image, **z** are latent variables used to generate **x:**

Sample from true conditional

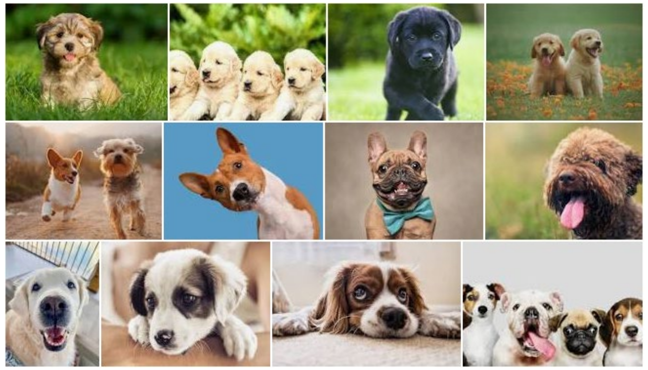
$p_{\theta^*}(x \mid z^{(i)})$



Sample from true prior

$z^{(i)} \sim p_{\theta^*}(z)$

- Body shape
- Facial structure
- Snout, eyes, ears
- Fur color gamut
- Etc.

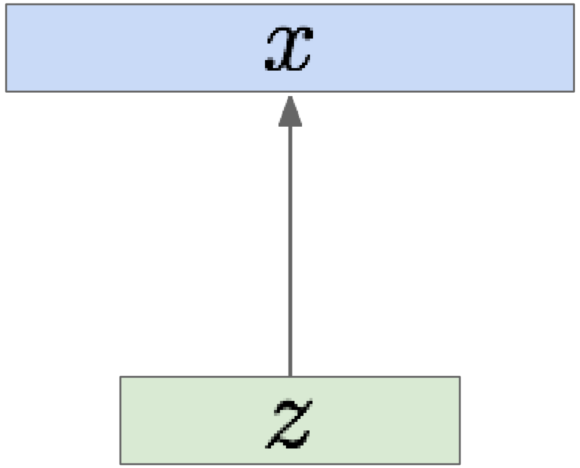


Variational Autoencoders

We want to estimate the true parameters θ^* of this generative model given training data x.

Sample from true conditional

$$p_{\theta^*}(x \mid z^{(i)})$$



Sample from true prior

$$z^{(i)} \sim p_{\theta^*}(z)$$

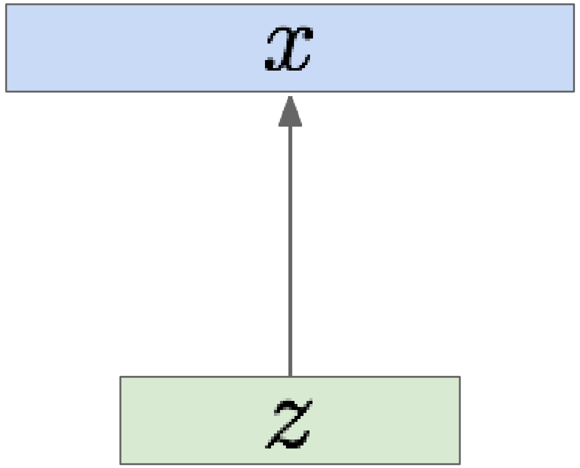
Kingma and Welling, "Auto-Encoding Variational Bayes", ICLR 2014

Variational Autoencoders

We want to estimate the true parameters θ^* of this generative model given training data x.

Sample from true conditional

$$p_{\theta^*}(x \mid z^{(i)})$$



Sample from true prior

$$z^{(i)} \sim p_{\theta^*}(z)$$

How should we represent this model?

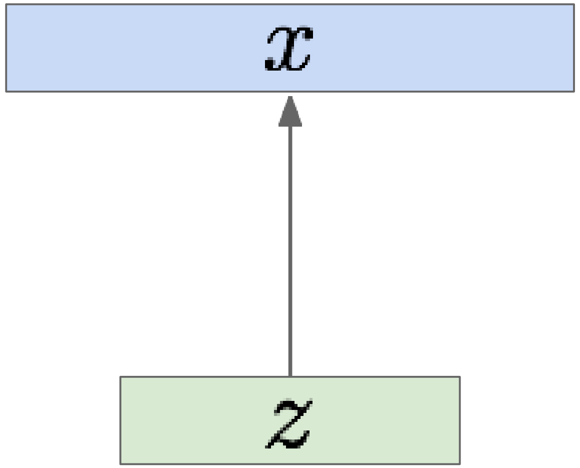
Kingma and Welling, "Auto-Encoding Variational Bayes", ICLR 2014

Variational Autoencoders

We want to estimate the true parameters θ^* of this generative model given training data x.

Sample from true conditional

$p_{\theta^*}(x \mid z^{(i)})$



Sample from true prior

$z^{(i)} \sim p_{\theta^*}(z)$

How should we represent this model?

Choose prior p(z) to be simple, e.g. Gaussian. Reasonable for latent attributes, e.g. pose, how much smile.

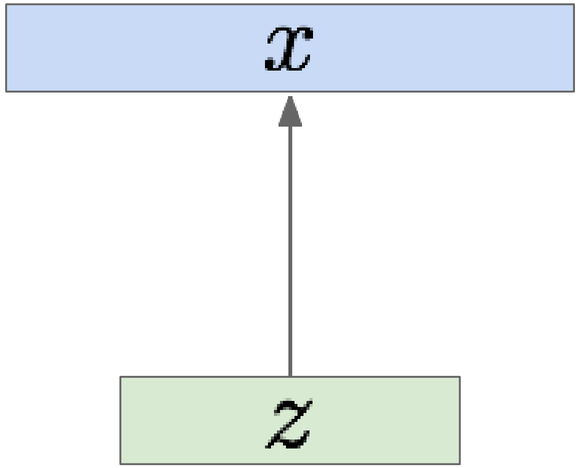
Kingma and Welling, “Auto-Encoding Variational Bayes”, ICLR 2014

Variational Autoencoders

We want to estimate the true parameters θ^* of this generative model given training data x.

Sample from true conditional

$$p_{\theta^*}(x \mid z^{(i)})$$



Sample from true prior

$$z^{(i)} \sim p_{\theta^*}(z)$$

$$z|x \sim \mathcal{N}(\mu_{z|x}, \sigma_{z|x})$$

(reasonable for latent attributes, e.g. pose, facial expression)

How should we represent this model?

- Choose prior p(z) to be simple, e.g. Gaussian.

Kingma and Welling, “Auto-Encoding Variational Bayes”, ICLR 2014

Variational Autoencoders

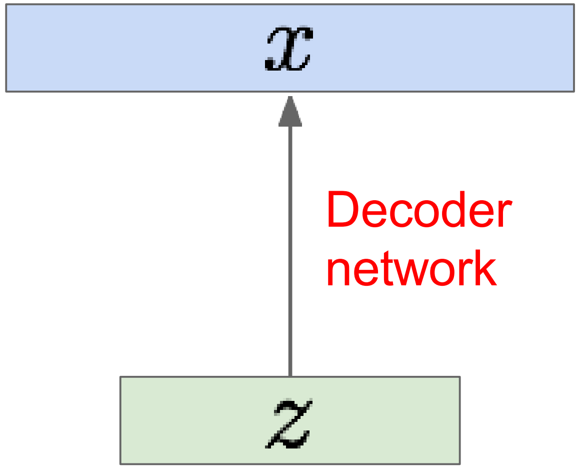
We want to estimate the true parameters θ^* of this generative model given training data x.

Sample from true conditional

$$p_{\theta^*}(x \mid z^{(i)})$$

Sample from true prior

$$z^{(i)} \sim p_{\theta^*}(z)$$



$$z|x \sim \mathcal{N}(\mu_{z|x}, \sigma_{z|x})$$

(reasonable for latent attributes, e.g. pose, facial expression)

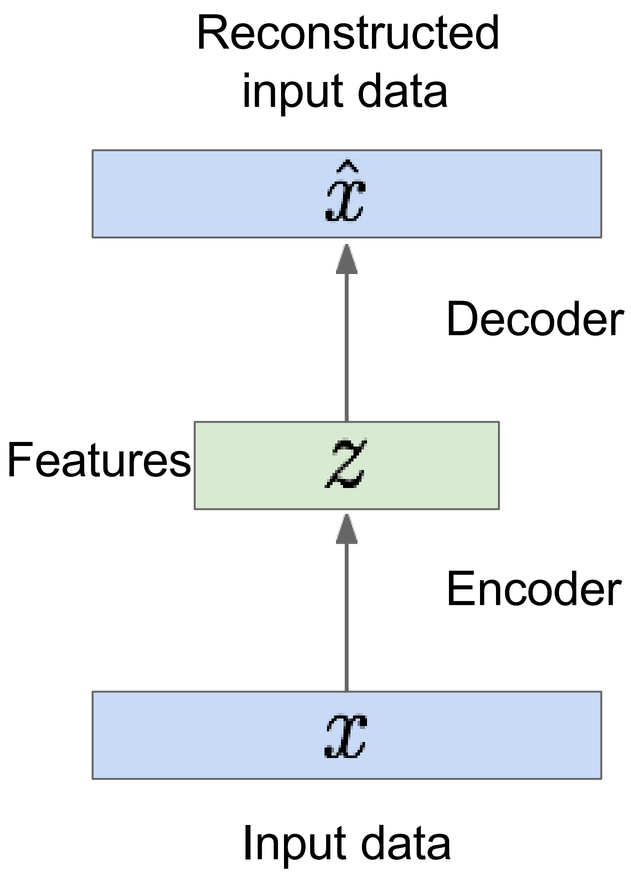
How should we represent this model?

- Choose prior p(z) to be simple, e.g. Gaussian.
- Conditional p(x|z) is complex (generates image) => represent with neural network

Kingma and Welling, “Auto-Encoding Variational Bayes”, ICLR 2014

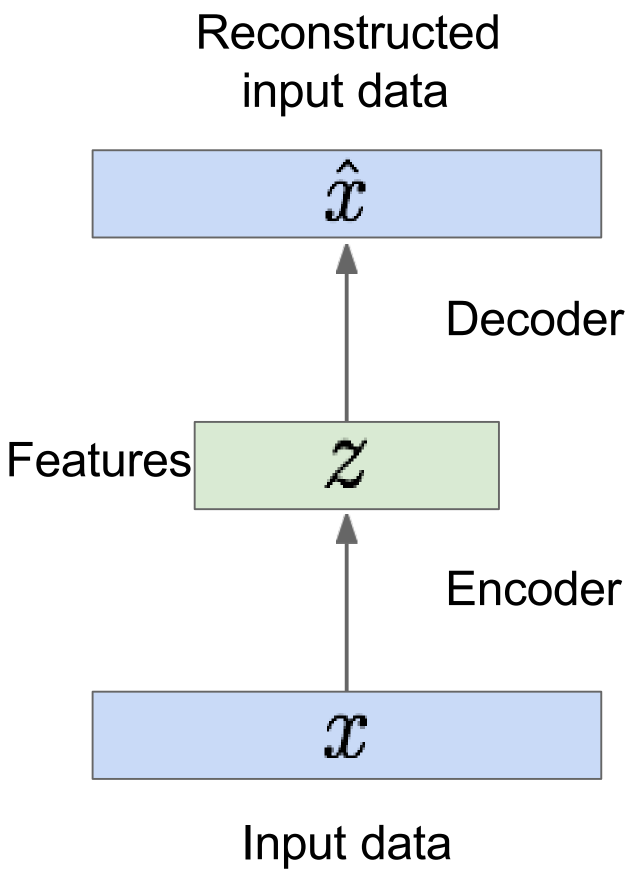
Autoencoders vs. Variational Autoencoders

Autoencoders

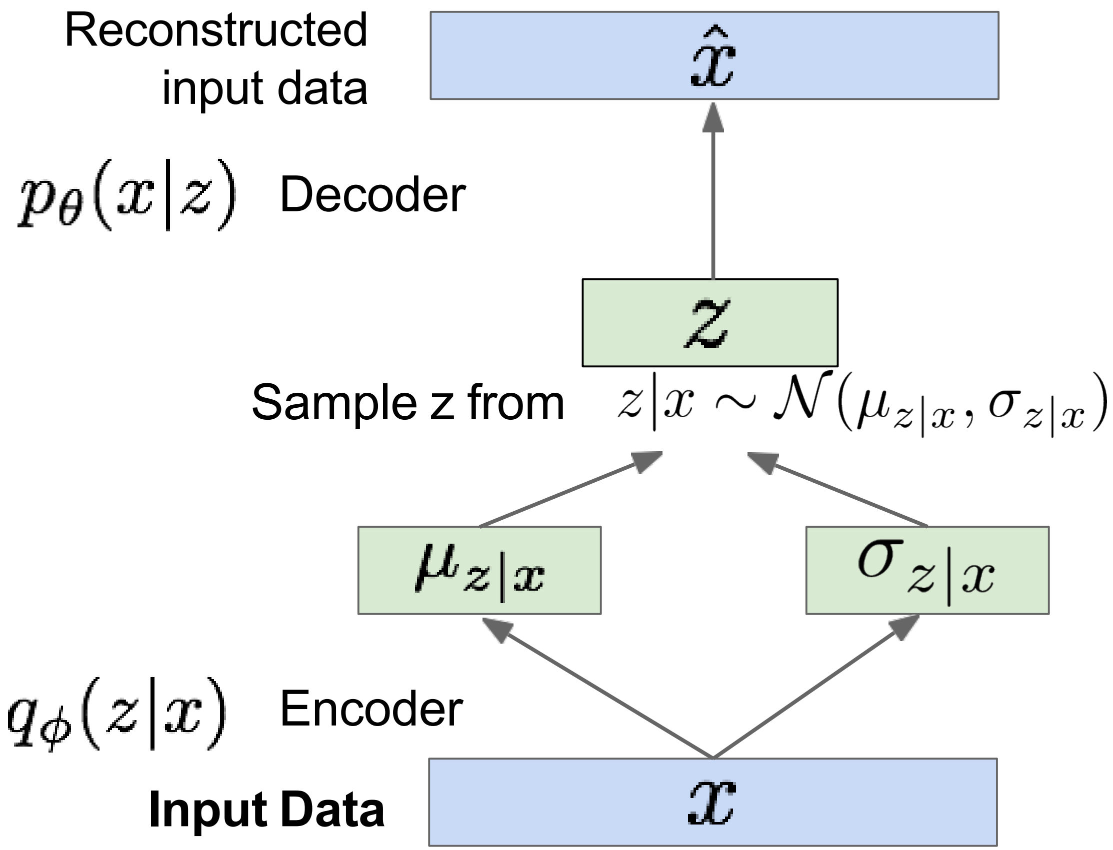


Autoencoders vs. Variational Autoencoders

Autoencoders



Variational Autoencoder

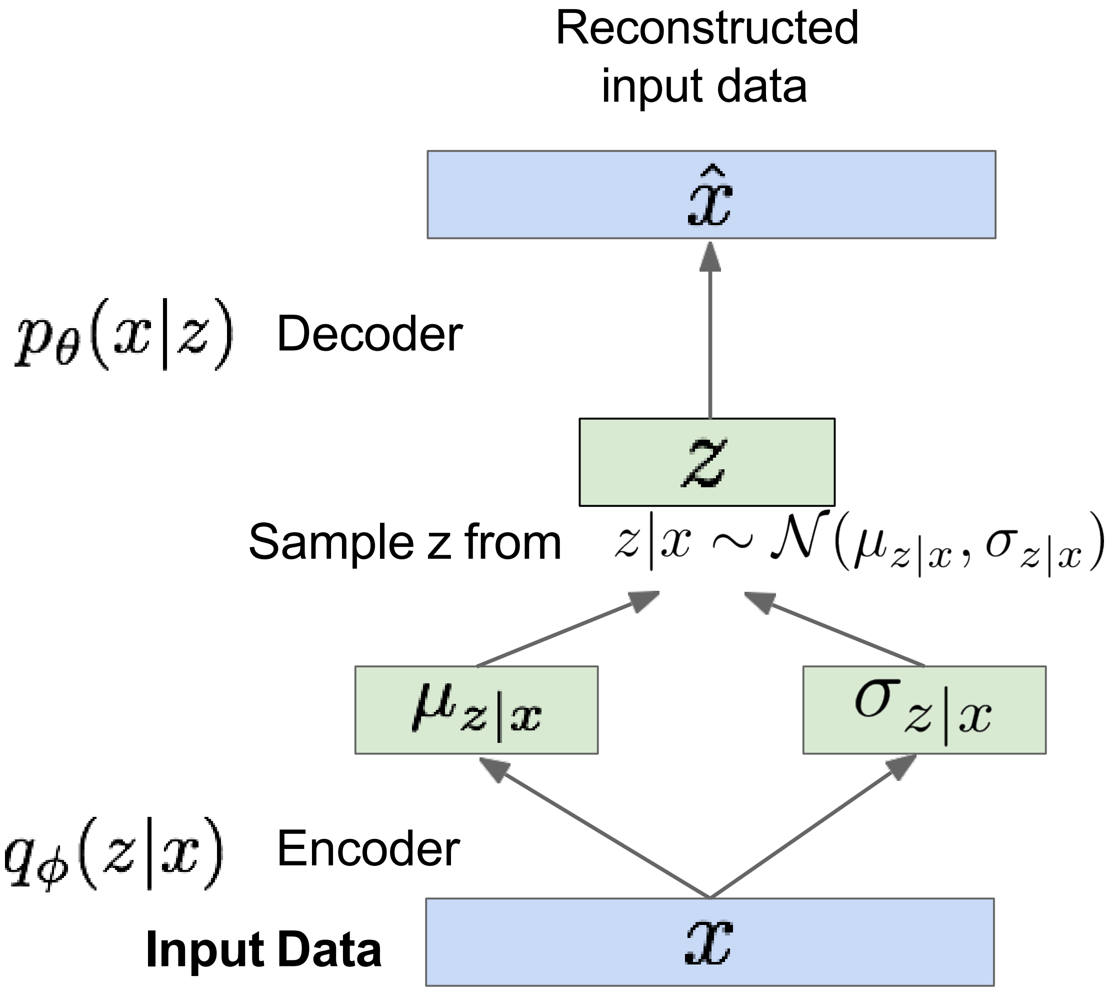


Variational Autoencoders

- Loss function:

(reconstruction loss) (regularization term)

$$\mathcal{L}(x, \phi, \theta) = \mathcal{L}(x, \hat{x}) + D(q_\theta(z|x) \| p(z))$$



Variational Autoencoders

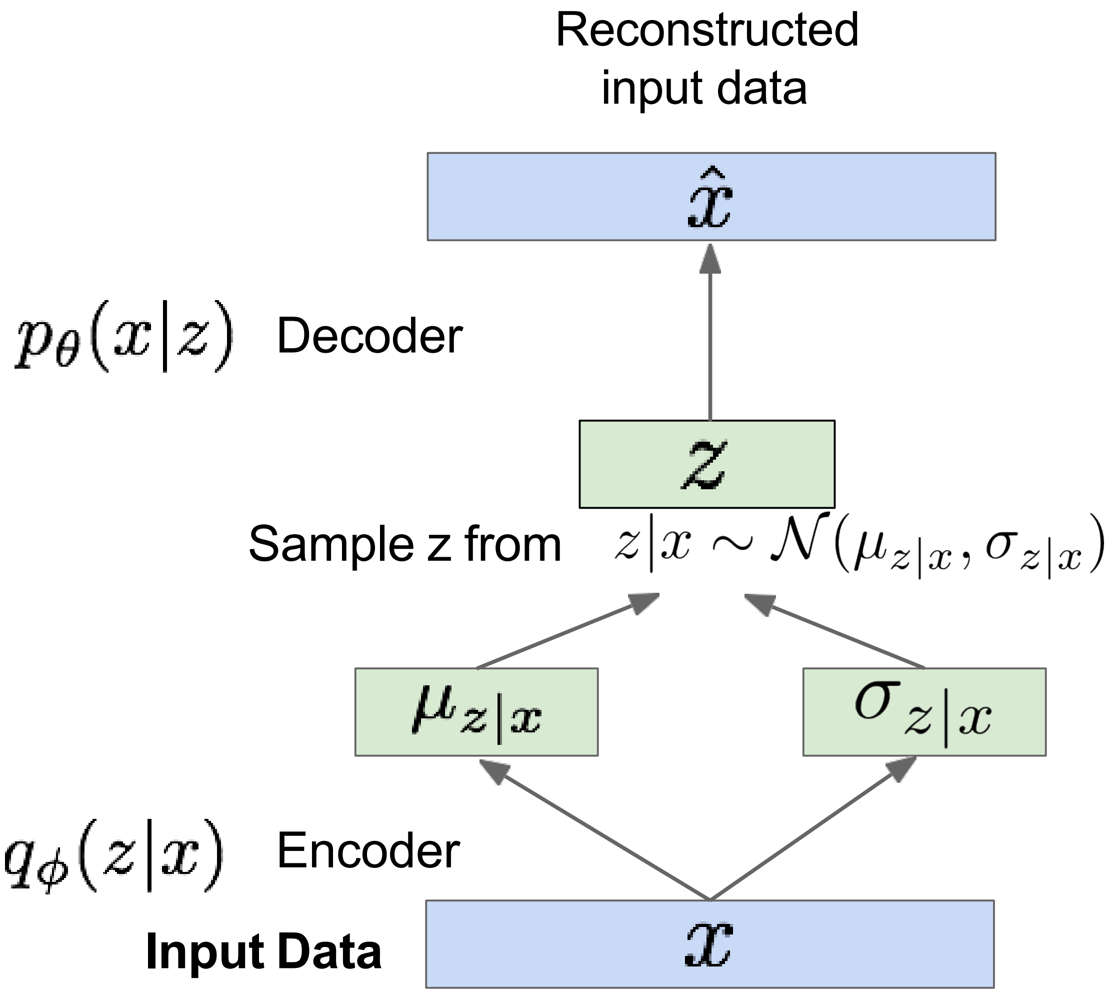
- Loss function:

(reconstruction loss) (regularization term)

$$\mathcal{L}(x, \phi, \theta) = \mathcal{L}(x, \hat{x}) + D(q_\theta(z|x) \| p(z))$$

<u>Common choice</u>

$$p(z) \sim \mathcal{N}(\mu = 0, \sigma = 1)$$



Variational Autoencoders

- Loss function:

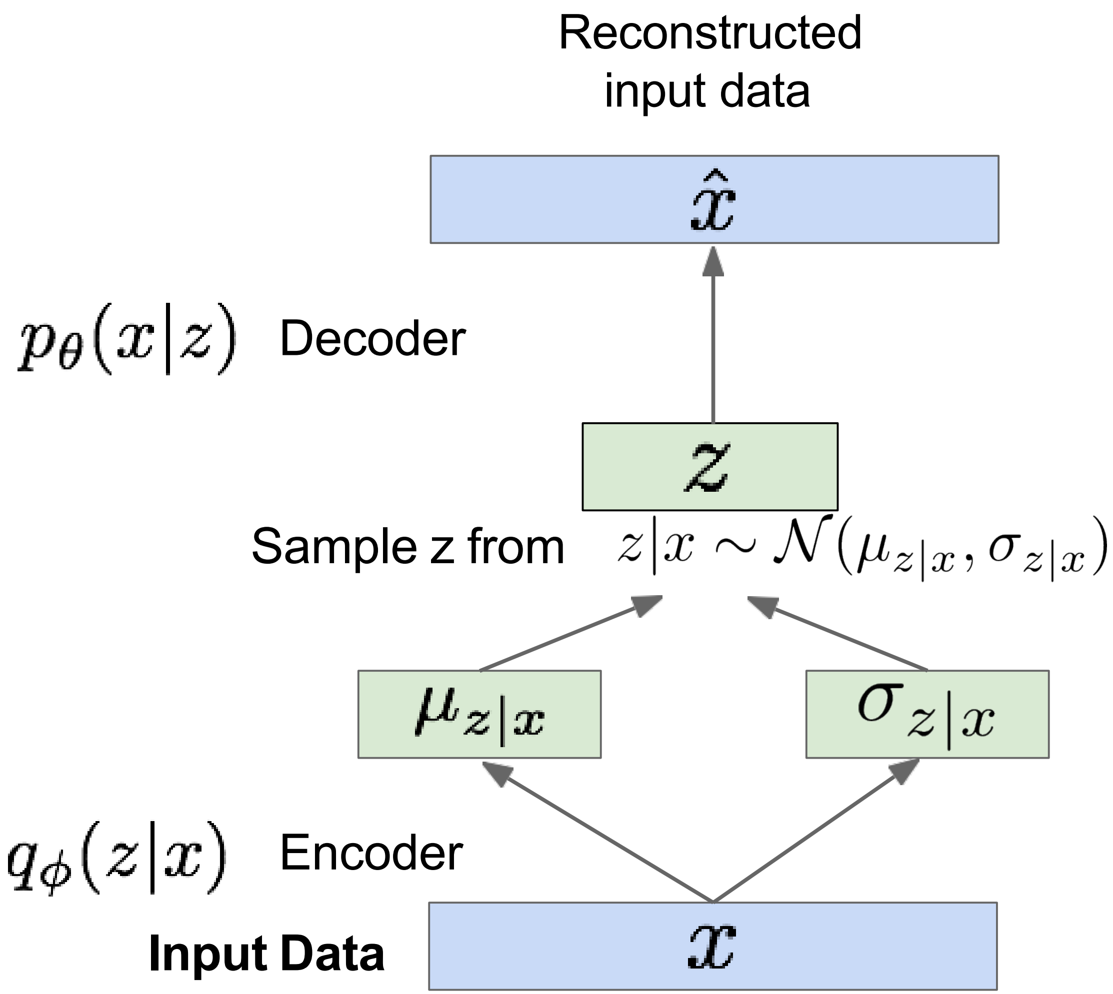
(reconstruction loss) (regularization term)

$$\mathcal{L}(x,\phi,\theta) = \mathcal{L}(x,\hat{x}) + D(q_\theta(z|x)\|p(z))$$

Common choice

$$p(z) \sim \mathcal{N}(\mu=0,\sigma=1)$$

- Makes encoder to distribute encodings evenly
- Prevent the network from clustering data points



Variational Autoencoders

- Loss function:

(reconstruction loss) (regularization term)

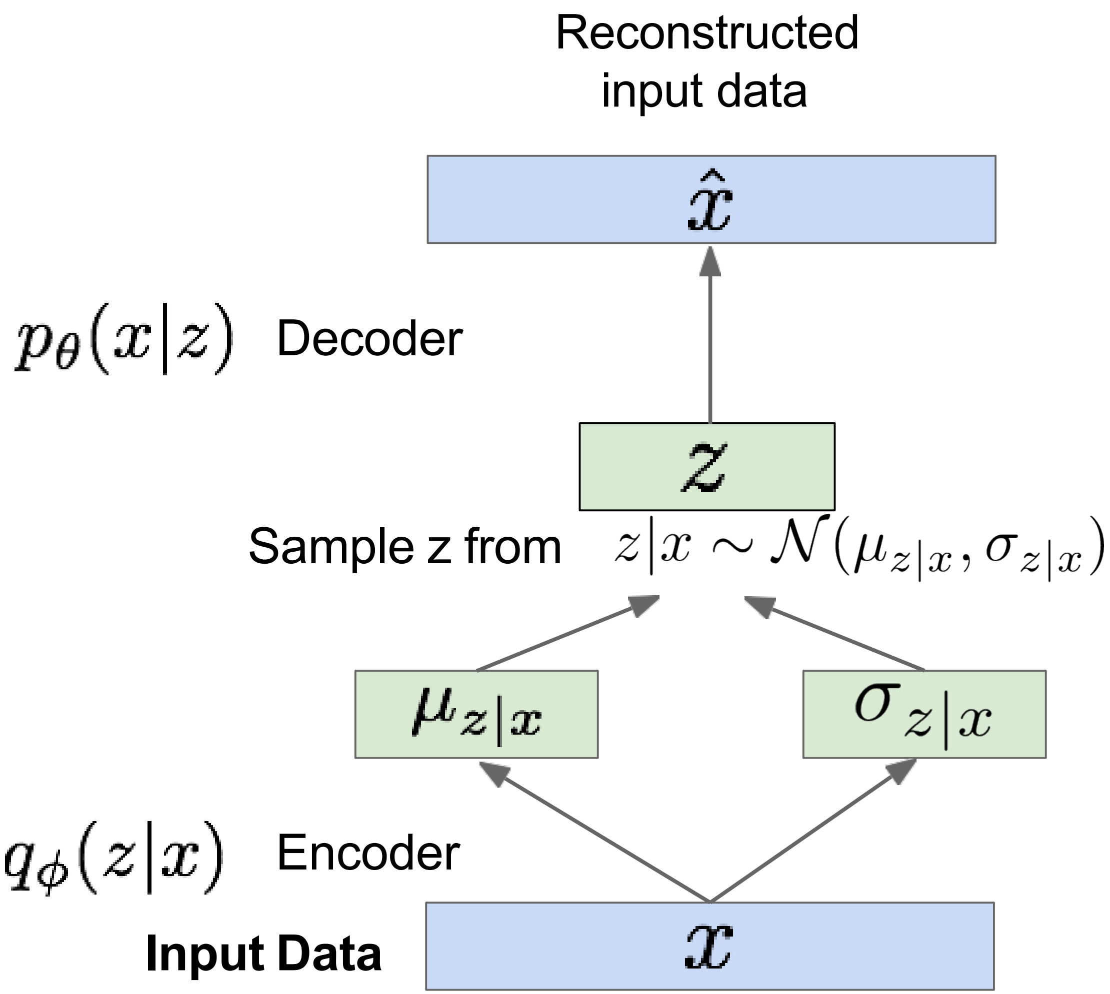
$$\mathcal{L}(x,\phi,\theta) = \mathcal{L}(x,\hat{x}) + D(q_\theta(z|x)\|p(z))$$

<u>Common choice</u>

$$p(z) \sim \mathcal{N}(\mu = 0, \sigma = 1)$$

Closed-form KL-divergence

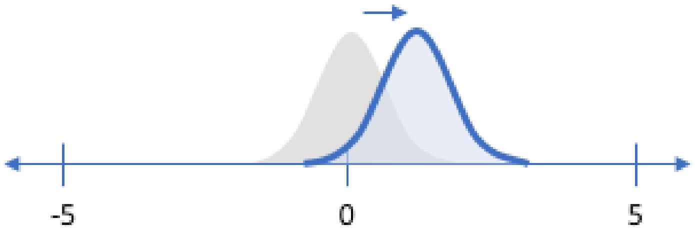
$$D(q_\theta(z|x)\|p(z)) = -\frac{1}{2}\sum_i(\sigma_i + \mu_i^2 - 1 - \log\sigma_i)$$



Variational Autoencoders

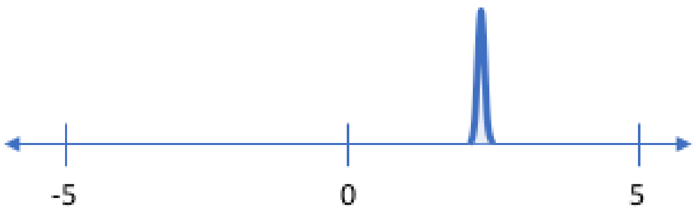
Why regularization?

Penalizing reconstruction loss encourages the distribution to describe the input



Our distribution deviates from the prior to describe some characteristic of the data

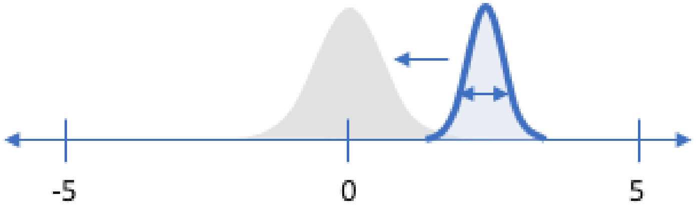
Without regularization, our network can “cheat” by learning narrow distributions



With a small enough variance, this distribution is effectively only representing a single value

Penalizing KL divergence acts as a regularizing force

Attract distribution to have zero mean

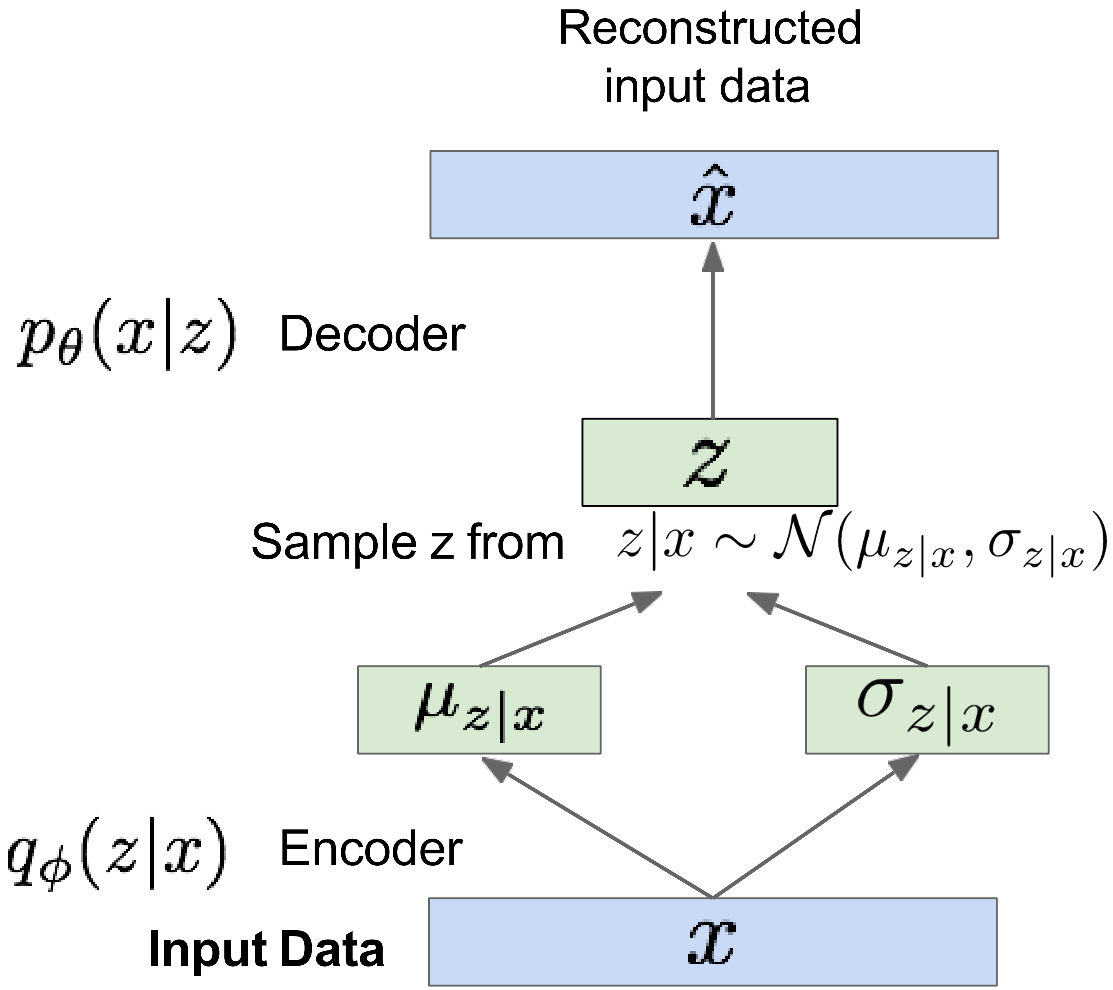


Ensure sufficient variance to yield a smooth latent space

https://www.jeremyjordan.me/variational-autoencoders/

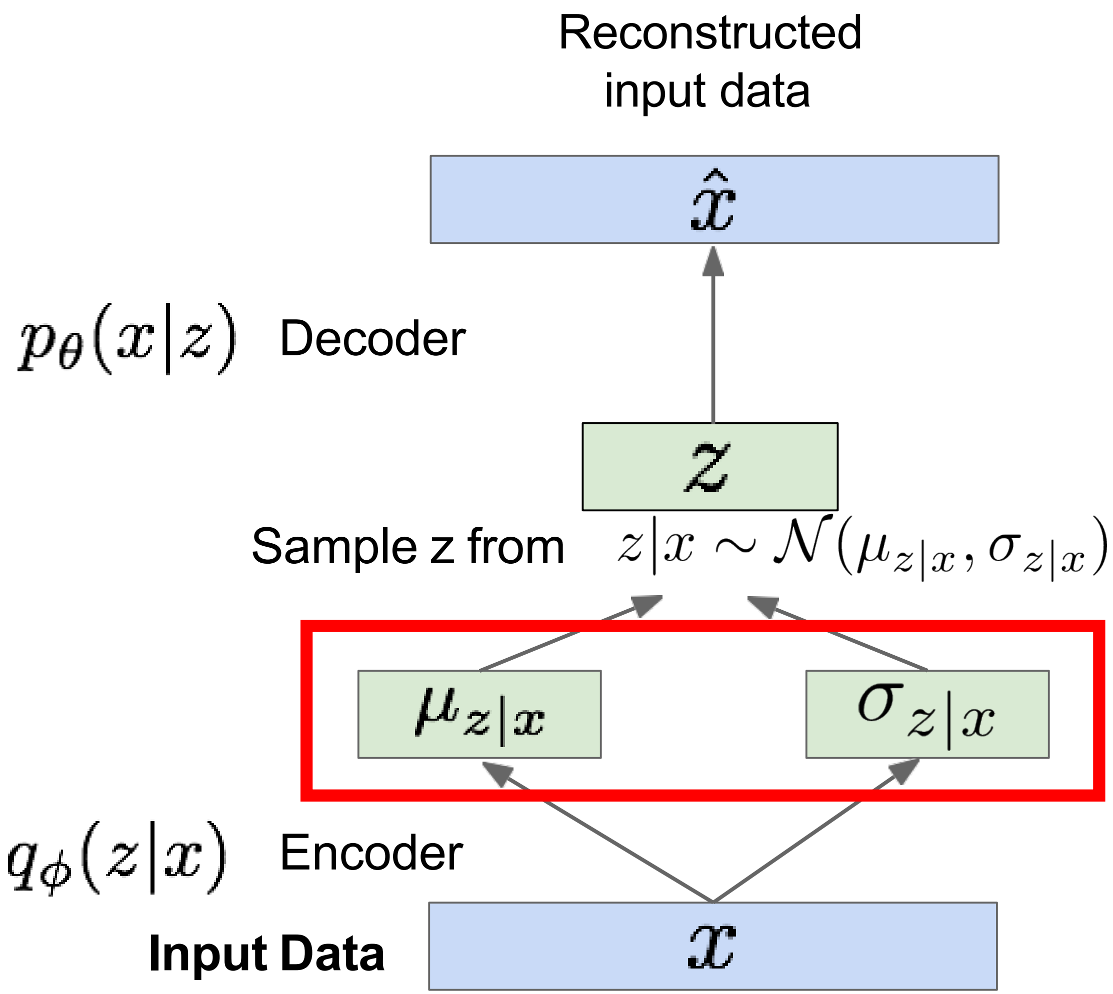
Variational Autoencoders

What about backpropagation?



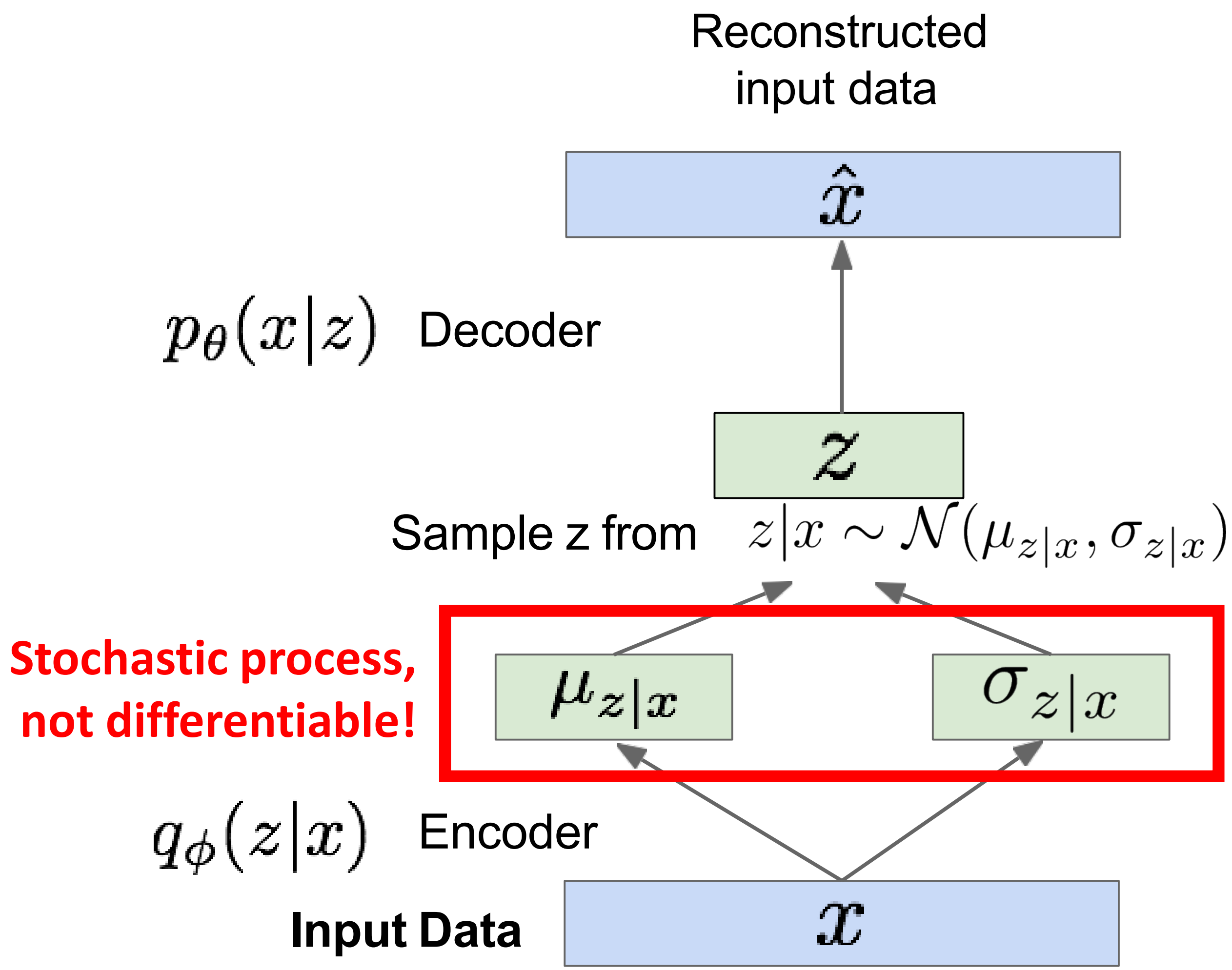
Variational Autoencoders

What about backpropagation?



Variational Autoencoders

What about backpropagation?



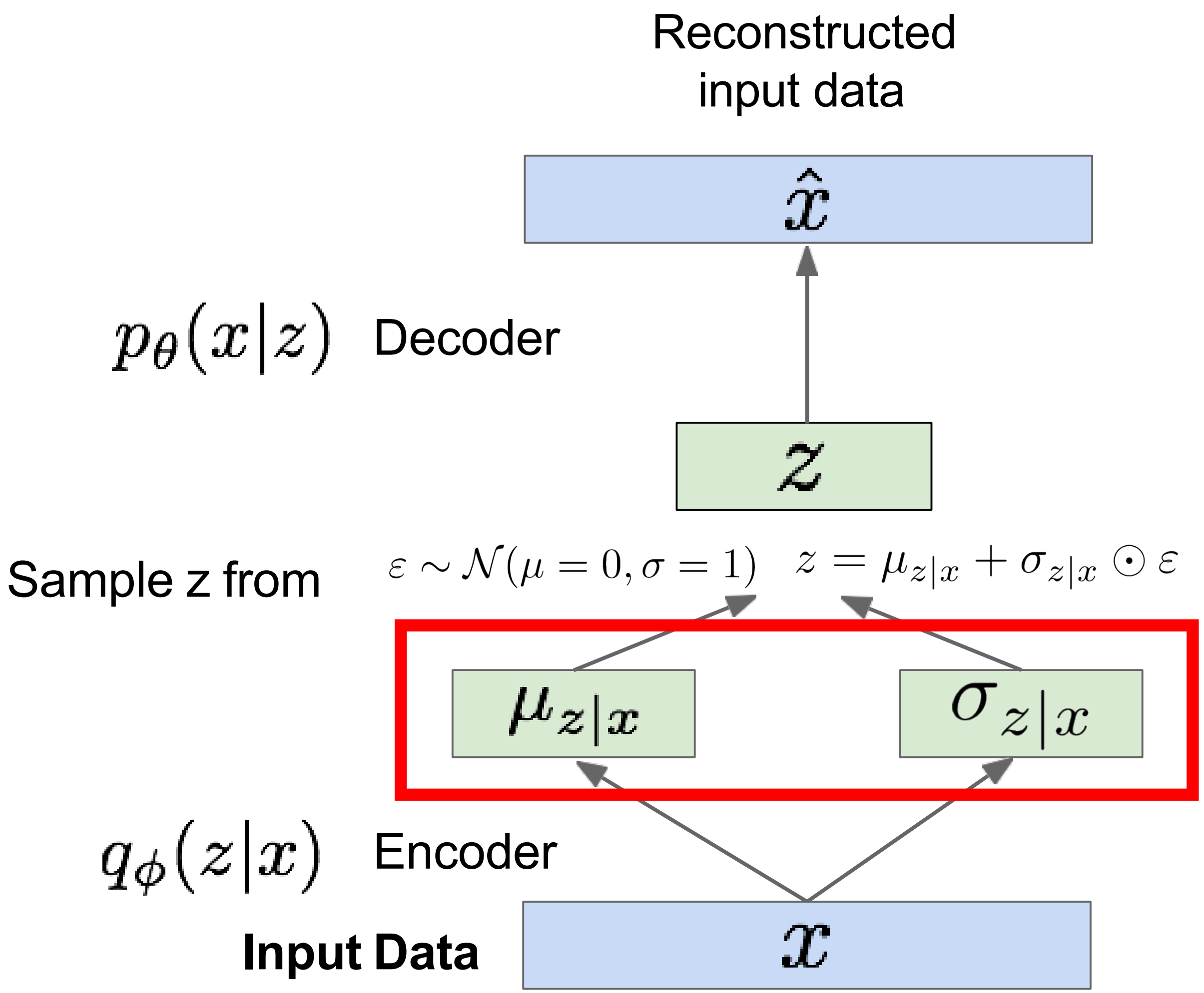
Variational Autoencoders

What about backpropagation?

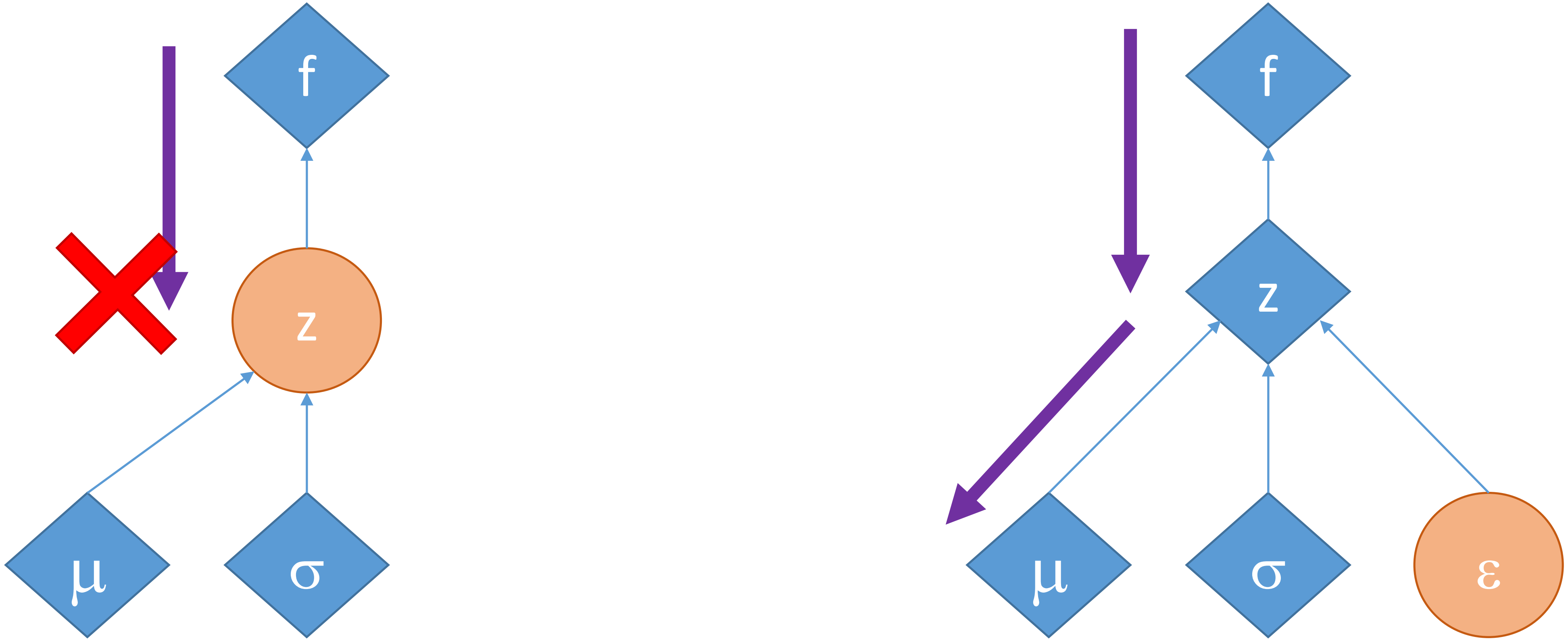
REPARAMETERIZATION TRICK

Input to training graph $\varepsilon \sim \mathcal{N}(\mu = 0, \sigma = 1)$

$$z = \mu_{z|x} + \sigma_{z|x} \odot \varepsilon$$



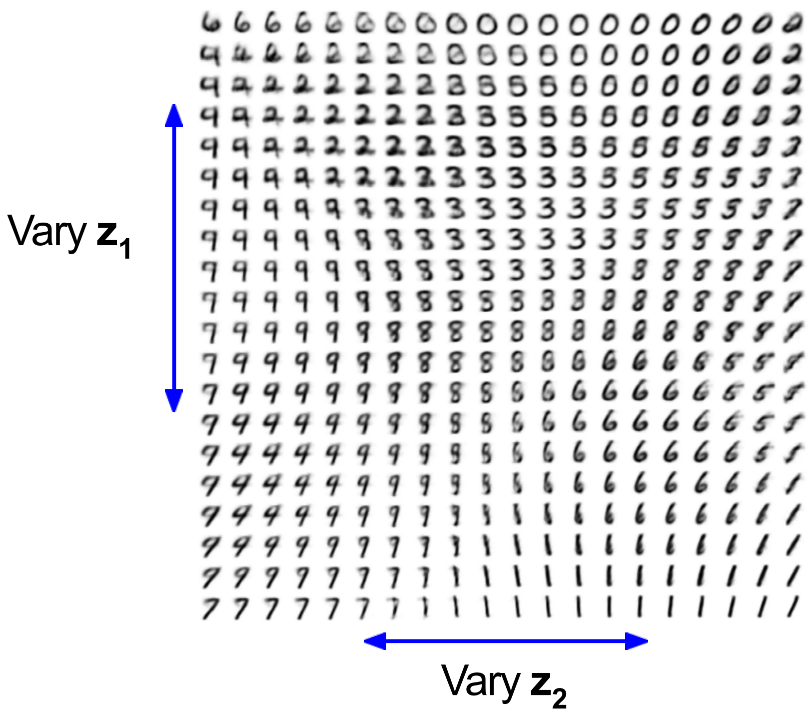
Variational Autoencoders

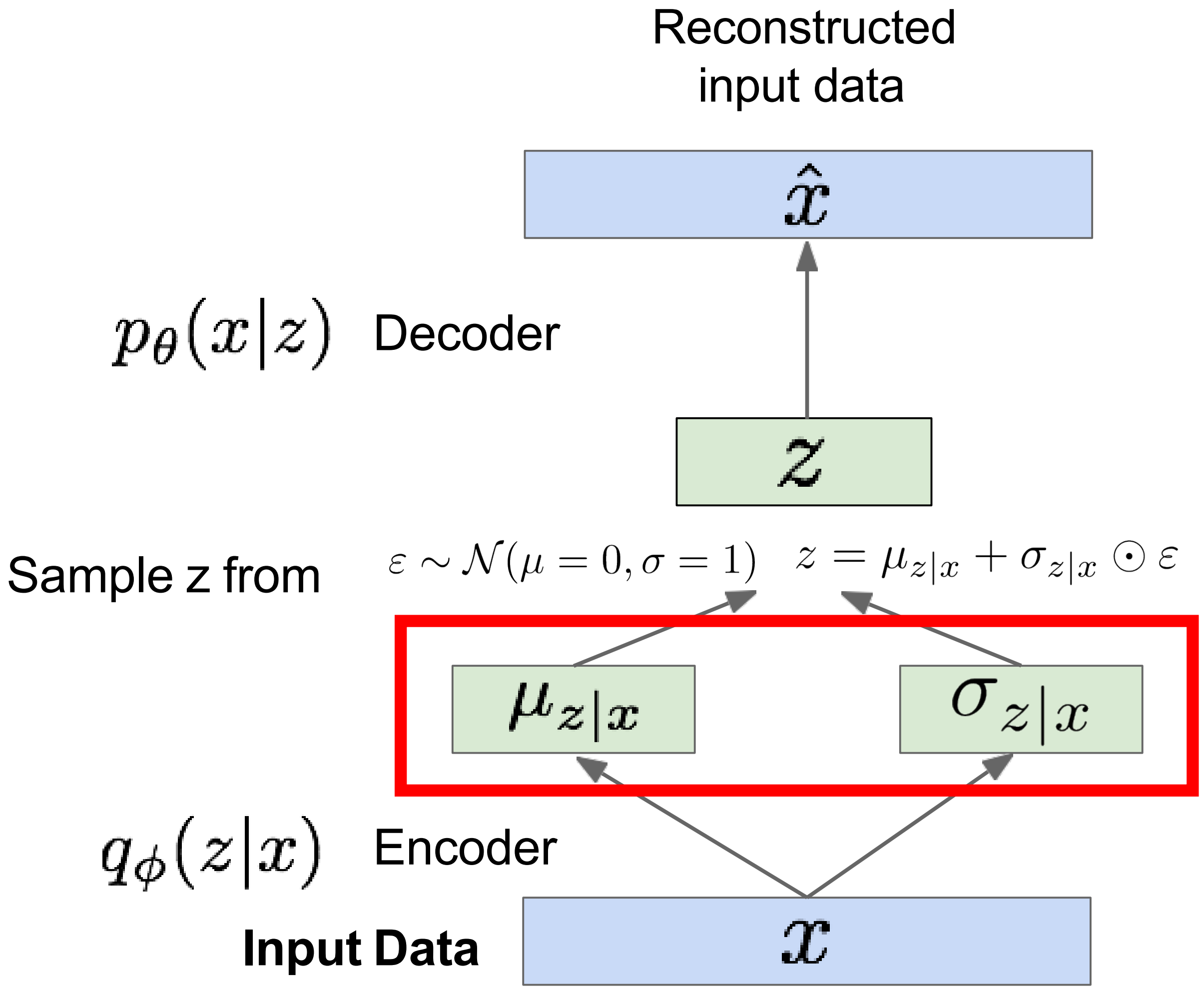


Variational Autoencoders: Examples

Data generation:

Change z to sample new outputs using the decoder network

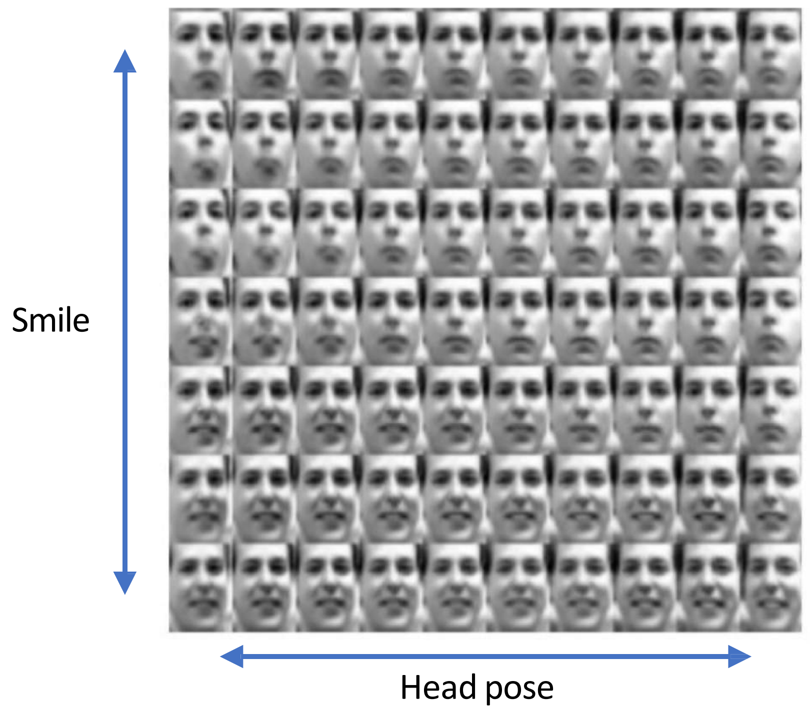


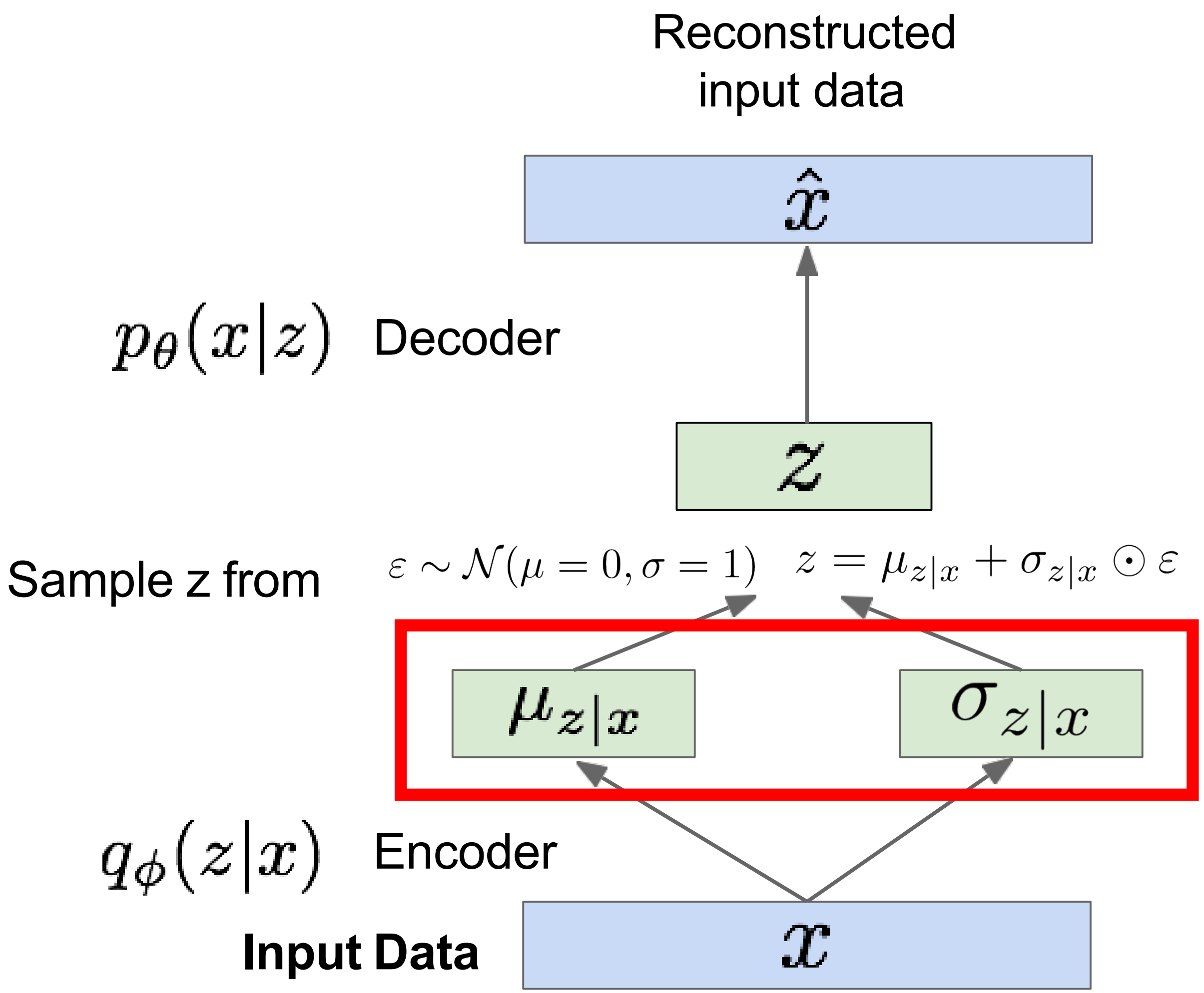


Variational Autoencoders: Examples

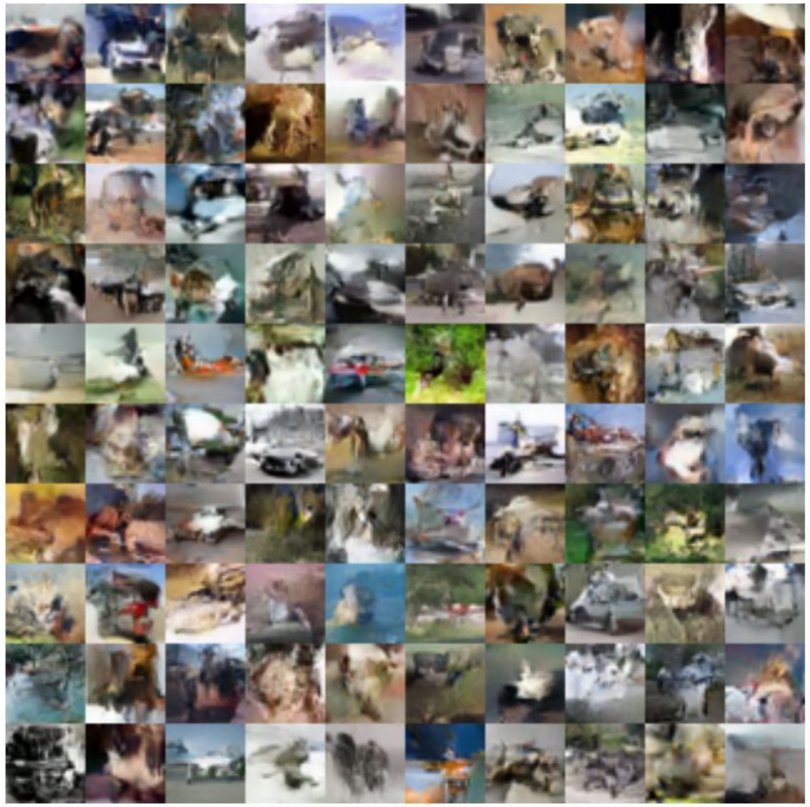
Data generation:

Change z to sample new outputs using the decoder network





Variational Autoencoders: Generating Data!



32x32 CIFAR-10

Labeled Faces in the Wild

Questions?

Taxonomy of Generative Models

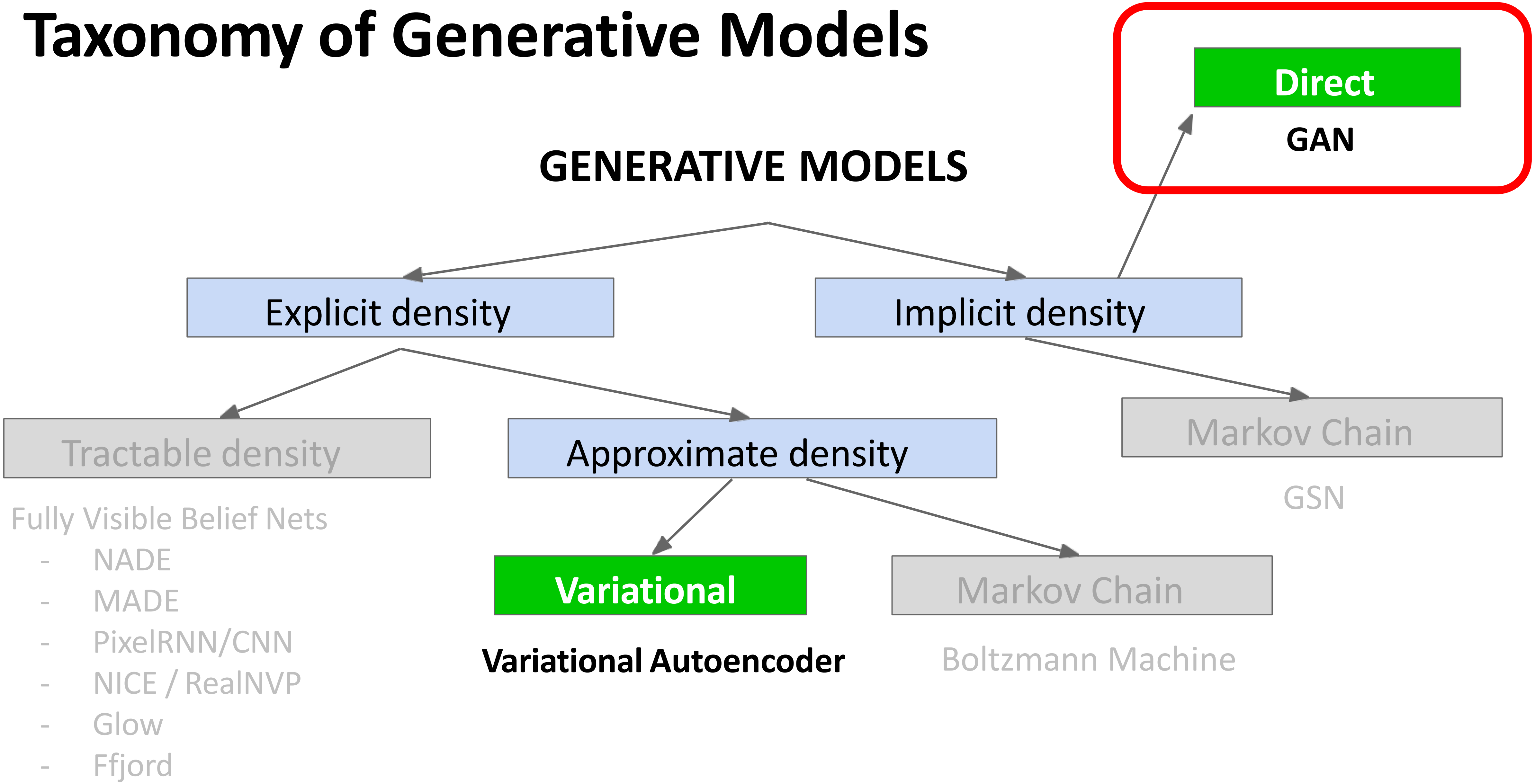


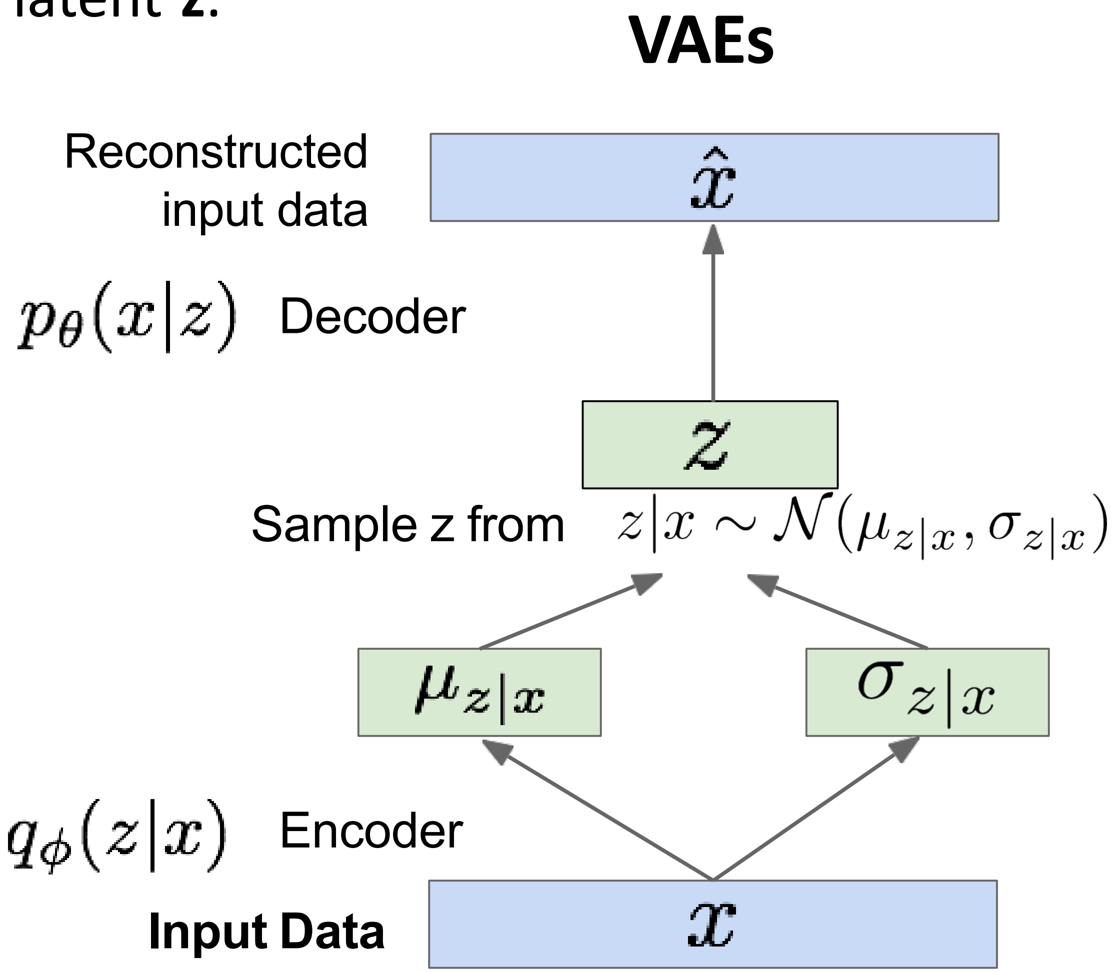
Figure copyright and adapted from Ian Goodfellow, Tutorial on Generative Adversarial Networks, 2017.

Statistical generative methods: **Generative adversarial networks**

GANs: Motivation

VAEs explicitly model a density function with latent **z**.

$$z|x \sim \mathcal{N}(\mu_{z|x}, \sigma_{z|x})$$



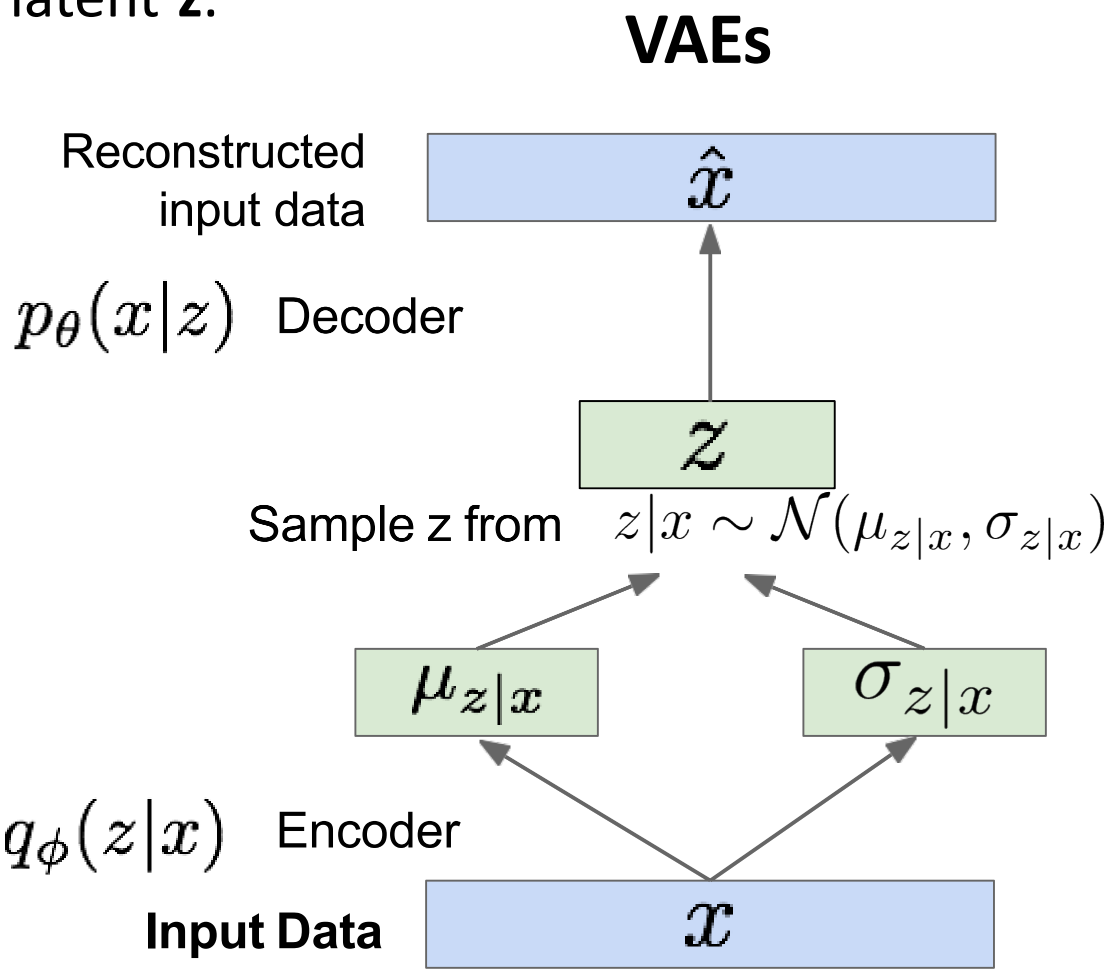
GANs: Motivation

VAEs explicitly model a density function with latent **z**.

$$z|x \sim \mathcal{N}(\mu_{z|x}, \sigma_{z|x})$$

Problem: Sampling complex, high-dimensional training distribution.

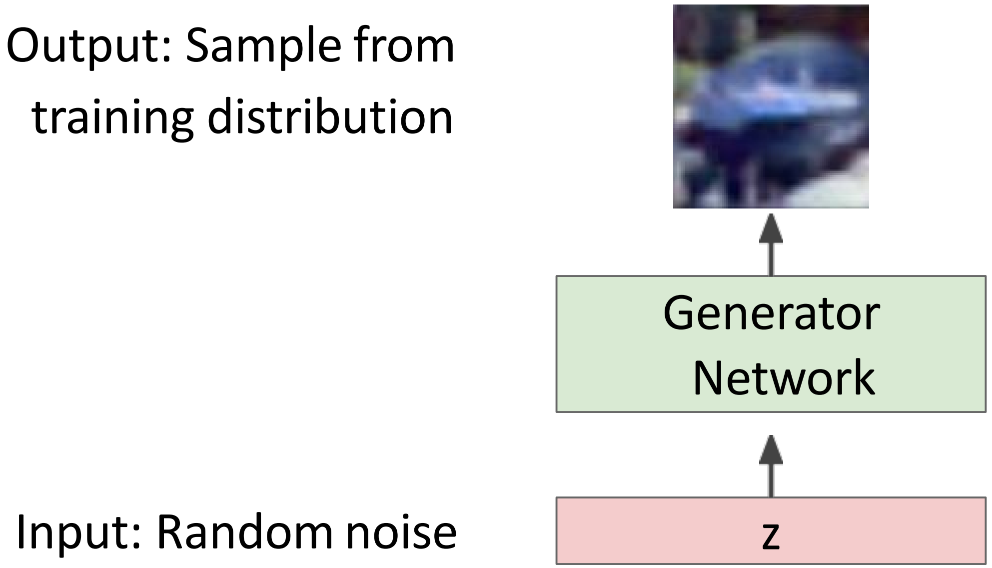
- No direct way to do this
- Explicitly modeling density may be impossible
- Maximizing likelihood not good indicator of performance
- Samples are blurrier and lower quality than e.g. GANs, diffusion models



Generative Adversarial Networks

What if we give up on explicitly modeling density, and just want the ability to sample?

GANs: Solution to avoid modeling any explicit density function.

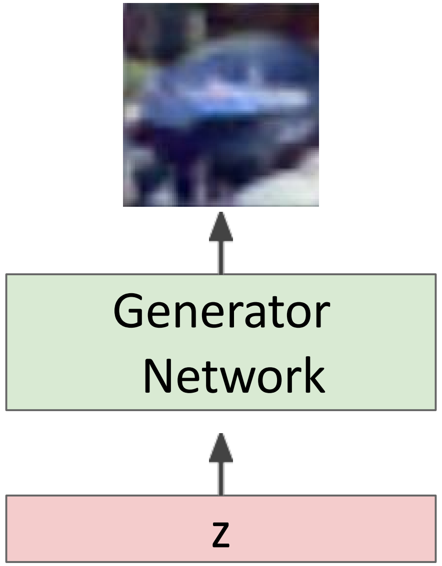


Generative Adversarial Networks

What if we give up on explicitly modeling density, and just want the ability to sample?

GANs: Solution to avoid modeling any explicit density function.

Output: Sample from training distribution



Input: Random noise

But we don’t know which sample z maps to which training image -> can’t learn by reconstructing training images

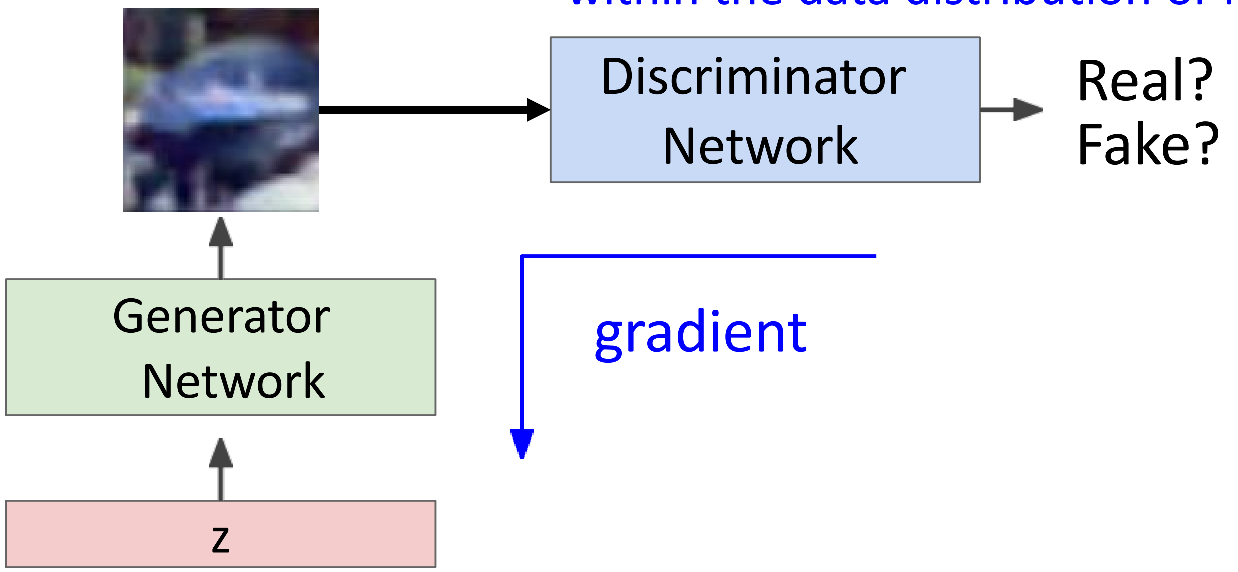
Generative Adversarial Networks

What if we give up on explicitly modeling density, and just want the ability to sample?

GANs: Solution to avoid modeling any explicit density function.

Learn whether the generated image is within the data distribution or not

Output: Sample from training distribution



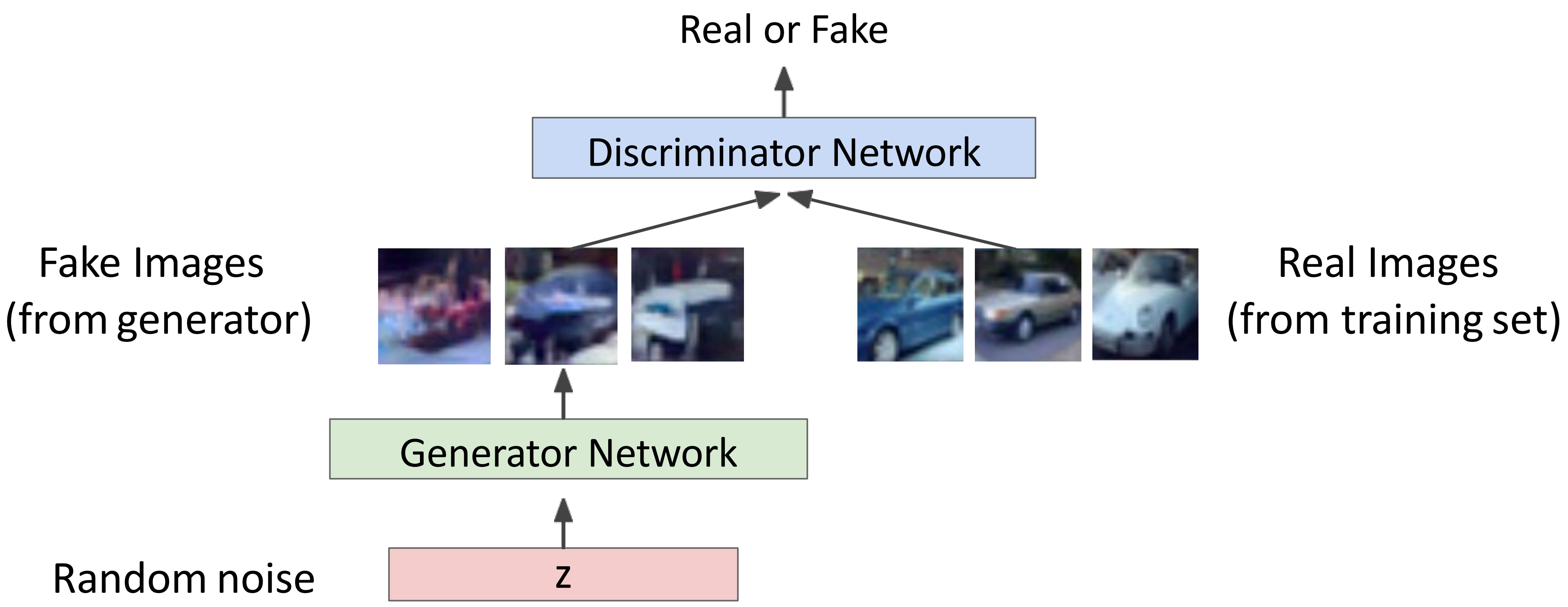
Input: Random noise

Generative Adversarial Networks

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Discriminator network: try to distinguish between real and fake images

Generator network: try to fool the discriminator by generating real-looking images



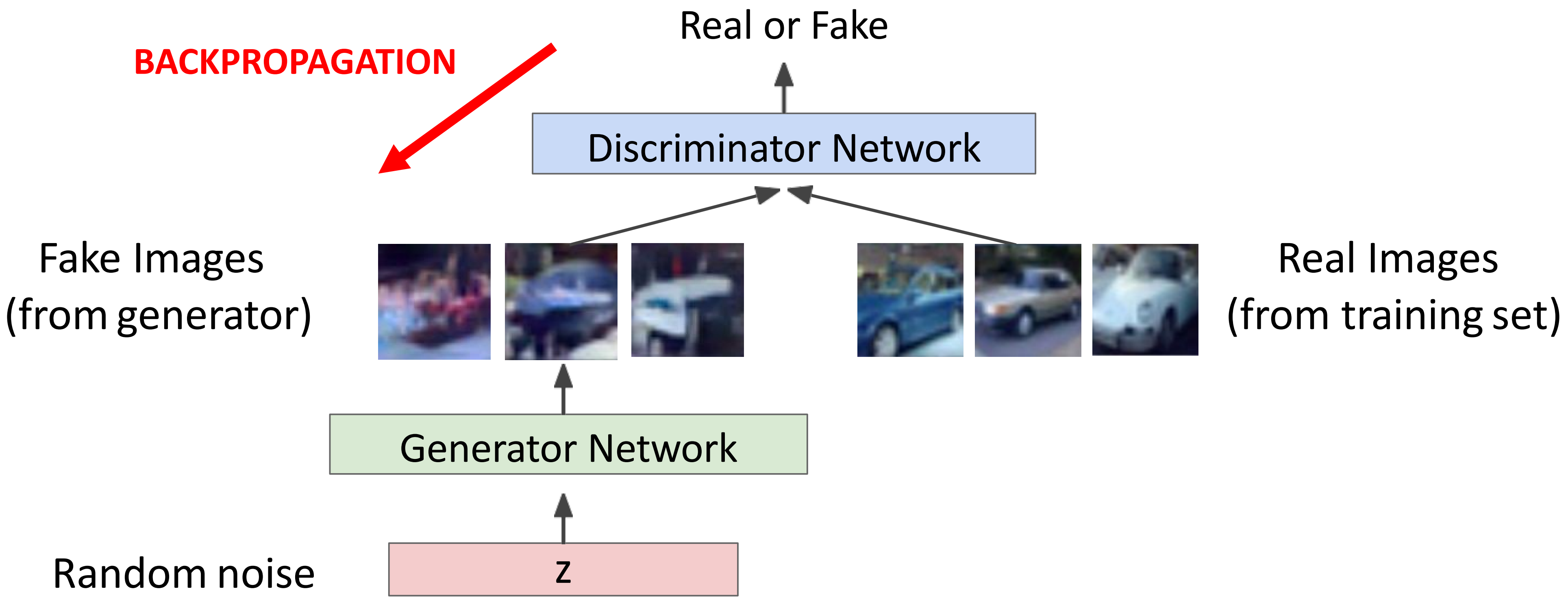
Fake and real images copyright Emily Denton et al. 2015. Reproduced with permission.

Generative Adversarial Networks

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Discriminator network: try to distinguish between real and fake images

Generator network: try to fool the discriminator by generating real-looking images



Fake and real images copyright Emily Denton et al. 2015. Reproduced with permission.

Generative Adversarial Networks

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Discriminator network: try to distinguish between real and fake images
Generator network: try to fool the discriminator by generating real-looking images

Train jointly in **minimax game**

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x\sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z\sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Generator objective

Discriminator objective

Generative Adversarial Networks

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Discriminator network: try to distinguish between real and fake images

Generator network: try to fool the discriminator by generating real-looking images

Train jointly in **minimax game**

Discriminator outputs likelihood in (0,1) of real image

Minimax objective function:

$$\underbrace{\min_{\theta_g}}_{\text{Generator objective}} \underbrace{\max_{\theta_d}}_{\text{Discriminator objective}} \left[\mathbb{E}_{x\sim p_{data}} \log \underbrace{D_{\theta_d}(x)}_{\text{probability that real data is classified as real}} + \mathbb{E}_{z\sim p(z)} \log(1 - \underbrace{D_{\theta_d}(G_{\theta_g}(z))}_{\text{probability that fake data is classified as real}})\right]$$

Generative Adversarial Networks

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Discriminator network: try to distinguish between real and fake images
Generator network: try to fool the discriminator by generating real-looking images

Train jointly in **minimax game**

Discriminator outputs likelihood in (0,1) of real image

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log \underbrace{D_{\theta_d}(x)}_{\text{probability that real data is classified as real}} + \mathbb{E}_{z \sim p(z)} \log(1 - \underbrace{D_{\theta_d}(G_{\theta_g}(z))}_{\text{probability that fake data is classified as real}}) \right]$$

Generator objective

Discriminator objective

- Discriminator (θ_d) wants to **maximize objective** such that D(x) is close to 1 and D(G(z)) is close to 0
- Generator (θ_g) wants to **minimize objective** such that D(G(z)) is close to 1 (discriminator is fooled into thinking generated G(z) is real)

Generative Adversarial Networks

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Alternate between:

1. **Gradient ascent** on discriminator

$$\max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

2. **Gradient descent** on generator

$$\min_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$$

Generative Adversarial Networks

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Alternate between:

1. **Gradient ascent** on discriminator

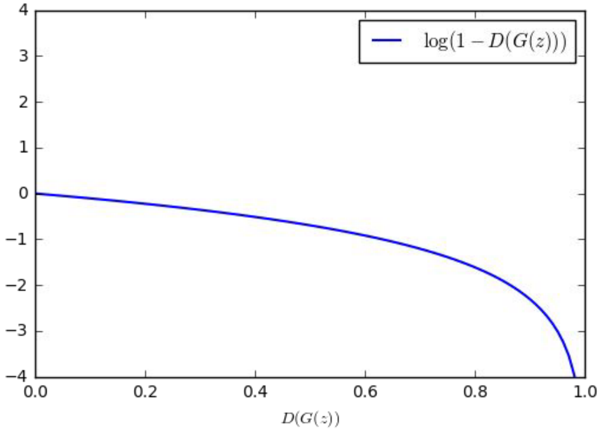
$$\max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

2. **Gradient descent** on generator

$$\min_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$$

In practice, optimizing this generator objective does not work well!

When sample is likely fake, want to learn from it to improve generator (move to the right on X axis).



Generative Adversarial Networks

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Alternate between:

1. **Gradient ascent** on discriminator

$$\max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

2. **Gradient descent** on generator

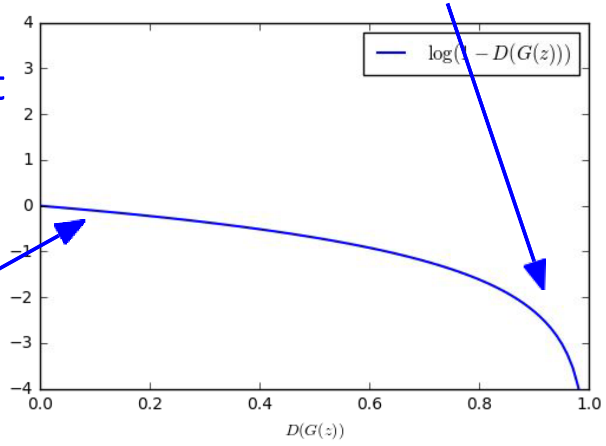
$$\min_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$$

In practice, optimizing this generator objective does not work well!

When sample is likely fake, want to learn from it to improve generator (move to the right on X axis).

But gradient in this region is relatively flat!

Gradient signal dominated by region where sample is already good



Generative Adversarial Networks

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Alternate between:

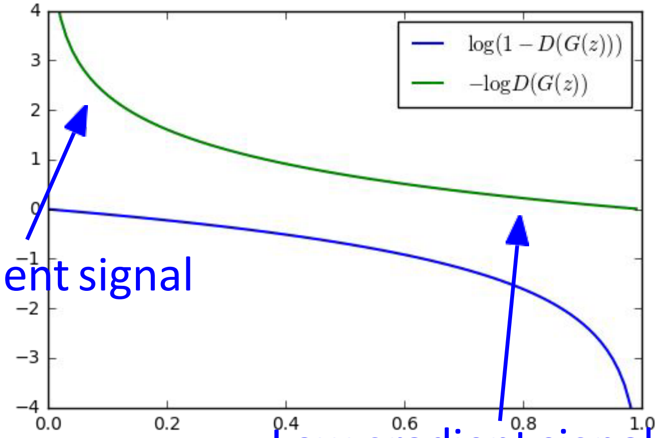
1. **Gradient ascent** on discriminator

$$\max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

2. **Instead: Gradient ascent** on generator, different objective

$$\max_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(D_{\theta_d}(G_{\theta_g}(z)))$$

Instead of minimizing likelihood of discriminator being correct, now **maximize likelihood of discriminator being wrong**.
Same objective of fooling discriminator, but now higher gradient signal for bad samples => works much better! Standard in practice.



High gradient signal

Low gradient signal

Generative Adversarial Networks

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Putting it together: GAN training algorithm

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \dots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
- Sample minibatch of m examples $\{\boldsymbol{x}^{(1)}, \dots, \boldsymbol{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\boldsymbol{x})$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[\log D_{\theta_d}(x^{(i)}) + \log(1 - D_{\theta_d}(G_{\theta_g}(z^{(i)}))) \right]$$

end for

- Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \dots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
- Update the generator by ascending its stochastic gradient (improved objective):

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} \log(D_{\theta_d}(G_{\theta_g}(z^{(i)})))$$

end for

Generative Adversarial Networks

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Putting it together: GAN training algorithm

Some find k=1 more stable, others use k > 1, no best rule.

Follow-up work (e.g. Wasserstein GAN, BEGAN) alleviates this problem, better stability!

for number of training iterations **do**
  for $\boxed{k \text{ steps}}$ **do**
  • Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \dots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
  • Sample minibatch of m examples $\{\boldsymbol{x}^{(1)}, \dots, \boldsymbol{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\boldsymbol{x})$.
  • Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[\log D_{\theta_d}(x^{(i)}) + \log(1 - D_{\theta_d}(G_{\theta_g}(z^{(i)}))) \right]$$

  end for
  • Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \dots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
  • Update the generator by ascending its stochastic gradient (improved objective):

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} \log(D_{\theta_d}(G_{\theta_g}(z^{(i)})))$$

end for

Arjovsky et al. "Wasserstein GAN." arXiv preprint arXiv:1701.07875 (2017)
Berthelot, et al. "Began: Boundary equilibrium generative adversarial networks." arXiv preprint arXiv:1703.10717 (2017)

Intuition behind GANs

Generator starts from noise to try to create an imitation of the data.

Discriminator

Generator

Fake data

Intuition behind GANs

Discriminator looks at both real data and fake data created by the generator.

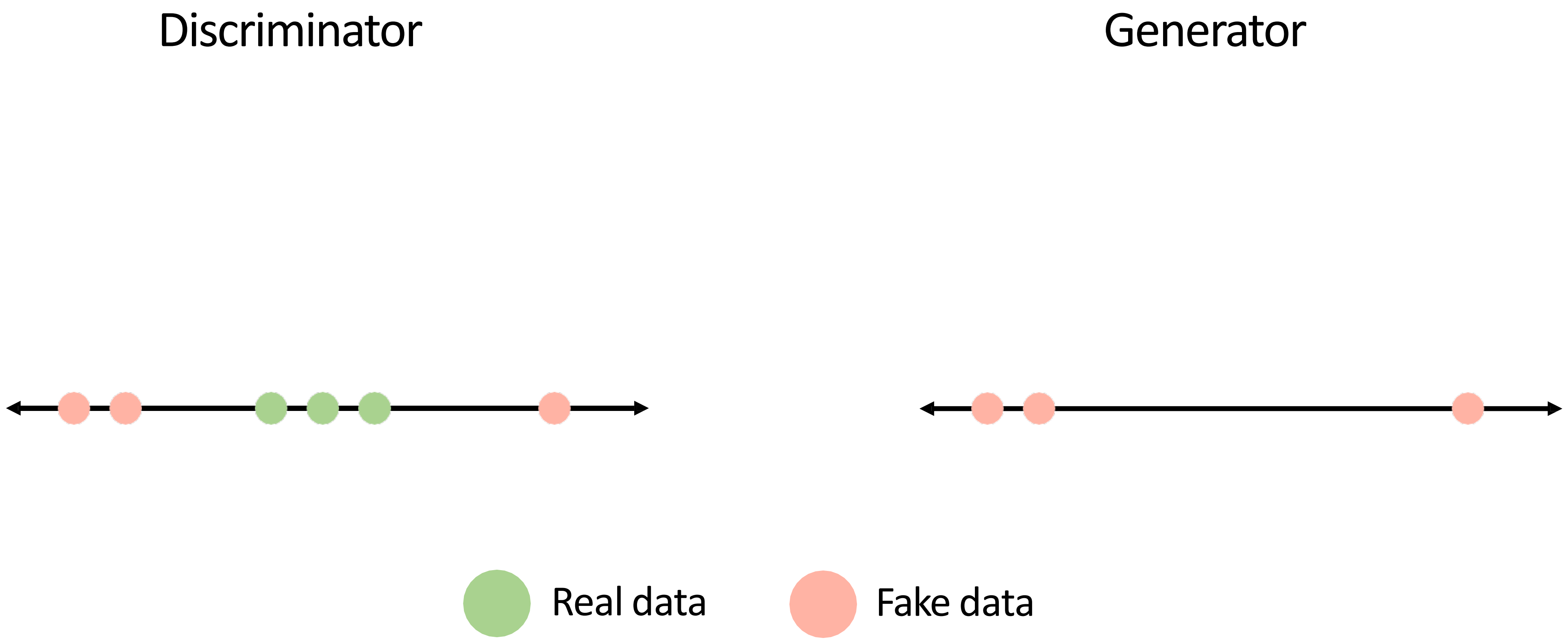
Discriminator

Generator

Fake data

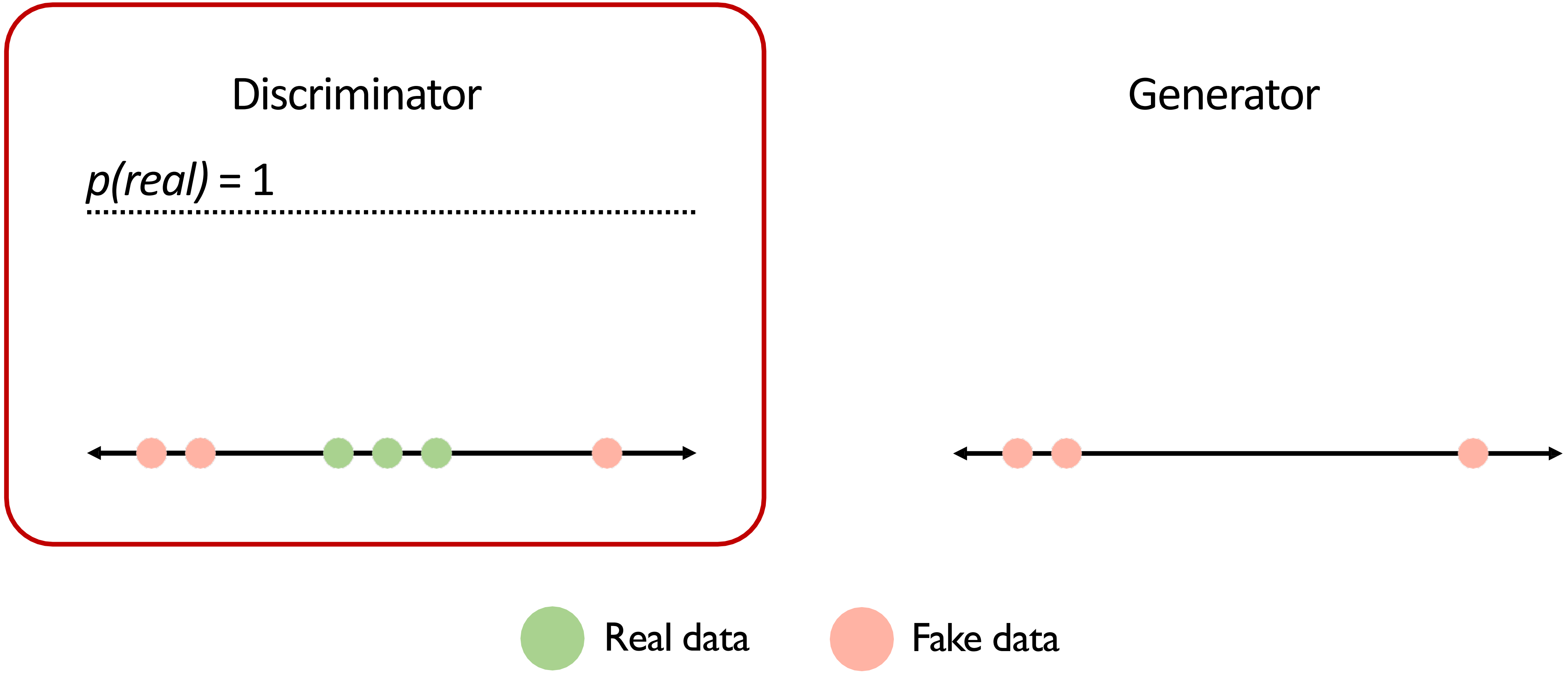
Intuition behind GANs

Discriminator looks at both real data and fake data created by the generator.



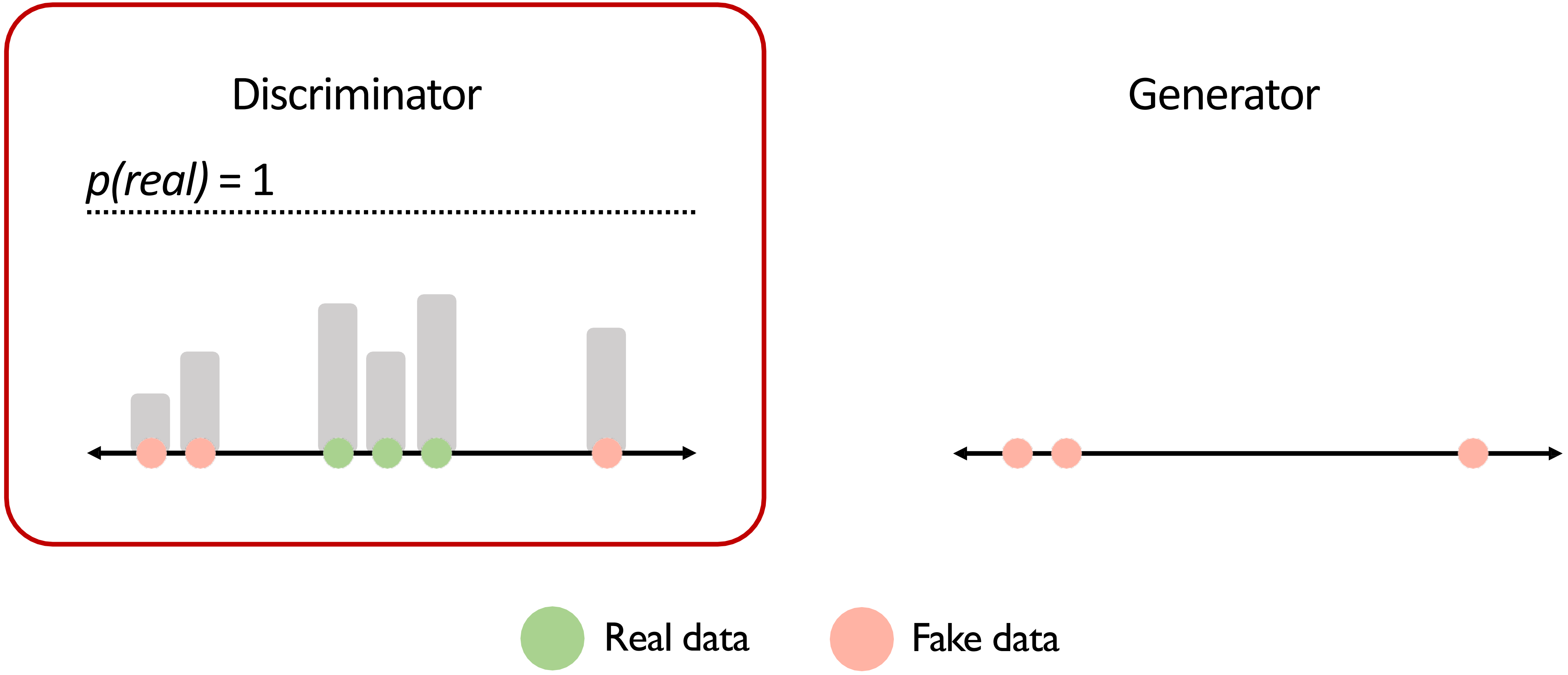
Intuition behind GANs

Discriminator tries to predict what's real and what's fake.



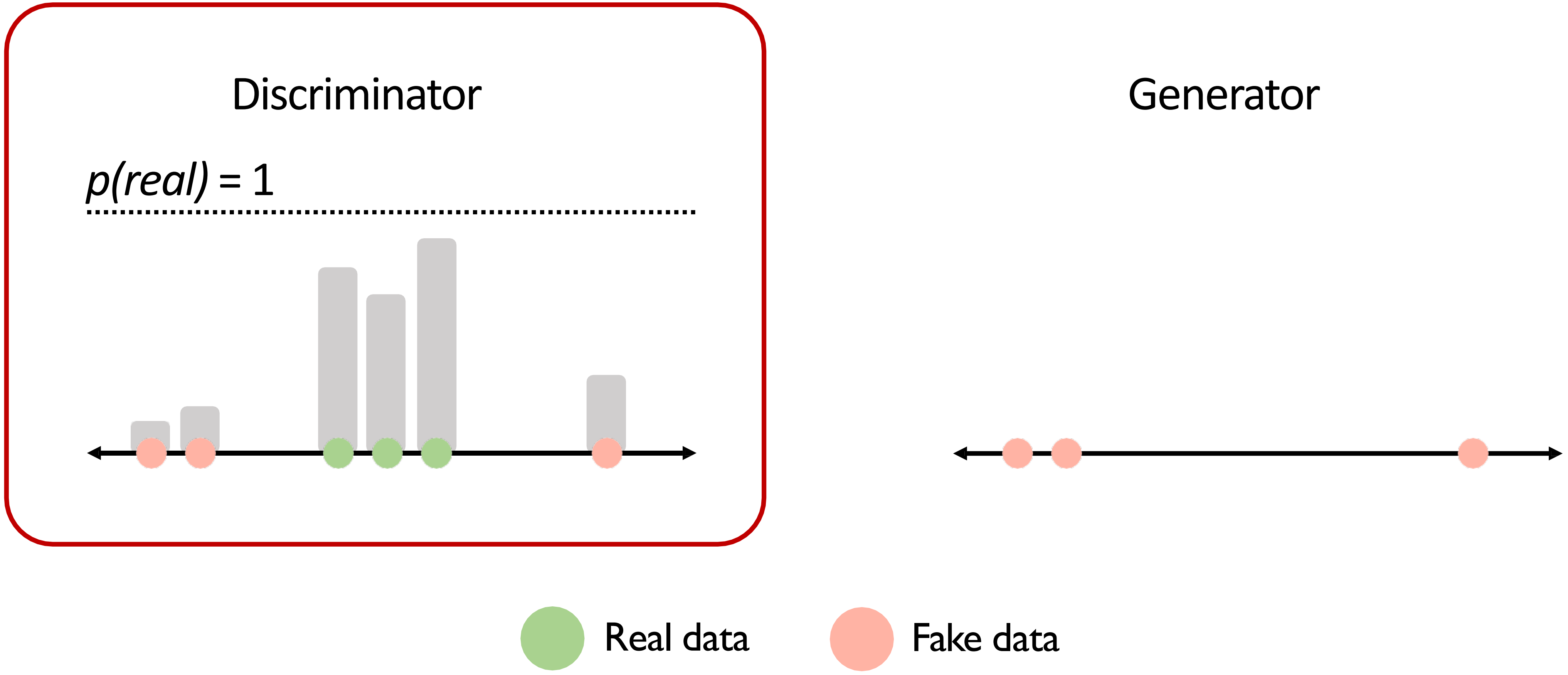
Intuition behind GANs

Discriminator tries to predict what's real and what's fake.



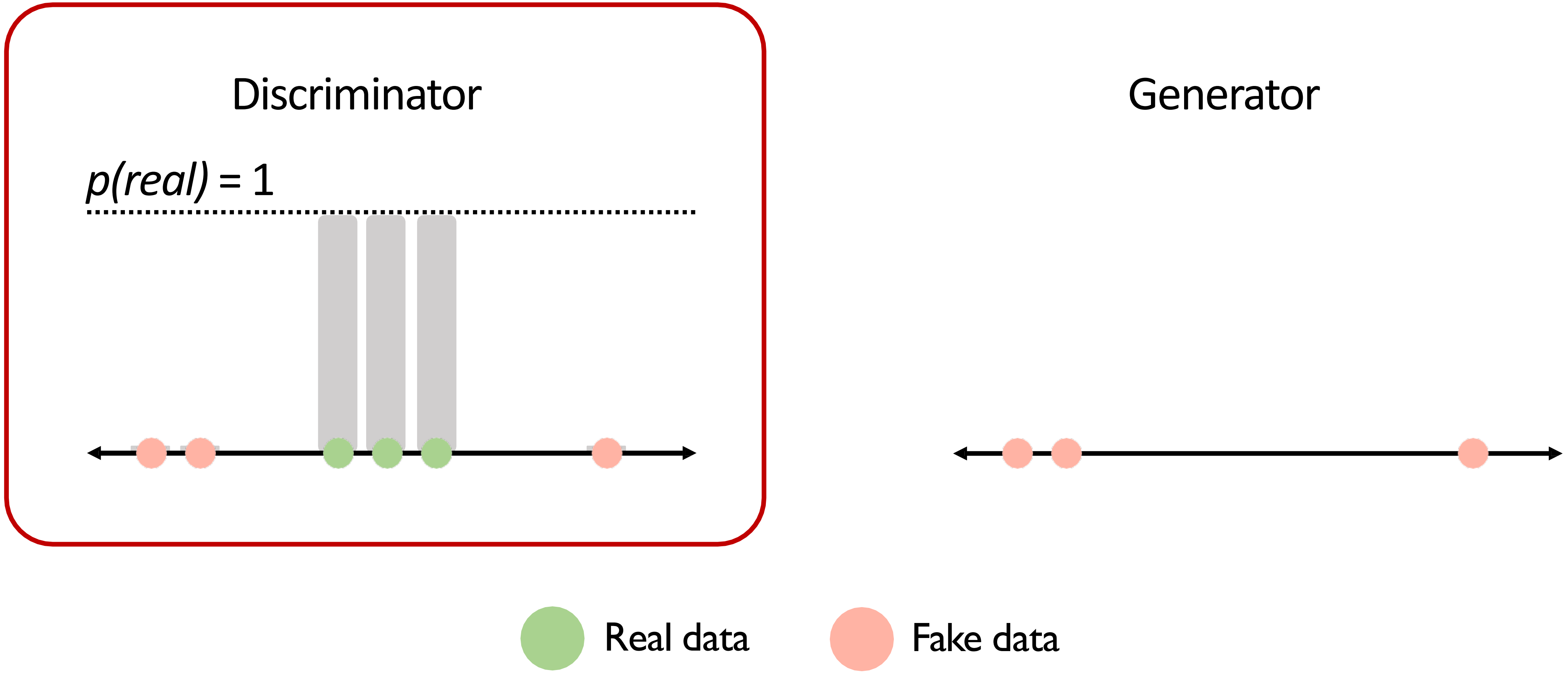
Intuition behind GANs

Discriminator tries to predict what's real and what's fake.



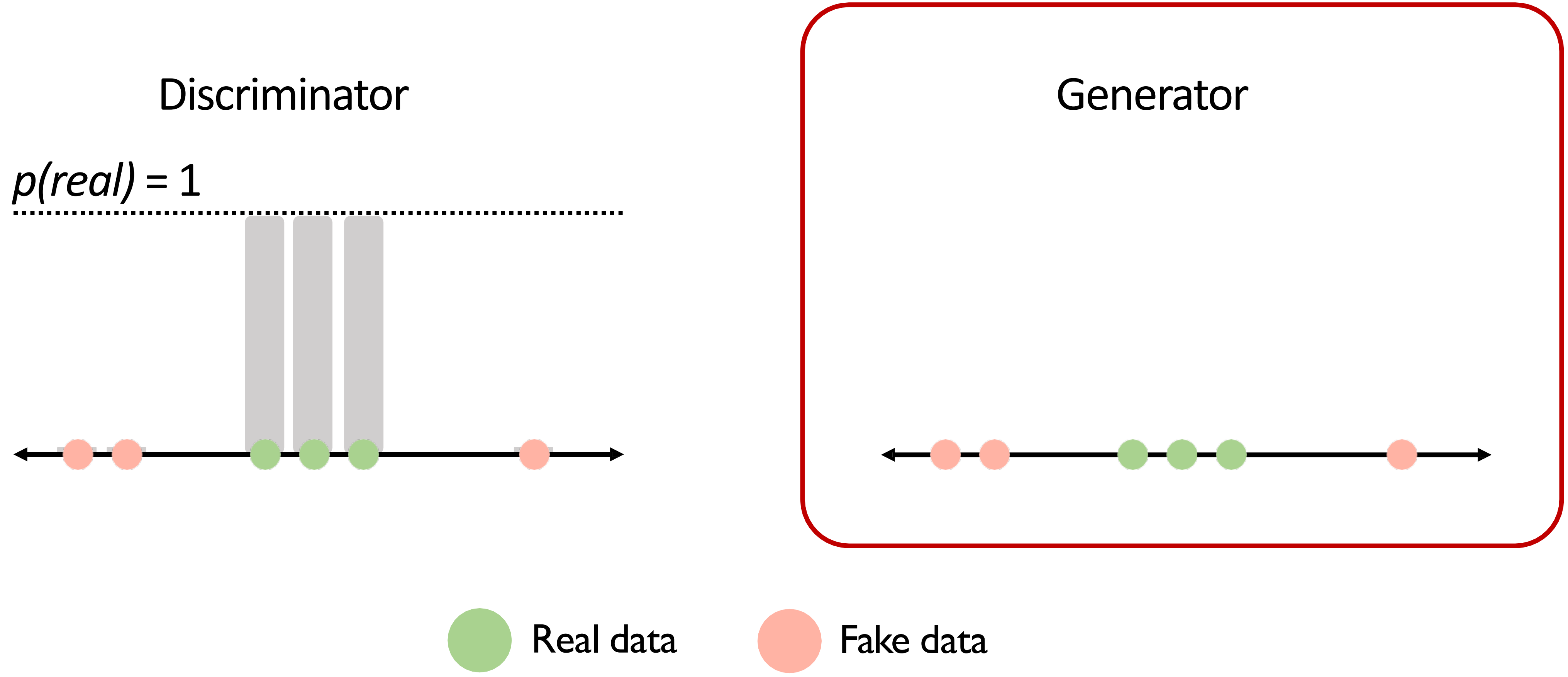
Intuition behind GANs

Discriminator tries to predict what's real and what's fake.



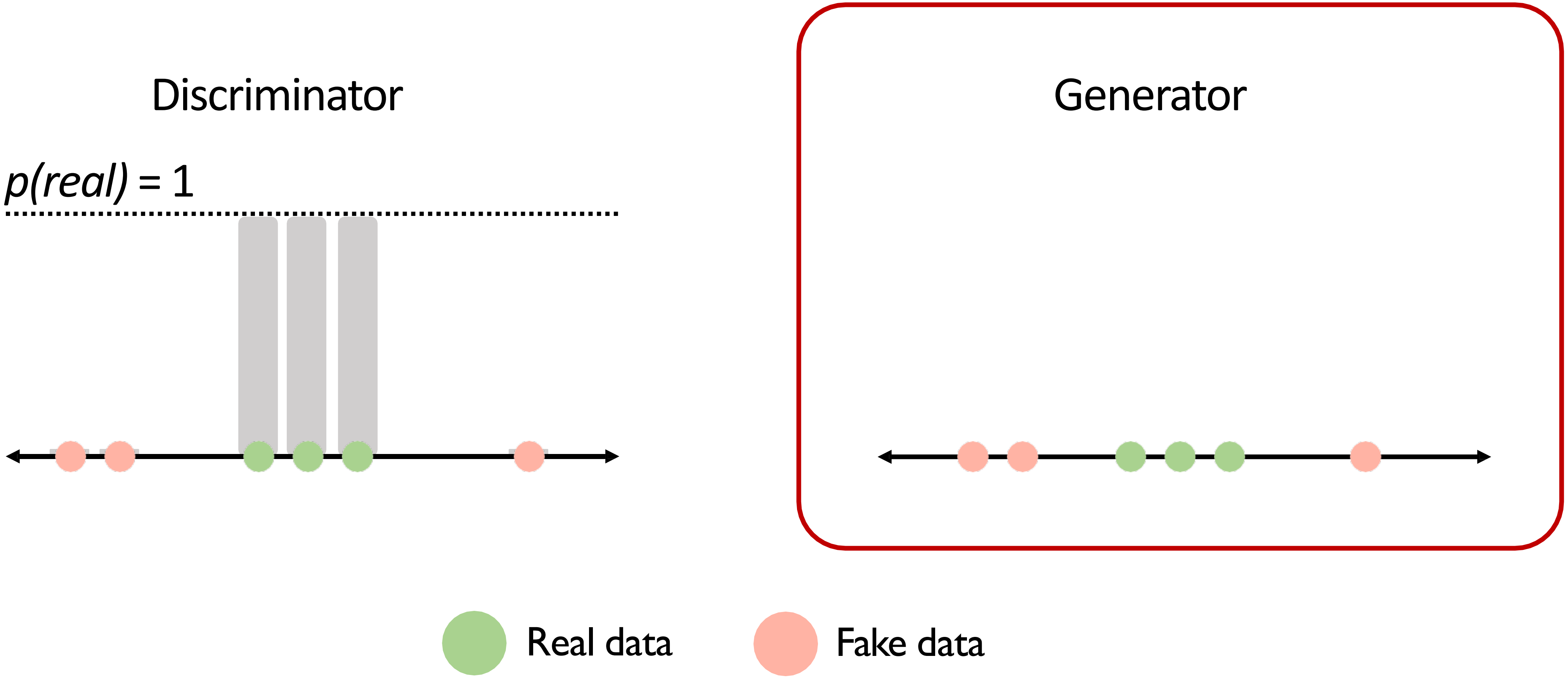
Intuition behind GANs

Generator tries to improve its imitation of the data.



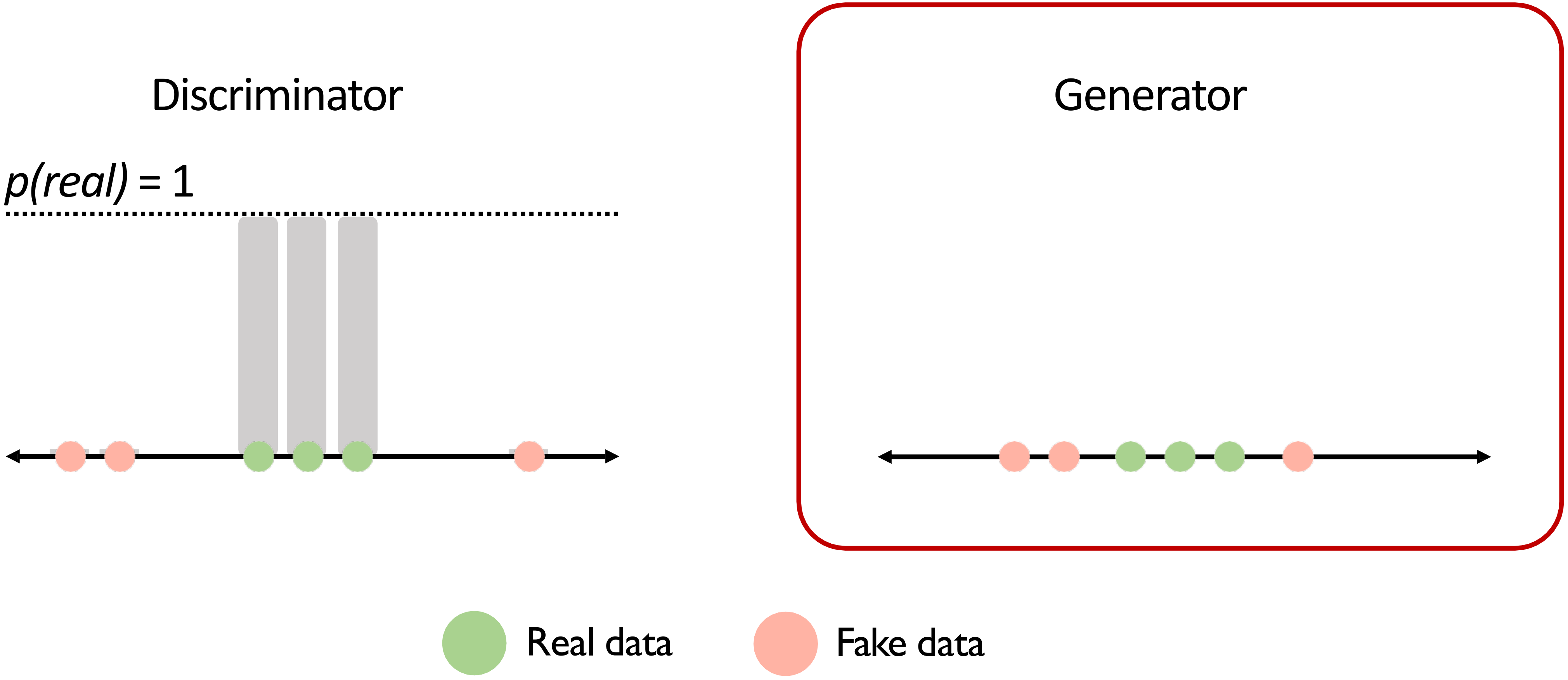
Intuition behind GANs

Generator tries to improve its imitation of the data.



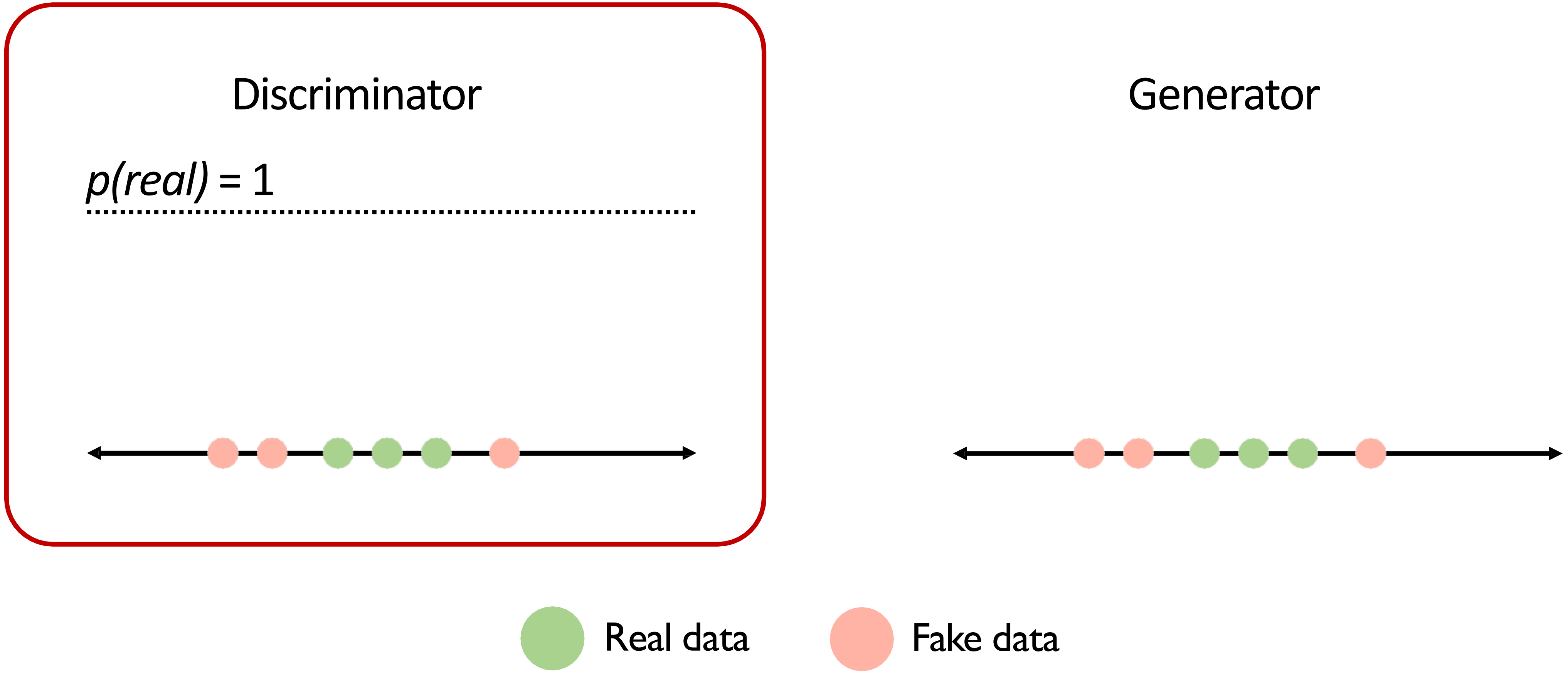
Intuition behind GANs

Generator tries to improve its imitation of the data.



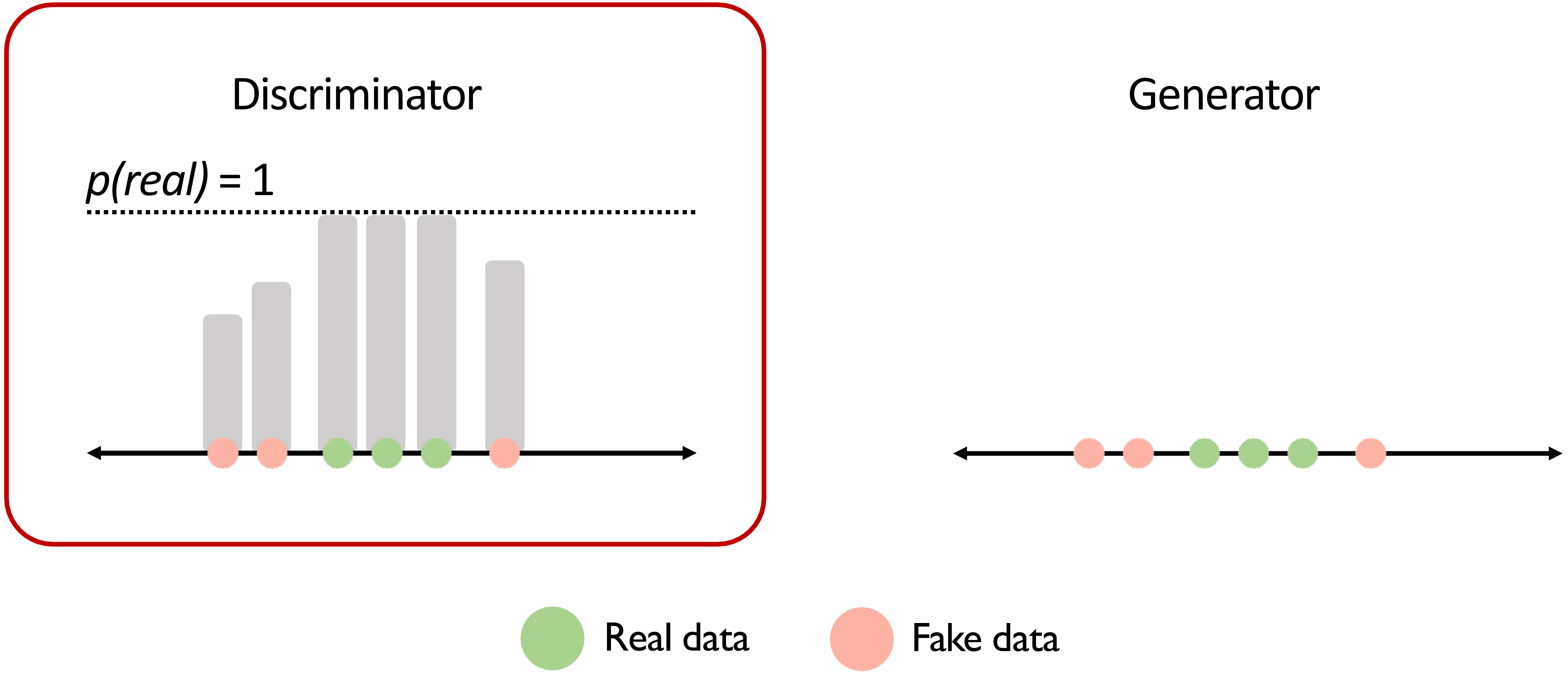
Intuition behind GANs

Discriminator tries to predict what's real and what's fake.



Intuition behind GANs

Discriminator tries to predict what's real and what's fake.



Intuition behind GANs

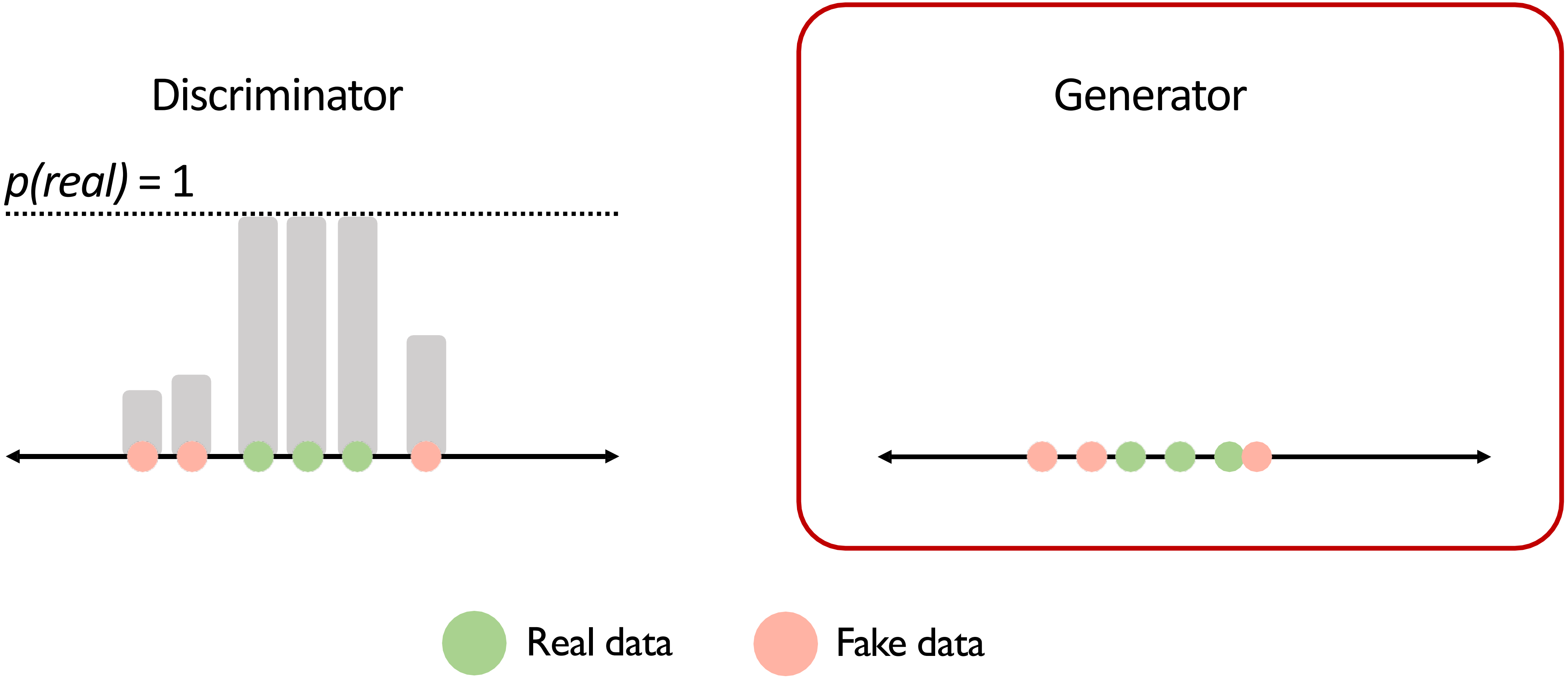
Discriminator tries to predict what's real and what's fake.

Intuition behind GANs

Discriminator tries to predict what's real and what's fake.

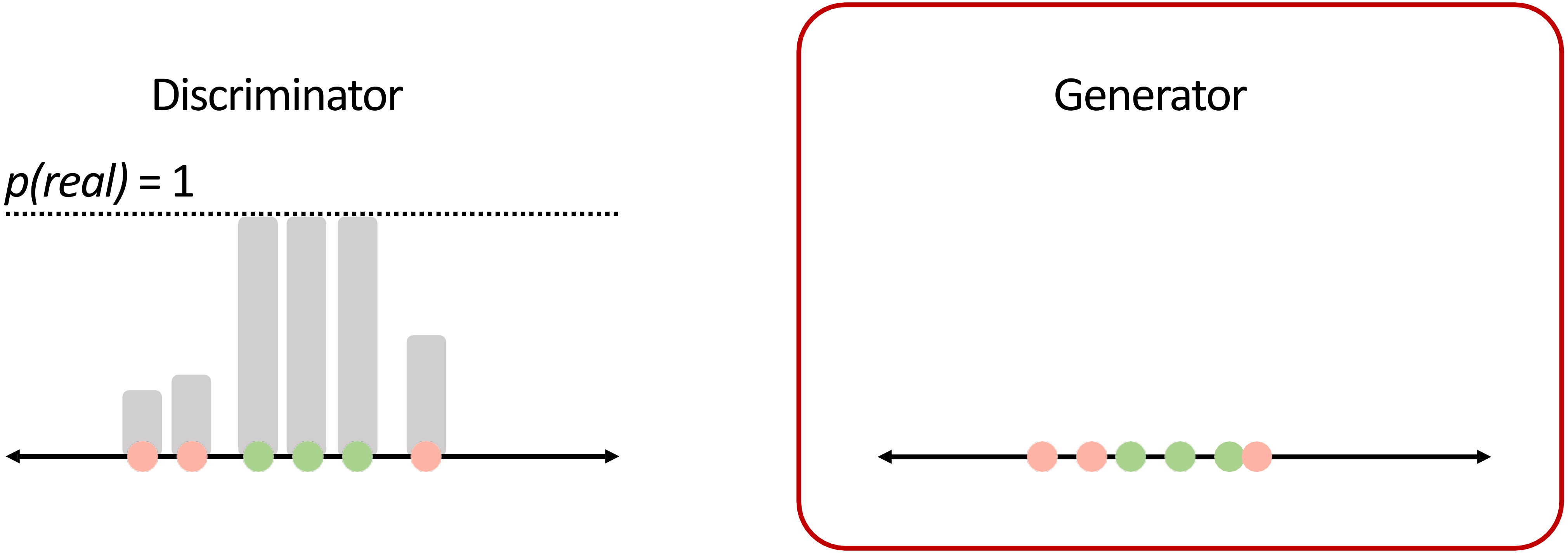
Intuition behind GANs

Generator tries to improve its imitation of the data.



Intuition behind GANs

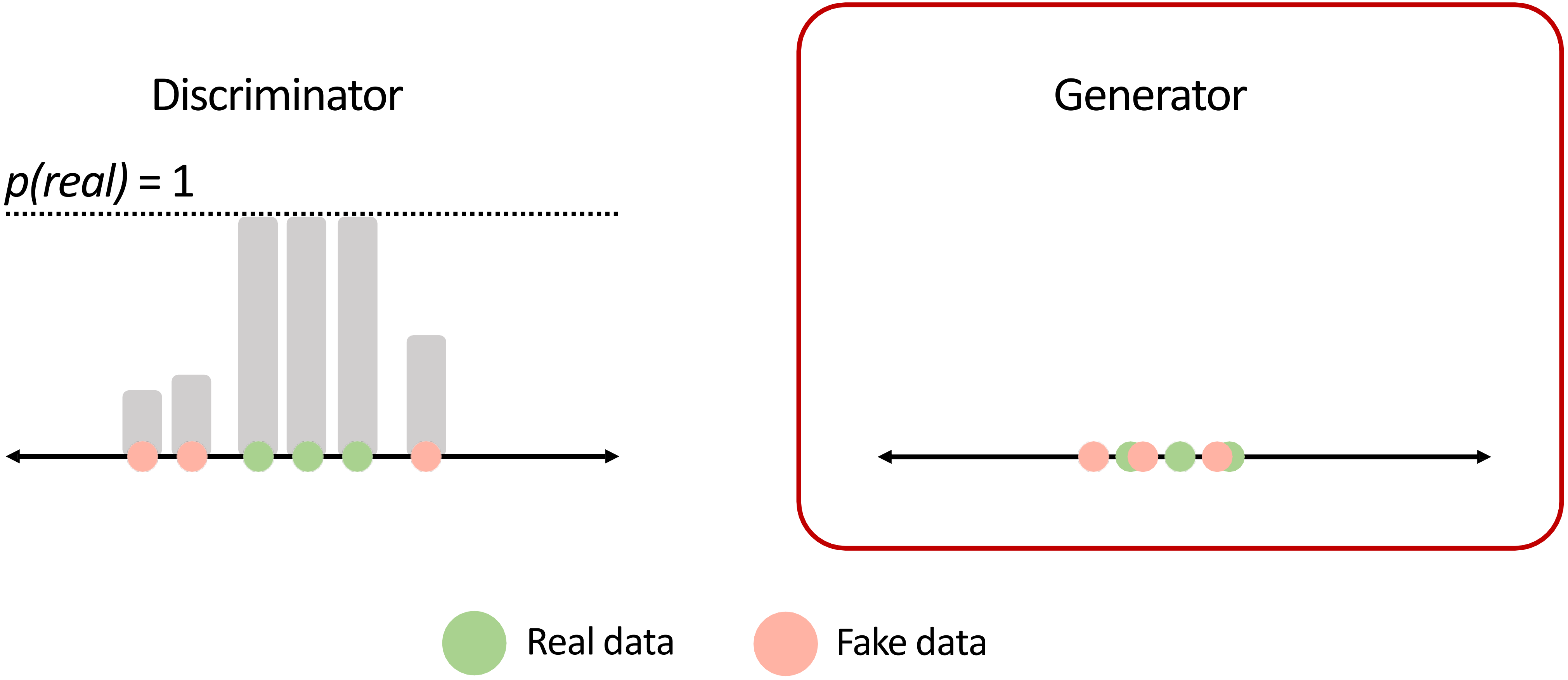
Generator tries to improve its imitation of the data.



Real data Fake data

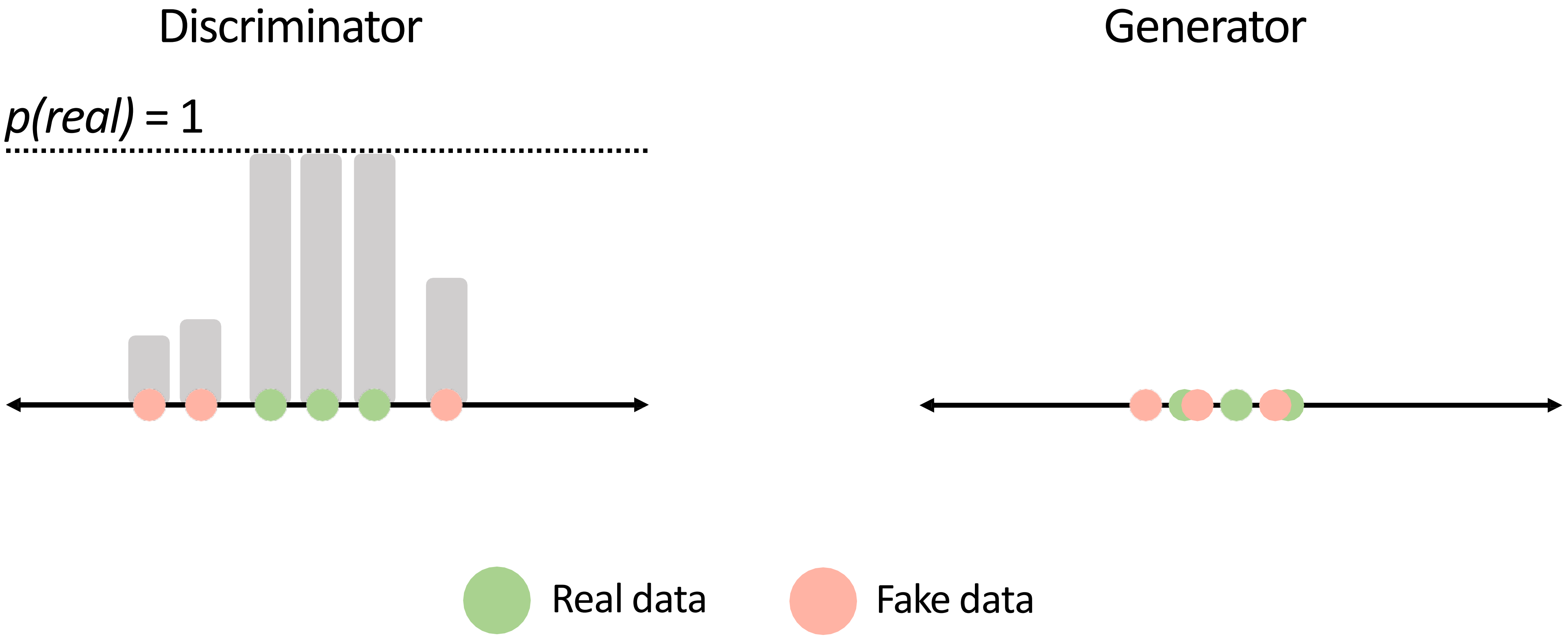
Intuition behind GANs

Generator tries to improve its imitation of the data.



Intuition behind GANs

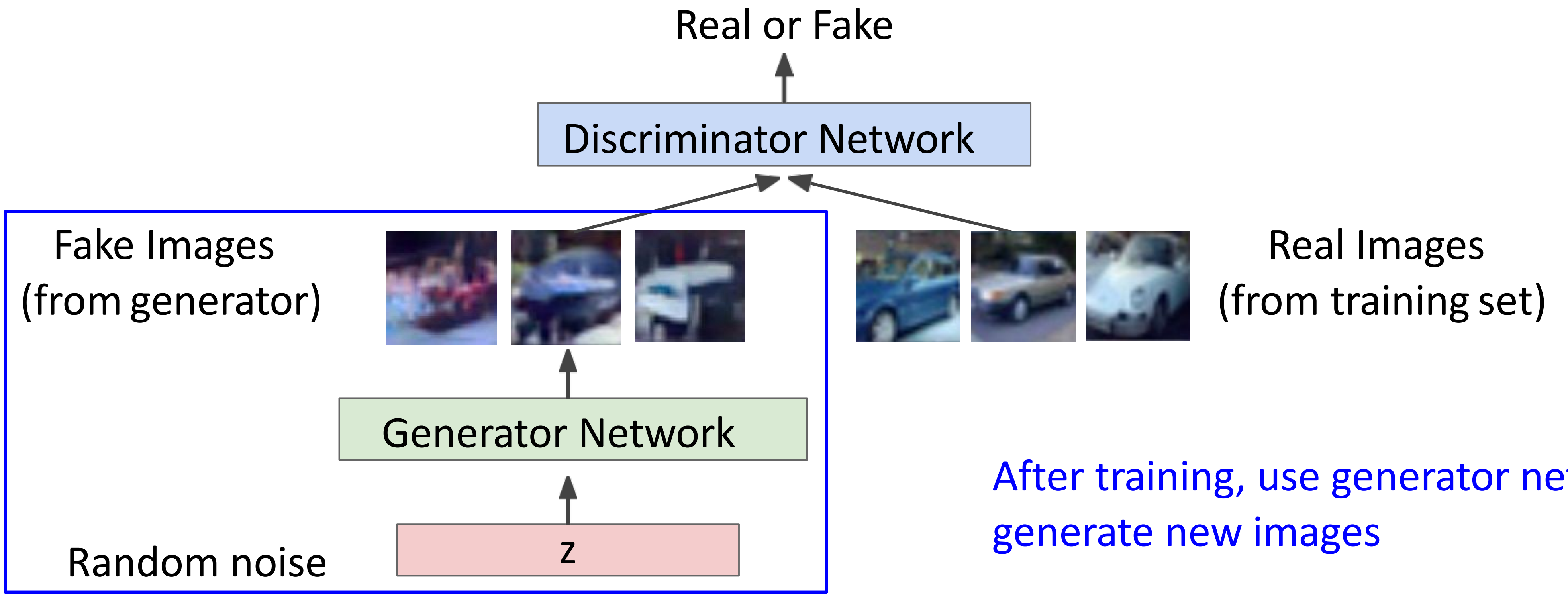
Discriminator tries to identify real data from fakes created by the generator.
Generator tries to create imitations of data to trick the discriminator.



Generative Adversarial Networks

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Generator network: try to fool the discriminator by generating real-looking images
Discriminator network: try to distinguish between real and fake images



After training, use generator network to generate new images

Generative Adversarial Nets: Convolutional Architectures

Samples from the model look much better!

Radford et al, ICLR 2016

Generative Adversarial Nets: Convolutional Architectures

Interpolating between random points in latent space

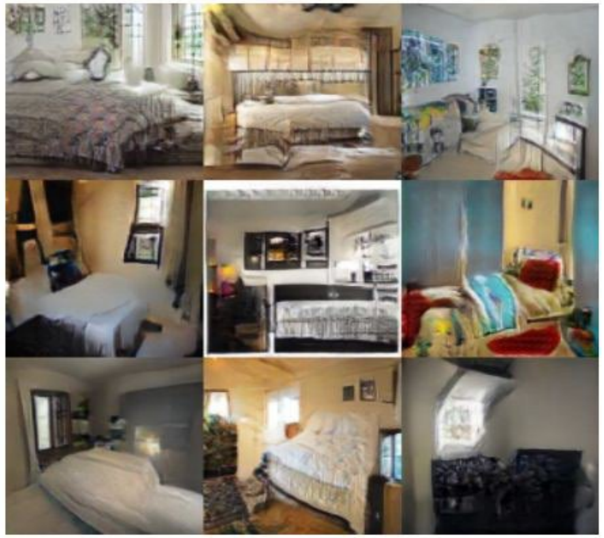
Radford et al, ICLR 2016

2017: Explosion of GANs

- GAN - Generative Adversarial Networks
- 3D-GAN - Learning a Probabilistic Latent Space of Object Shapes via 3D Generative-Adversarial Modeling
- acGAN - Face Aging With Conditional Generative Adversarial Networks
- AC-GAN - Conditional Image Synthesis With Auxiliary Classifier GANs
- AdaGAN - AdaGAN: Boosting Generative Models
- AEGAN - Learning Inverse Mapping by Autoencoder based Generative Adversarial Nets
- AffGAN - Amortised MAP Inference for Image Super-resolution
- AL-CGAN - Learning to Generate Images of Outdoor Scenes from Attributes and Semantic Layouts
- ALI - Adversarially Learned Inference
- AM-GAN - Generative Adversarial Nets with Labeled Data by Activation Maximization
- AnoGAN - Unsupervised Anomaly Detection with Generative Adversarial Networks to Guide Marker Discovery
- ArtGAN - ArtGAN: Artwork Synthesis with Conditional Categorial GANs
- b-GAN - b-GAN: Unified Framework of Generative Adversarial Networks
- Bayesian GAN - Deep and Hierarchical Implicit Models
- BEGAN - BEGAN: Boundary Equilibrium Generative Adversarial Networks
- BiGAN - Adversarial Feature Learning
- BS-GAN - Boundary-Seeking Generative Adversarial Networks
- CGAN - Conditional Generative Adversarial Nets
- CaloGAN - CaloGAN: Simulating 3D High Energy Particle Showers in Multi-Layer Electromagnetic Calorimeters with Generative Adversarial Networks
- CCGAN - Semi-Supervised Learning with Context-Conditional Generative Adversarial Networks
- CatGAN - Unsupervised and Semi-supervised Learning with Categorical Generative Adversarial Networks
- CoGAN - Coupled Generative Adversarial Networks
- Context-RNN-GAN - Contextual RNN-GANs for Abstract Reasoning Diagram Generation
- C-RNN-GAN - C-RNN-GAN: Continuous recurrent neural networks with adversarial training
- CS-GAN - Improving Neural Machine Translation with Conditional Sequence Generative Adversarial Nets
- CVAE-GAN - CVAE-GAN: Fine-Grained Image Generation through Asymmetric Training
- CycleGAN - Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks
- DTN - Unsupervised Cross-Domain Image Generation
- DCGAN - Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks
- DiscoGAN - Learning to Discover Cross-Domain Relations with Generative Adversarial Networks
- DR-GAN - Disentangled Representation Learning GAN for Pose-Invariant Face Recognition
- DualGAN - DualGAN: Unsupervised Dual Learning for Image-to-Image Translation
- EBGAN - Energy-based Generative Adversarial Network
- f-GAN - f-GAN: Training Generative Neural Samplers using Variational Divergence Minimization
- FF-GAN - Towards Large-Pose Face Frontalization in the Wild
- GAWWN - Learning What and Where to Draw
- GeneGAN - GeneGAN: Learning Object Transfiguration and Attribute Subspace from Unpaired Data
- Geometric GAN - Geometric GAN
- GoGAN - Gang of GANs: Generative Adversarial Networks with Maximum Margin Ranking
- GP-GAN - GP-GAN: Towards Realistic High-Resolution Image Blending
- IAN - Neural Photo Editing with Introspective Adversarial Networks
- iGAN - Generative Visual Manipulation on the Natural Image Manifold
- IcGAN - Invertible Conditional GANs for image editing
- ID-CGAN - Image De-raining Using a Conditional Generative Adversarial Network
- Improved GAN - Improved Techniques for Training GANs
- InfoGAN - InfoGAN: Interpretable Representation Learning by Information Maximizing Generative Adversarial Nets
- LAGAN - Learning Particle Physics by Example: Location-Aware Generative Adversarial Networks for Physics Synthesis
- LAPGAN - Deep Generative Image Models using a Laplacian Pyramid of Adversarial Networks

2017: Explosion of GANs

Better training and generation



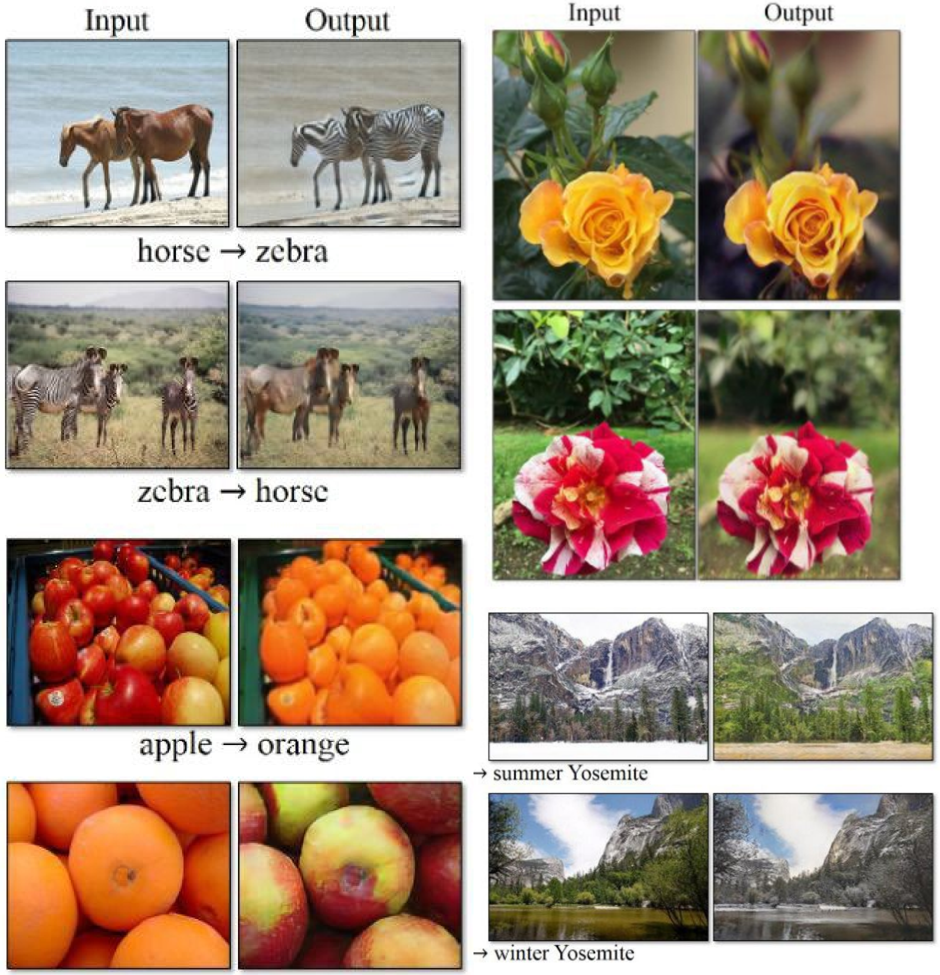
LSGAN, Zhu 2017.

Wasserstein GAN, Arjovsky 2017.
Improved Wasserstein GAN, Gulrajani 2017.

Progressive GAN, Karras 2018.

2017: Explosion of GANs

Source->Target domain transfer



CycleGAN. Zhu et al. 2017.

Text -> Image Synthesis

Reed et al. 2017.

Many GAN applications

Pix2pix. Isola 2017. Many examples at
https://phillipi.github.io/pix2pix/

Style-based generator: results

Karras et al.,Arxiv 2018.

2019: BigGAN

Brock et al., 2019

GANs

Don't work with an explicit density function
Take game-theoretic approach: learn to generate from training distribution through 2-player game

Pros:

- Beautiful, state-of-the-art samples!

Cons:

- Trickier / more unstable to train
- Can't solve inference queries such as p(x), p(z|x)

Active areas of research:

- Better loss functions, more stable training (Wasserstein GAN, LSGAN, many others)
- Conditional GANs, GANs for all kinds of applications

Tips and tricks for trainings GANs https://github.com/soumith/ganhacks
"The GAN Zoo" https://github.com/hindupuravinash/the-gan-zoo

NEXT

(e.g., images of dogs)

Differentiable?

VAEs, GANs

NEURAL RENDERING

PHYSICALLY BASED RENDERING

Data — Prior Knowledge

(physics, materials, ...)

Statistical modeling

Pros:
- Fully automated training
- Good performance (real-time, interactive)

Cons:
- Requires lots of training data
- No guaranteed intuitive or semantic control of results*

Computer graphics

Pros:
- Full control of scene parameters (cameras, geometry, lighting)
- Faithful representation of real world

Cons:
- Lots of manual work (scene modeling, animation)
- Long generation (render) times